Parsek — Logistics (Supply Routes) Design

Design specification for Parsek's stock-first automated cargo delivery system — covering Supply Run detection, Supply Route creation, dispatch scheduling, resource/inventory transfer between vessels, endpoint resolution, transfer window computation, the provenance → transfer → disposition target model with its flow-closure validity rule, and the milestone roadmap (M1-M6) to feature-complete logistics.

Parsek is a KSP1 mod for time-rewind mission recording. Players fly missions, commit recordings to a timeline, rewind to earlier points, and see previously recorded missions play back as ghost vessels alongside new ones. This document specifies how committed recording chains can be confirmed as Supply Runs, then turned into Supply Routes that repeat the stock resource and inventory transfers the player already performed.

Current to: Parsek 0.10.5. Milestones M1-M6 (section 19.4) all shipped by v0.10.3, and section 6 also records two later additions: the post-M6 all-or-nothing destination-capacity gate and the v0.10.5 `RouteHeld` hold rows. The v0.10.5 Logistics window redesign (Active / Paused / Candidates sections, the Interact grid, and the Route History window that replaced the old route step list and `Log (Mission)` button) is described in `docs/dev/design-gui-inventory.md` sections 3.6-3.7, not here. **Version:** 0.8 (merges the former standalone `parsek-logistics-provenance-and-roadmap.md` into section 19: the target gameplay model and the verified milestone sequence to feature-complete. 0.7: v0 implemented and shipped — a route is a looped Mission segment plus a delivery binding, superseding the bespoke chain-sequential playback of 0.5. See section 0, which is authoritative.) **Status:** IMPLEMENTED (v0) + ACCEPTED ROADMAP (section 19). Supply Routes v0 shipped in Parsek 0.10.0 (developed on the `logistics-v0-implementation` branch, PR #875, plus the run-cost-display, recovery-credit, and Logistics-window UI follow-ups). Phases 0-6 of `docs/dev/done/plans/plan-logistics-routes-on-missions.md` are built, tested (full xUnit suite green), and merged; the locked Missions subsystem is consumed read-only (its git diff is empty across the whole branch). Section 0 (the route-on-Missions architecture, including 0.9 dispatch cadence and 0.10 the code map) describes the implemented system and is authoritative for what exists. Section 19 (the atomic route model: provenance → transfer → disposition, flow closure as the single validity rule, and milestones M1-M6) is authoritative for where the feature is going; every implementation claim in it was verified against post-0.10.1 code (19.7). Sections 1-18 are the original standalone-route design, retained for the durable core (data model, delivery execution, endpoint resolution, edge cases) and for design rationale; where any of them conflicts with section 0 or section 19, those win. The v0 cut was deliberately narrow (docking-only, delivery-only, single-stop, same-body, KSC-origin routes), but milestones M1-M6 have all since shipped: non-KSC origin debit (M1), mod resources + harvest provenance (M2), pickup + mixed windows (M3), multi-stop / multi-origin / round-trip linking (M4a/b/c), inter-body re-aimed routes (M5), and legibility including map route lines (M6). The claw/grapple is the second connection producer (shipped 2026-07-07), after docking. Only stock crossfeed / fuel-line connection producers and crew delivery remain out of scope by doctrine (sections 17 and 19). Non-KSC origin routes SHIPPED with milestone M1 (0.10.2-dev, 2026-06-10): the v0 stub in `LiveRouteRuntimeEnvironment.OriginHasCargo` is un-stubbed, so docked-start non-KSC routes now gate all-or-nothing on the live origin vessel's stored cargo and physically debit it at each loop-route dock crossing (loaded and unloaded vessels). Two M1 scope notes: inventory-payload origin debit is deferred to M3 (a non-KSC route with a non-empty inventory cost manifest holds with reason `inventory-origin-debit-unsupported`, M1 plan decision D6), and the physical debit is loop-path-only (the legacy non-loop self-timer dispatch path keeps v0 behavior byte-identical, decision D11). See 19.4 M1 for per-item status. The Career funds debit for KSC origins is unchanged. Milestone M2 (resource generality + harvest provenance) SHIPPED 2026-06-12: any defined resource routes (modded included), every active-stop recording carries a full-run transport cargo manifest, witnessed harvest windows admit mining runs as routes (harvest-origin routes debit nothing), and the KSC funds charge for routes built from new recordings is launch-manifest based; see 19.4 M2 for per-item status and 19.2.4 for the funds-basis decision. Milestone M3 (direction generality: pickup) SHIPPED + in-game validated 2026-06-15 (M3a, branch `logistics-m3-direction`, PR #1160): a route can load a resource OR a stored part from a vessel onto the transport (the reverse of delivery) and a single dock can both deliver and pick up; pickup routes dispatch and physically debit the source once per cycle (resources + inventory, loaded or unloaded; idempotent across save/reload via the new `RouteCargoPickedUp` row + dispatch-keyed replay backstop), and docked-origin routes now debit stored-part inventory (the M1 `inventory-origin-debit-unsupported` carve-out is lifted). All 5 in-game `LogisticsPickupRuntimeTests` passed live. Multi-origin runs + cargo escrow (19.2.5) are resequenced to M4. See 19.4 M3 for per-item status. **Prerequisite:** Phase 10 (Location Context) is complete and Phase 11 provides the base vessel-level resource/inventory manifests. Logistics v0 also required the connection-scoped capture extensions listed in section 4.9, plus the actual-code alignment tasks in section 4.10; these are in place. **Out of scope:** Ghost playback engine behavior, broad recording/chain semantics, and game-actions recalculation architecture. Logistics does add route-proof recording metadata as an additive prerequisite; see `parsek-flight-recorder-design.md` and `parsek-game-actions-and-resources-recorder-design.md` for the underlying systems.

0. Architecture update (v0.10.0): Supply routes on the Missions foundation

This section is authoritative and supersedes the playback assumptions in the rest of this document. Sections 1-11 below describe the original v0.5 model in which a route owned a bespoke "chain-sequential" replay; that replay is replaced by the Missions subsystem (landed on main, v0.10.0). The route's data model, resource/funds/ledger, endpoint resolution, and proof capture are unchanged.

0.1 Why this changed

When this document was first written, no abstraction existed above raw recordings, so a route had to define its own ordered "source chain" and drive a bespoke replay. The Missions subsystem now provides exactly that layer: a Mission is a persisted, named selection over a recording tree (`Mission`, `MissionStore`); a segment of it is an interval selection (`MissionIntervalSelection`); and a Mission loops as one unit over a shared span clock in flight, the Space Center, and the Tracking Station (`MissionLoopUnitBuilder` → `GhostPlaybackLogic.LoopUnitSet` → the engine). A looped Mission is, by design, the precursor of a supply route (`design-mission-abstractions.md` section 6). So a route no longer invents replay; it reuses Missions.

0.2 Locked constraint: consume Missions, never modify it

The Missions subsystem (`Mission`, `MissionStore`, `MissionLoopUnitBuilder`, `MissionIntervalSelection`, `MissionStructure` / `MissionThroughLine` / `MissionComposition`, and the engine's `LoopUnit` / `LoopUnitSet` / `SetLoopUnits` machinery) is LOCKED. Logistics consumes its public components and must not change its types, its one-loop-per-tree rule, or the engine's single-owner index model. Every integration below is a new logistics-side consumer wired at the host seam, not an edit to a Missions file.

0.3 The route, restated in the Missions model

A Supply Route v0 is three things bound together:

1. **A backing Mission segment** — the run from launch (A) to the cargo delivery at the dock (B). Rendering STOPS at the docking moment (playtest follow-up): the docked-together combined vessel (the dock-merged child, spanning dock..undock) is NOT rendered. Expressed as a Mission over the source tree with an interval selection of `[launch .. dock]` (drop the dock-merged child + all post-dock peel intervals: `MissionIntervalSelection` end-trims a vessel by excluding its trailing interval keys). This is "a loopable mission in the backend."
2. **A delivery binding** — the dock / transfer / undock cargo proof (`RouteConnectionWindow`) that says WHAT moves and the dock UT at which it moves. Logistics-only; Missions carries no resource deltas. Unchanged from sections 4.9 / 5.2.
3. **A route lifecycle** — status, dispatch schedule, funds, endpoint resolution, ledger. Owned by the route (decision b, section 0.5). The durable core from sections 6-7 is kept.

Creation / verification is unchanged in spirit (section 5): a route is offered when a committed tree proves launch → dock → cargo delta → undock. What changes is that the proven run is expressed as a Mission segment rather than a bespoke chain, and it then "appears in the supply routes list as a loopable run."

0.4 Rendering: a route reuses the loop engine as a consumer

A route does NOT render anything itself (no bespoke replay; the discarded `logistics-route-replay` `OffsetReplay` is fully superseded). When a route is in a ghost-driving state, logistics builds a `LoopUnit` for its backing Mission segment through the EXISTING `MissionLoopUnitBuilder.Build`, and contributes it to the per-frame `LoopUnitSet` that the host already pushes to the engine (the `ParsekFlight.DriveMissionLoopUnits` seam, mirrored in `ParsekKSC` / `ParsekTrackingStation`). All consuming locked components:

- The route owns its backing Mission DEFINITION (tree id + `[launch..dock]` interval selection), held by logistics (in the route entity / `RouteStore`), NOT inserted into `MissionStore` — the store would prune / normalize it by tree and surface it as a player mission. The Mission object is constructed by logistics and handed to `MissionLoopUnitBuilder.Build` as a one-element list (the builder gates on `Mission.LoopPlayback`, so the route sets that on its own object). The single helper `RouteBackingMission` owns this geometry seam (the only place logistics reads the locked Missions composition pipeline): `ComputeExcludedIntervalKeys` derives the post-DOCK trim (the docked combined-vessel child and everything after the dock), `ComputeMemberRecordingIds` derives the rendered `[root..dock]` member-recording set (so a multi-recording flight widens `RecordingIds` / `SourceRefs` past the dock-child leaf and `RevalidateSources` tracks the whole rendered path), and `BuildMission` materializes the route-owned Mission on demand. The window START derives from the tree ROOT launch, NOT the mid-flight dock child; the END is the connection window's `DockUT` (rendering stops at the dock; the `segmentEndUT` param of `RouteBackingMission`). `RouteAnalysisEngine` / `RouteCandidateFinder` verify PROOF only and no longer scan trajectory geometry. The post-dock trim is primarily the `StartUT >= dock` boundary (the merged child starts exactly at the dock, a clean recording boundary); the terminal-undock-child collection remains as belt-and-suspenders for the post-undock survivor legs.

- **The backing-mission selection freezes to creation-time members (M-MIS-9).** A branch landing on the route's source tree AFTER creation without touching the member path (a re-fly fork rooted in the excluded post-undock subtree, or a switch-fly continuation merging onto the same committed tree) would otherwise silently join the synthesized backing mission, extending the loop-unit span and inflating the dispatch cadence. `RouteBackingMission.BuildMission` therefore unions an auto-excluded key set into the synthesized mission (`ComputeAutoExcludedNewIntervalKeys` , re-derived only when a tree-topology signature of counts plus rolling id hashes changes, cached on two non-serialized Route fields; no codec change). Two prongs: (1) base-id rule - every selectable composition key whose BASE recording id (the synthetic `/segN` marker stripped) was not known at creation, where known ids are `SourceRefs[].RecordingId` UNION the base ids of the creation-time `ExcludedIntervalKeys` ; (2) UT end-trim - composition keys are POSITIONAL, so growth that adds a peel edge on a member/excluded through-line renumbers/mints base-KNOWN tail keys past the creation-time index range; the creation-time trim is re-derived against the CURRENT tree from the persisted `RecordedDockUT` , excluding any selectable interval starting at/after the dock regardless of its key string. A pre-dock `/segN` re-peel of a known member recording stays included; a route with an EMPTY creation-time excluded set keeps the honest whole-segment fallback (no freeze); empty `SourceRefs` fails open (no freeze); a route without a persisted `RecordedDockUT` keeps base-id protection only (the UT prong skips, Verbose-logged - effectively empty population, since the dock UT and the excluded keys shipped in the same codec change). Member-path branches remain covered by `RouteStore.RevalidateSources` as before - and the freeze RELIES on that: pre-dock renumbering of the member path (which could make a stale creation-time excluded key string denote a pre-dock member window) is unreachable because any pre-dock member mutation changes a SourceRef fingerprint and flips the route off ghost-driving first. Known residual: the per-cycle recovery-credit sum is whole-tree by contract (gotcha G1) and is NOT covered by this freeze - tracked as todo M-MIS-9-R1.
- At the host push seam, the route's unit(s) are unioned into the same `LoopUnitSet` the player missions produce, before the single `SetLoopUnits` call. Union is on disjoint committed indices only (`LoopUnitSet.OwnerByIndex` is single-valued — one owner per recording index).
- Delivery fires when that loop clock crosses the recorded dock UT each cycle, reusing the existing delivery / funds / ledger path (sections 6.3-6.5). The dock UT is a phase within the backing Mission's span.

0.5 Two clocks: route lifecycle is separate from manual mission looping (decision b)

"Looping a mission" (the Missions-window checkbox — a visual repeat, no delivery, no cost, player-controlled) and "running a supply route" (delivery + funds + dispatch + status) are SEPARATE features. The route's runtime does not depend on the player's manual mission-loop checkbox: it loops via its own backing Mission, not the player's toggle. (The two are kept mutually exclusive per tree by the guard in section 0.6 — setting a route disables the manual toggle — but that is a UX guard, not a runtime dependency.) The route parameterizes its backing Mission's loop (period = dispatch interval, phase anchor = dispatch epoch) from its own schedule, and the loop renders those parameters. So there are two conceptual clocks — the route's dispatch schedule driving the backing Mission's loop — with the route as the driver.

0.6 Render parallelism: what works, what is deferred

- **Across different trees (the common case):** a route on tree X and a manual mission loop on tree Y both render at once. Their committed indices are disjoint, so their units never collide. Fully supported.
- **Same tree, same recordings — mutual exclusion (the rule):** the engine keys each committed recording index to exactly ONE loop owner (`LoopUnitSet.OwnerByIndex` is single-valued) and `MissionStore` enforces one loop per tree, so a route loop and a manual loop on the SAME tree cannot both render as two ghosts. Rather than resolve this at render time, v0 forbids the state outright: **a tree is either a supply route or a manually looped mission, never both.** Setting / activating a supply route for a tree turns OFF and disables (greys) the manual Loop toggle on that tree's mission rows; while a route is set on the tree, manual looping cannot be turned on; clearing the route re-enables the toggle. So the collision can never arise. This is also the correct visual (one looping ghost of a run, not two identical overlapping ones).
- **True parallel rendering of one recording on two independent clocks is DEFERRED.** It would require a multi-owner engine redesign, which is forbidden here. This matches the deferral already recorded in `design-mission-abstractions.md` open question 7 ("True multi-Mission looping of the SAME recording ... deferred to logistics").

0.7 What this supersedes, keeps, discards

- **Supersedes (replaced by Missions):** the bespoke "chain-sequential ghost playback" of sections 1.2 and 3 (Terminology: "Supply Route ... Uses chain-sequential ghost playback (not the per-recording loop system)") and any route-owned visual replay. The route's visual is now a Mission loop.
- **Keeps (durable core, unchanged):** the data model and serialization (section 4); route-proof recording metadata + capture (4.9, 5.2); delivery execution, per-resource independent fill, and the NO_FLOW / flow-state gate (6.3-6.5); endpoint resolution for the live DELIVERY target (section 7 — distinct from rendering; Missions does not resolve delivery targets); funds; the ledger Route module; and the dispatch-eligibility checks (6.1).
- **Discards:** the separate `logistics-route-replay` branch (`OffsetReplayUnit` / `RouteReplayPlanner`) — fully superseded by `MissionLoopUnitBuilder` + the two-cadence `LoopUnit` . Confirmed verified-absent in this line's `Source/` (grep + the

`RouteReplayBranchAbsentTests` reflection guard, which fails the build if any such type reappears). Agents do not delete branches; the user prunes the `logistics-route-replay` branch.

0.8 v0 scope (keep it simple)

- v0 supply routes are **launch → dock only** (one outbound run ending at the DOCK / delivery moment; the docked period AND the post-undock return tail are not part of the route segment). Rendering stops at the dock, which here is a clean recording boundary (the transport's solo recording ends at the couple; the merged combined-vessel child starts there), so trimming to the dock does NOT hit the deferred "isolate the docked stretch within one recording" gap. "Undock → undock" (a shuttle already in orbit cycling between stations) SHIPPED with M-MIS-5 P2b (2026-07-07): a run whose tree roots mid-flight but begins at a recorded docked-origin window is accepted, with the route spanning [origin undock .. delivery dock] and the depot behind the origin window as the origin. Isolating a mid-recording docked stretch with no undock edge stays deferred (the parent plan's PARKED shape).
- **Same-body routes only for v0** (rover-to-base, tanker-to-LKO-station). The delivery clock builds the backing Mission with the SAME `bodyInfo` the render uses (DEL-1), so a same-body ORBITAL route phase-locks to the launch-pad rotation window exactly like a looped Mission (delivery fires on the relaunch the player sees), while a suborbital / atmospheric route stays uncompressed. Interplanetary re-aim / synodic windows exist in `MissionLoopUnitBuilder` (the `bodyInfo` seam) and stay deferred.

0.9 Dispatch cadence (Phase 6)

A route's effective dispatch cadence is `N x` the backing mission's NATURAL relaunch period, where that period is whatever the Missions layer already computes for the run: the span (suborbital same-body), the launch-pad-rotation window (orbital same-body), or the synodic transfer window (inter-body, via `MissionPeriodicity` / re-aim). `N` (`Route.CadenceMultiplier`) is an integer `>= 1`.

- **Default `N = 1` is the FLOOR / minimum loop time.** The route dispatches as fast as the run allows; it cannot go faster (the Phase 5 `DispatchInterval >= span` clamp and the Missions periodicity already enforce the floor). The player raises `N` in the Logistics window's Every stepper (Advanced mode; Basic shows the interval read-only) to launch LESS often (`2x`, `3x`, ...). The stepper shows both the multiplier and the resulting human cadence (e.g. `1x (~14m)` / `2x (~28m)`); `1x` is annotated as the minimum.

- **v0 same-body:** `DispatchInterval = N x TransitDuration` (derived in `RouteBuilder` at creation, recomputed by `RouteCadence.ApplyMultiplier` when `N` changes); `CadenceMultiplier` `N` is the player handle. For a suborbital / atmospheric route the loop clock's cadence IS that `DispatchInterval`. For an ORBITAL same-body route the backing unit (built with `bodyInfo = FlightGlobalsBodyInfo.Instance`, matching render) phase-locks: `PhaseAnchorUT` snaps to the launch-pad rotation window and `CadenceSeconds` quantizes up to a multiple of the rotation period, so the delivery clock and the rendered ghost share one cadence (DEL-1). Delivery fires when the loop clock reaches the recorded dock phase, not at cycle start (DEL-2).

- **Synodic-via-Missions reuse (the seam; ACTIVATED by M5):** `RouteLoopClock` threads the backing-mission `LoopUnit`'s OWN relaunch schedule (`RelaunchSchedule`) + loiter cuts (`LoiterCuts`) into `GhostPlaybackLogic.TryComputeSpanLoopUT` instead of hardcoding `null`. For a v0 same-body route `RelaunchSchedule` / `LoiterCuts` are `null`, so the schedule passthrough itself is a no-op; same-body cadence alignment comes from the `bodyInfo` phase-lock above (DEL-1), not from this passthrough. When an inter-body route is later enabled, the backing Mission's unit carries the Missions-layer synodic / re-aim schedule, and delivery then fires on the SAME re-aimed launch UTs the ghost renders on — a route backed by a looped Mission inherits `MissionPeriodicity` / re-aim scheduling for free through this passthrough. M5 (BUILT) activates the seam: an inter-body route dispatches on the same rendered windows, with `N` applied as a RESIDUAL modulo on the window index (OFF for a flat route; 1 for a zero-drift scheduled route, whose minSpacing throttle already consumes `N` Missions-side; `N` for re-aim synodic windows, delivering every `N`th rendered window - see 19.4 M5). The `LoopUnit` schedule fields are consumed READ-ONLY (no locked Missions / engine file is edited).

- **Phase-anchor determinism caveat (Rec-5; see report risk #11).** A route's effective dispatch grid (`PhaseAnchorUT`) is fully recorded-deterministic for a CREATE-ACTIVE route — `LoopAnchorUT` is seeded from the recording's `StartUT` and the loop builder floors the anchor to the recorded `spanEnd`, so the grid is pinned to a fixed recorded UT and is reproduced exactly across a rewind. A PAUSE→ACTIVATE route instead pins `PhaseAnchorUT` to the LIVE activation UT (`RouteOrchestrator.cs:205`, floored by `MissionLoopUnitBuilder`'s `Math.Max(anchor, spanEnd)`), so its post-rewind determinism is CONDITIONAL on the re-fly reproducing the activation event at the same UT. `LoopAnchorUT` round-trips the `.sfs` ROUTES node, so a rewind that does not re-pause/re-activate keeps the original anchor and is deterministic; a divergent re-fly that re-activates at a different UT shifts the grid. This is a determinism nuance only (not a paradox) and does not affect the Rec-1 ledger-row fix.

0.10 Implemented code map (v0)

v0 (Phases 0-6) is built. The route-on-Missions system lives under `Source/Parsek/Logistics/` and consumes the locked Missions subsystem read-only. The seam files added or reshaped for the re-founding:

- `RouteStatusPolicy` (Phase 0): single source of truth over all 9 `RouteStatus` values. `BindsTree` (does the route own the tree's loop) and `GhostDriving` (does the route contribute a `LoopUnit` this frame). Exhaustive switch, throwing default, so a new status cannot silently fall through.
- `RouteBackingMission` (Phase 1): the only place logistics reads the locked Missions composition pipeline. `ComputeExcludedIntervalKeys` derives the post-undock trim, `ComputeMemberRecordingIds` derives the rendered `[root..undock]` member set, `BuildMission` materializes the route-owned Mission on demand for `MissionLoopUnitBuilder.Build` (never inserted into `MissionStore`).
- `RouteTreeGuard` (Phase 2): `IsTreeBoundToActiveRoute` + `ForceClearManualLoopForRouteTree`. Backs the mutual-exclusion guard (section 0.6) that greys the manual Loop toggle on both the Missions tab and the Recordings tab (per-recording / group / chain / bulk sites).
- `RouteGhostDriverSelector` (Phase 3): `SelectGhostDrivingBackingMissions`. The host unions these into the existing `LoopUnitSet` before its single `SetLoopUnits` push, at the three seams `ParsekFlight` / `ParsekKSC` / `ParsekTrackingStation` (`DriveMissionLoopUnits`).
- `RouteOrchestrator` (Phase 4): delivery on the loop clock. `EmitLoopCycle` fires the FULL cycle (origin/funds debit via the extracted dispatch body + delivery, one `cycleId`) when the clock crosses the recorded dock UT, reusing the section 6.3-6.5 path. On the ERS/ELS audit allowlist for the member-index alignment read.
- `RouteLoopClock` (Phase 4 + 6): wraps `GhostPlaybackLogic.TryComputeSpanLoopUT` as the per-cycle crossing predicate, and (Phase 6) threads the backing unit's own `RelaunchSchedule` + `LoiterCuts` into it (the section 0.9 synodic passthrough seam; a no-op for v0 same-body).
- `RouteBuilder` (Phase 5) + `RouteCreationDialog`: creation from a proven tree. Clamps `DispatchInterval >= span` and widens the member set to `[root..undock]`. The dialog defaults the interval to the full root-to-undock span.
- `Route.CadenceMultiplier` + `RouteCadence` + `RouteCodec` (Phase 6): the cadence handle `N` (clamped `>= 1`), its sparse serialization (omitted when 1), and `DispatchInterval = N x span`. The cadence stepper lives in `LogisticsWindowUI` (section 0.9).

Coverage: roughly 85 phase-affected xUnit cases plus the full suite green (13,513 passing, 0 failing); in-game coverage in `Source/Parsek/InGameTests/LogisticsRouteOnMissionsRuntimeTests.cs` (live delivery at the dock, cadence stepper) and `LogisticsRouteTreeGuardRuntimeTests.cs` (the mutual-exclusion guard). The locked-Missions git diff is empty across the whole branch and the ERS/ELS grep gate is green.

---

1. Introduction

> **Legacy section.** Sections 1-18 are the original standalone-route design. They are kept for the durable core (data model in section 4, delivery in 6.3-6.5, endpoint resolution in section 7, edge cases in section 10) and for rationale. For how a route actually renders and clocks in the shipped v0, section 0 is authoritative: a route is a looped Mission segment, not a bespoke chain-sequential replay. Read "chain-sequential ... replays the source chain" in this section as the superseded v0.5 model (see section 0.7).

This document specifies how Parsek turns a player-flown Supply Run into an automated Supply Route. A Supply Route is chain-sequential: the player flies a cargo mission, docks to a destination, uses stock KSP transfer systems, undocks, commits the recording, confirms that the run should become a route, and Parsek replays the source chain on each dispatch cycle while repeating the recorded cargo transfer.

- Logistics window / Supply Routes vocabulary
- Supply Run detection and route analysis
- How stops and delivery manifests are derived from docking windows in the committed chain
- Chain-sequential playback: source recordings play in order, delivery between recordings, restart after dispatch interval
- The dispatch cycle: check timing, check capacity, charge or deduct origin cargo, deliver
- Endpoint resolution: surface proximity fallback vs orbital PID matching
- Transfer window scheduling via synodic period computation
- Future round-trip linking for paired one-way routes
- Cargo modification on unloaded vessels (resources via ProtoPartResourceSnapshot, inventory via STOREDPARTS ConfigNodes)
- Timeline integration with explicit route ledger modules and epoch isolation for revert safety
- Module architecture: self-contained `Logistics/` directory with 4 integration seams
- Edge cases: destruction, full tanks, competing routes, time warp, reverts

1.1 What happens when the player creates a route

The player flies a mission normally — drive a rover with fuel to a base, launch a tanker to an orbital station, or send a cargo lander to a depot. During the flight, the transport docks to the destination, transfers cargo through stock KSP systems, then undocks. Docking ports and the claw/grapple (`ModuleGrappleNode`) are the two shipped connection producers. The route model names the broader concept "stock connection window" so stock crossfeed/fuel-line support could be added later without replacing the route shape.

After the recording commits, Parsek analyzes the committed chain. If it finds a complete docking connection window with a delivery resource or inventory delta, the merge/recordings UI prompts: **"Create Supply Route from this Supply Run?"** The player sees a route summary (origin, endpoint, delivery manifest, KSC dispatch cost if any, transit time), sets or accepts the dispatch interval, and confirms. The Supply Route goes live.

No explicit **Record Supply Run** button is required for v1. Reducing actions is the better default: if the player docked, moved cargo, undocked, and committed, Parsek should assume route creation is likely and ask. A later helper button may still mark intent or suppress unrelated prompts, but it must not be required for correctness.

1.2 What happens each cycle

On schedule, the route scheduler begins a chain-sequential cycle. It tells the ghost playback engine to play the first source recording when visuals are available, but route state advances by UT. When the recorded delivery boundary UT is reached, delivery happens at the endpoint. The scheduler continues through the remaining source recordings, then waits for the dispatch interval before starting the next cycle.

Before each cycle, the route evaluates whether dispatch is possible. It checks the destination (is there room?), the origin (are there enough resources or inventory items, unless it is KSC), the KSC funds cost in Career, and whether the transit window is open. If everything checks out, origin cargo is deducted or the KSC dispatch funds cost is charged. The ghost replays the recorded chain visually during transit.

1.3 What the player sees

Situation	What happens
Eligible Supply Run committed	Parsek offers to create a Supply Route from the run.
Supply Route confirmed	Route analysis extracts the endpoint and cargo manifest from the committed chain. Player sees summary, sets interval, confirms.
Route dispatches on schedule	Proven non-KSC origin cargo is deducted, or KSC dispatch cost is charged in Career. Ghost begins chain-sequential replay.
Route delivers	Cargo (resources, inventory) appears at the resolved endpoint vessel as the ghost reaches the recorded transfer point.
Destination tanks full	Dispatch waits on the same scheduled cycle. Origin NOT deducted, nothing delivered. The ghost still flies the route (the world looks busy) but transfers/charges nothing; the route status names the blocking reason.
Origin runs out of resources	Dispatch delayed until resources available. Route resumes automatically.
Destination destroyed (surface)	Nearest-compatible vessel fallback can reconnect to a rebuilt base at the same location.
Destination destroyed (orbital)	Route halted (`EndpointLost`). Player must re-target to new station.
Source recording missing	Route halted (`MissingSourceRecording`). No cargo transfers without the proof recording.
Source recording deleted during transit	Current cycle aborts before delivery and route becomes `MissingSourceRecording`.

Situation	What happens
Source recording rewritten/superseded	Route halted (`SourceChanged`). Player must recreate or explicitly reanalyze the route from the current proof data.
Player reverts past a dispatch	Epoch isolation invalidates route events. Stock quicksave/load restores stock vessels; Parsek timeline rewind uses route ledger modules to reverse or mask route effects before live cargo mutation is allowed. A rewind refunds the later committed cycles instead of retiring their rows (owner rulings 2026-10-21, section 10.6).
Two routes linked as round-trip	Linked from a route's detail panel: they alternate, Route A completes → Route B dispatches → B completes → A dispatches.

The Logistics window itself (sections, columns, the Interact grid, the detail block, the Route History window, Basic vs Advanced) is described in `docs/dev/design-gui-inventory.md` section 3.6.

1.4 Example: fuel delivery rover

```
SUPPLY RUN:
  Phase 1: Rover departs KSC runway with 200 LF
  Phase 2: Rover docks at base (connection EndResources: 195 LF)
  Phase 3: Player transfers 150 LF to base (rover: 45 LF, base: +150 LF)
  Phase 4: Rover undocks and drives clear
  Player commits recording; Parsek offers "Create Supply Route?"

ROUTE ANALYSIS PRODUCES:
  Origin:     KSC runway area (Career dispatch cost = stock part cost + used/delivered cargo cost)
  Endpoint:   base location (from stock connection target PID + connection coordinates)
  Delivery:   150 LF per cycle (rover loses 150 LF; base gains 150 LF and retains it through undock)
  Transit:    chain duration (~10 min)
  Interval:   player-set (minimum = chain duration)

EACH CYCLE (chain-sequential):
  UT=0:     Dispatch. Career funds charged if applicable. Ghost rover starts the first source recording.
  UT=stop:  Recorded delivery boundary reached. 150 LF added to base, clamped to tank capacity.
            Ghost continues through the rest of the source chain.
  UT=total: Chain complete. Cycle done. Wait for dispatch interval.
            If base full at dispatch check -> dispatch waits; no LF added, origin not deducted.
```

1.5 Example: Minmus mining supply chain

```
ROUTE A: KSC -> Minmus Base (fuel supply)
  Origin:     KSC (Career dispatch cost charged)
  Delivery:   800 LF + 978 Ox per cycle
  Result:     mining base stays fueled for drill + ISRU operation

ROUTE B: Minmus Base -> Kerbin Depot (ore delivery)
  Origin:     Minmus base (non-KSC -- Supply Run starts docked to the base depot)
  Cost:       1200 Ore + 600 LF + 733 Ox per cycle (used/delivered manifest)
  Delivery:   1200 Ore per cycle
  Gate:       dispatch delayed until base has all required resources

CHAIN BEHAVIOR:
  Route A delivers fuel -> base mines ore -> Route B ships ore to depot.
  If Route A stops -> base runs dry -> Route B pauses indefinitely.
  If depot tanks full -> Route B waits on its current scheduled cycle, base accumulates ore.
```

---

2. Design Philosophy

These principles govern every design decision in the logistics system. They are listed here because they inform every section that follows.

2.1 Realism and fidelity

1. **Fly it once, automate it forever.** Routes replicate exactly what the player did during the recording. The delivery amount, transit duration, and fuel cost are all derived from the real flight — not configured abstractly.

2. **Transit takes real time.** The route duration equals the recording duration. A 3-year Eeloo transfer takes 3 years per cycle. A 10-minute rover drive takes 10 minutes. No shortcuts.

3. **Transfer windows are respected.** Inter-body routes dispatch at the synodic period of the origin and destination bodies, phase-anchored to the original Supply Run start UT. Same-body routes can dispatch at any time but not faster than the original recording.

2.2 Resource safety

4. **Stock proof-of-work.** Parsek may automate stock actions the player already performed, but it must not invent storage, cargo, transfer rules, crew, or production rules. A Supply Route exists because a committed Supply Run proves the transport, path, stock connection, cargo delta, and disconnect. In v1 that proof is specifically dock, deliver, and undock.

5. **No infinite cargo glitches.** Routes deliver exactly what was transferred during the recording — no more, no less. KSC origins are not free in Career: each dispatch charges stock-realistic funds for the source vessel parts plus the resource/inventory quantities used or delivered by the Supply Run. Non-KSC origins deduct the resource/inventory quantities used or delivered from a real origin vessel, but v1 only allows that when the run starts docked to that origin depot and records its PID. Recovery funds from the original flown vessel are one-time unless a later round-trip design explicitly models repeat recovery.

6. **Don't waste origin cargo.** Origin is only deducted if at least one delivery item can be accepted at the destination. If destination is completely full, the dispatch waits on the same scheduled cycle and origin pays nothing. Per-item delivery is independent: each item fills what fits, and shortfalls are logged and shown instead of silently disappearing.

7. **Dock + deliver + undock required in v1.** A route can only be created from a recording chain where the transport forms a detected docking connection to an endpoint, delivery cargo moves from the transport into the endpoint while docked, the endpoint retains that cargo through undock, and the transport undocks afterward. Claw/grapple (`ModuleGrappleNode`) SHIPPED as the second connection producer; only stock crossfeed/fuel-line paths remain out of scope by doctrine (continuous-flow, no couple event).

2.3 Abstraction model

8. **No physical vessel during transit.** Route execution is pure math — deduct at origin, wait, add at destination. The ghost is visual only. No physics, no collisions, no orbit propagation.

9. **Endpoints are vessels first, locations second.** Endpoint PID is the primary identity. Surface endpoints may fall back to a single nearest compatible vessel near the recorded coordinates, but Parsek does not create an abstract area warehouse. Orbital endpoints use PID only. This keeps delivery close to stock vessel semantics while still tolerating surface base rebuilds.

10. **The system doesn't produce cargo.** Parsek moves resources and inventory items between stock vessels. Mining, ISRU, solar power, manufacturing, and crew hiring are the player's responsibility. Routes chain naturally: the output of one feeds the input of another.

2.4 Timeline integration

11. **Dispatches and deliveries are ledger-backed timeline events.** v1 live stock mutation is gated on adding explicit route dispatch/delivery `GameActionType` entries and route ledger modules. Reverts invalidate dispatches from abandoned timelines. Stock quicksave/load restores stock vessels from `.sfs`; Parsek timeline rewind must use the route ledger modules to reverse or mask KSC funds, physical origin debits, and endpoint deliveries. Until that mechanism exists, route creation can analyze candidates but must not mutate stock vessels. **(v0 reconcile — see §10.6 + report `docs/dev/done/research/logistics-time-rewind-compat-report.md`):** as shipped, physical route effects ride the full-world quicksave restore (NOT a reversing route module — `RouteModule` is observe-only) and funds ride the ledger recalc; the Rewind-to-Separation route-row-drop fix (Rec-1, PR #1242, superseding draft #1221; *verified in-game 2026-07-08*) closes the "funds charged / cargo not re-delivered" gap by dropping free-standing route rows with `UT > the rewind cutoff` at restore. The non-rewind discard (no quicksave) residual was RATIFIED as by-design (maintainer option C, 2026-07-06): both funds and cargo persist on a plain discard, recorded as an observability-only `[Rec-3 residual]` diagnostic (PR #1243); reverse writers are declined.

12. **Routes persist across scenes and save/load.** All route state is serialized in the .sfs. The scheduler runs in all scenes via ParsekScenario.

3. Terminology

Logistics — the player-facing feature/window that manages Supply Routes.

Supply Route — a separate entity that defines one recurring cargo transfer from an origin to one endpoint. Created from a committed Supply Run after player confirmation. Renders as a looped Mission segment via `MissionLoopUnitBuilder`; the earlier chain-sequential ghost playback model is superseded (section 0.7).

Supply Run — the concrete player-flown recording chain that proves a route. It contains the transport path, stock connection window, resource/inventory delta, and disconnect. In v1 the connection window is dock/undock.

Route stop / endpoint — the destination vessel and location for the route. v1 exposes one endpoint per Supply Route. The data model keeps a `Stops` list so multi-stop Supply Runs can be added later without replacing the save shape.

Stock connection window: the bounded time interval where the transport is connected to the endpoint by a stock mechanism and cargo can move. Two producers are wired through capture and analysis: docking-port dock/undock and claw/grapple couple/release (`ModuleGrappleNode`; the decompile proved both ride the same `Part.Couple` / `Part.Undock` event pair, see `docs/dev/research/claw-grapple-coupling-internals.md`). Other stock transfer paths (crossfeed/fuel-line) are not window-shaped and stay out (19.6 rule 10).

Endpoint — origin or destination location of a route. Defined by body, coordinates, and the target vessel PID from the stock connection window. Surface endpoints can fall back to one nearest compatible vessel near the recorded coordinates. Orbital endpoints use PID only.

Delivery manifest — the per-resource and inventory amounts delivered at the endpoint. The base case is delivery (resource and inventory deltas representing cargo leaving the transport and appearing on the endpoint part set while docked); pickup routes and mixed pickup/delivery windows SHIPPED in M3, so a single dock can also load cargo from a source vessel onto the transport.

Cost manifest — the resource and inventory quantities used or delivered by the Supply Run. For non-KSC origins, this is deducted from the recorded start-docked origin depot each cycle.

KSC dispatch cost — in Career, the funds cost charged when a KSC-origin route dispatches. It is computed from stock costs for the source vessel parts plus the resource/inventory quantities used or delivered by the Supply Run.

Dispatch — the moment a route cycle begins. Origin resources are checked and deducted only after the route ledger event is recorded. A timeline event is created.

Delivery — the moment delivery occurs at a stop boundary. Resources and inventory are added to the endpoint vessel only through the route delivery ledger/applier path. A timeline event is created.

Round-trip link — future scheduling constraint pairing two one-way routes: "don't dispatch me until my partner completes."

Route analysis engine — pure logic that walks the committed Supply Run chain to extract the endpoint, delivery manifest, source recording IDs, and transit duration from recording data (resource/inventory manifests, stock connection target PID, connection kind, location context).

---

4. Data Model

4.1 ResourceAmount

`ResourceAmount` is the existing internal type in `Source/Parsek/ResourceManifest.cs`; logistics reuses it and must not introduce a duplicate definition.

```
internal struct ResourceAmount
{
    public double amount;
    public double maxAmount;
}
```

4.2 InventoryPayloadItem

```
internal class InventoryPayloadItem
{
    public string PartName;                // stock part name from STOREDPART
    public string VariantName;             // variant/theme identity when present
    public int Quantity;                   // stack quantity for identical payload snapshots
    public int SlotsTaken;                 // stock inventory slot cost
    public double UnitDryCost;             // dry part/module cost, excluding stored resources
    public Dictionary<string, double> StoredResources; // resources carried inside the stored part
    public string IdentityHash;            // hash of the canonical STOREDPART snapshot
    public ConfigNode StoredPartSnapshot;  // full stock STOREDPART node for reconstruction
}
```

Inventory route manifests use exact stored-part payload snapshots, not part-name counts. Two stored parts with different variants, resource contents, module state, or stock snapshot data are different payload items. `Quantity` may compress identical snapshots only after the canonical `STOREDPART` payload is equal. `StoredPartSnapshot` must preserve the stock node identity as `STOREDPART`; wrapper nodes used by Parsek serialization must be stripped before hashing or reconstructing inventory.

`IdentityHash` is computed from a deterministic canonical form, not from `ConfigNode.ToString()` directly. Canonicalization must include the node name, sorted value entries by key then value, and child nodes sorted by node name then canonical payload. Numeric values that logistics writes must use `ToString("R", CultureInfo.InvariantCulture)` before hashing. Existing stock string values are preserved as strings. The hash input must exclude transient ordering, whitespace, comments, `slotIndex`, stack `quantity`, and vessel-local lifecycle fields inside the nested `STOREDPART > PART` snapshot (`persistentId`, launch/attachment/position/rotation/temperature/derived cost-mass style fields) so a save/load round trip, slot move, stack split, or dock-to-undock inventory move does not change payload identity. Slot usage and quantity remain separate manifest fields, while nested `RESOURCE` and `MODULE` payload state remains identity-defining.

4.3 RouteEndpoint

```
internal struct RouteEndpoint
{
    public string bodyName;        // e.g., "Kerbin", "Mun"
    public double latitude;        // from dock event trajectory point
    public double longitude;
    public double altitude;
    public uint vesselPersistentId;         // 0 = KSC sentinel for KSC origins; non-zero for all stops and non-KSC or
igins
    public bool isSurface;         // true = landed/splashed/prelaunch endpoint; enables surface fallback
}
```

`vesselPersistentId = 0` is reserved for KSC origins where there is no stock origin vessel to resolve. Non-KSC origins and all route stops must use a non-zero vessel PID captured from the stock connection boundary.

4.4 RouteConnectionKind

```
internal enum RouteConnectionKind
{
    DockingPort,
    Grapple,        // stock Advanced Grabbing Unit / claw, if target detection is reliable
    StockCrossfeed, // stock transfer/crossfeed path, if endpoint detection is reliable
    Unknown
}
```

4.5 RouteStatus

```
internal enum RouteStatus
{
    Active,             // dispatching on schedule
    InTransit,          // dispatched, waiting for transit duration to elapse
    WaitingForResources, // origin exists but lacks resources — delayed
    WaitingForFunds,    // Career KSC-origin route lacks dispatch funds — delayed
    DestinationFull,    // destination can't accept delivery — waiting for capacity
    EndpointLost,       // destination/origin vessel gone (orbital PID miss or no surface vessels)
    MissingSourceRecording, // route source recording chain is gone; route cannot dispatch
```

```
    SourceChanged,      // source recording exists but no longer matches the route proof fingerprint
    Paused              // player manually paused
}
```

4.6 RouteStop

```
internal class RouteStop
{
    public RouteEndpoint Endpoint;                          // where this stop is
    public RouteConnectionKind ConnectionKind;               // how the Supply Run connected
    public Dictionary<string, double> DeliveryManifest;     // per-resource delivery amounts (positive only in v1)
    public List<InventoryPayloadItem> InventoryDeliveryManifest; // exact stored-part payloads delivered in v1
    public int SegmentIndexBefore;                          // 0-based source recording whose completion UT triggers t
his stop
    public double DeliveryOffsetSeconds;                    // seconds from CurrentCycleStartUT to this stop boundary
}
```

v1 exposes a single-stop route in the UI. Multi-stop Supply Runs are a planned extension; the list shape stays now so save data does not need to be replaced later.

4.7 Route

```
internal class Route
{
    // Identity
    public string Id;                   // unique route ID (GUID)
    public List<string> RecordingIds;   // ordered chain of source recording IDs
    public string Name;                 // player-visible name (editable); default: the source mission's name, number
ed "Name [2]"
    public List<RouteSourceRef> SourceRefs; // immutable source proof/version refs captured at route creation

    // Endpoints
    public RouteEndpoint Origin;
    public List<RouteStop> Stops;       // ordered stops along the route
    public bool IsKscOrigin;            // true = Career charges KSC funds instead of physical origin cargo

    // Resource transfer
    public Dictionary<string, double> CostManifest;       // per-resource quantities used or delivered
    public List<InventoryPayloadItem> InventoryCostManifest; // exact stored-part payloads used or delivered
    public double KscDispatchFundsCost;                   // stock part + used/delivered cargo funds per KSC dispatch

    // Timing
    public double TransitDuration;      // seconds (= total chain duration)
    public double DispatchInterval;     // seconds between cycle starts
    public double DispatchWindowEpochUT; // original flight start UT; anchors inter-body synodic phase
    public double DispatchWindowPeriod;  // INFORMATIONAL ONLY since 2026-09-02 (see 'Route render scope' below); alwa
ys 0 in practice
    public double NextDispatchUT;       // UT of next scheduled dispatch
    public double? CurrentCycleStartUT;  // UT when the in-transit cycle began; null when idle
    public double? NextEligibilityCheckUT; // retry backoff for resource/funds waits; null when not waiting
    public int CurrentSegmentIndex;     // 0-based active source-recording index; -1 when not in transit

    // Per-stop pending delivery (computed at each stop boundary during transit)
    public double? PendingDeliveryUT;    // UT when next route boundary is due (null if not in transit)
    public int PendingStopIndex;        // stop due at PendingDeliveryUT, or -1 when current boundary has no stop

    // Linking
    public string LinkedRouteId;        // paired route for round-trip (null if standalone)

    // State
    public RouteStatus Status;
    public bool PauseAfterCurrentCycle; // pause requested while InTransit; transition to Paused after completion
    public int CompletedCycles;         // total successful cycle completions
    public int SkippedCycles;           // reserved diagnostic counter for explicit skip policies; v1 wait states do
not increment it
}
```

```
internal class RouteSourceRef
{
```

```
    public string RecordingId;
    public string TreeId;
    public int TreeOrder;
    public int RecordingFormatVersion;
    public int RecordingSchemaGeneration;
    public int SidecarEpoch;
    public double StartUT;
    public double EndUT;
    public string RouteProofHash;  // hash of route-relevant connection windows/manifests used by this route
}
```

Forward compatibility with multi-stop routes: A v1 Supply Route has `Stops.Count == 1`. Later multi-stop routes can reuse the same list without changing the top-level route save node.

Route playback segment definition: In this document, a route playback segment is one source recording in `RecordingIds`. `CurrentSegmentIndex` indexes `RecordingIds` directly: `-1` means no active transit, `0` means the first source recording is active, `1` means the second source recording is active, and so on. It does not index trajectory samples, internal `TrackSection` ranges, or lower-level chain-manager segments. `RouteStop.SegmentIndexBefore` is the source-recording index whose visual completion boundary normally corresponds to that stop, but delivery timing is pinned by `RouteStop.DeliveryOffsetSeconds`.

Source immutability contract: A route stores its timing, delivery offsets, manifests, and source proof fingerprints when it is created. The scheduler uses the stored route fields, not live recording durations, to decide dispatch and delivery timing. At dispatch and before each in-transit delivery, route revalidation compares `SourceRefs` against the current committed recordings. A missing source sets `MissingSourceRecording`; a source with the same ID but changed route-relevant epoch/fingerprint sets `SourceChanged`. Route effects stop until the player revalidates/recreates the route from the current committed recording data.

4.8 Serialization format

```
ROUTE
{
    id = <guid>
    name = Mun Fuel Run

    RECORDING_IDS
    {
        id = <recording-guid-1>
        id = <recording-guid-2>
        id = <recording-guid-3>
    }
    SOURCE_REFS
    {
        SOURCE
        {
            recordingId = <recording-guid-1>
            treeId = <tree-guid>
            treeOrder = 0
            recordingFormatVersion = 1
            recordingSchemaGeneration = 4
            sidecarEpoch = 12
            startUT = 42654.0
            endUT = 47000.0
            routeProofHash = 93C2...
        }
    }

    ORIGIN
    {
        bodyName = Kerbin
        latitude = -0.0972
        longitude = -74.5577
        altitude = 75.2
        vesselPersistentId = 0
        isSurface = True
    }

    STOP
    {
        ENDPOINT
        {
            bodyName = Mun
```

```
            latitude = 3.2001
            longitude = -45.1234
            altitude = 612.5
            vesselPersistentId = 67890
            isSurface = True
        }
        connectionKind = DockingPort
        segmentIndexBefore = 1
        deliveryOffsetSeconds = 600.0
        DELIVERY_MANIFEST
        {
            LiquidFuel = 150.0
            Oxidizer = 183.3
        }
        INVENTORY_DELIVERY_MANIFEST
        {
            ITEM
            {
                partName = smallSolarPanel
                variantName =
                quantity = 2
                slotsTaken = 2
                unitDryCost = 75.0
                identityHash = 8F17...
                STOREDPART
                {
                    // canonical stock STOREDPART snapshot, including variant/module/resource payload
                }
            }
        }
    }
    isKscOrigin = True
    kscDispatchFundsCost = 12500.0
    transitDuration = 12345.6 // total source path duration; delivery timing uses STOP.deliveryOffsetSeconds
    dispatchInterval = 43200.0
    dispatchWindowEpochUT = 42654.0
    dispatchWindowPeriod = 0.0
    nextDispatchUT = 258654.0
    // currentCycleStartUT omitted when null
    // nextEligibilityCheckUT omitted when null
    currentSegmentIndex = -1
    // pendingDeliveryUT omitted when null
    pendingStopIndex = -1
    linkedRouteId =
    status = Active
    pauseAfterCurrentCycle = False
    completedCycles = 5
    skippedCycles = 0

    COST_MANIFEST
    {
        LiquidFuel = 155.0
        Oxidizer = 183.3
    }
    INVENTORY_COST_MANIFEST
    {
        ITEM
        {
            partName = smallSolarPanel
            variantName =
            quantity = 2
            slotsTaken = 2
            unitDryCost = 75.0
            identityHash = 8F17...
            STOREDPART
            {
                // same canonical payload identity used for delivery and cost
            }
        }
    }
    // KSC-origin routes use ORIGIN.vesselPersistentId = 0. Non-KSC origins and all STOP endpoints use real vessel PID
s.
    // When Status == InTransit, pendingDeliveryUT/pendingStopIndex identify the next due boundary.
    // When Status == InTransit, currentCycleStartUT identifies the scheduler elapsed time for visual handoff.
    // When Status == WaitingForResources or WaitingForFunds, nextEligibilityCheckUT gates retry polling.
```

```
    // Actual deliverable amounts are recomputed at delivery time from current endpoint capacity.
    // Delivery timing uses serialized deliveryOffsetSeconds; live recording duration changes do not shift a route.
}
```

Routes are stored in their own `ROUTES` ConfigNode section inside ParsekScenario's save data, alongside recordings and game actions. Additive — saves without routes load fine. Routes reference recordings by ID but are independent entities. A route whose source recordings are missing is disabled, not allowed to keep moving cargo without the proof recording.

4.9 Recording extensions (Phase 11 + Logistics prerequisites)

Phase 11 adds base vessel-level manifest metadata to recordings. The current code has `ResourceAmount`, `InventoryItem`, `Recording.StartResources`, `Recording.EndResources`, `Recording.StartInventory`, `Recording.EndInventory`, manifest codecs, and the resource/inventory extractors in `VesselSpawner`. Those are useful diagnostics, but they are not sufficient proof for Logistics.

Logistics also needs connection-scoped manifests: during docking, KSP merges vessels, so aggregate vessel resources cannot prove that cargo moved between the transport and endpoint. Route analysis must filter manifests by the original transport part persistent IDs and original endpoint part persistent IDs captured at the docking boundary, then resolve those same part sets after undock.

Resource manifests (current Phase 11 base):

```
public Dictionary<string, ResourceAmount> StartResources;  // manifest at recording start
public Dictionary<string, ResourceAmount> EndResources;     // manifest at recording end
```

`ResourceAmount` is a struct with `amount` and `maxAmount` fields. Resources are summed across all parts. ElectricCharge and IntakeAir are excluded (environmental noise). Extracted by `VesselSpawner.ExtractResourceManifest(ConfigNode vesselSnapshot)`.

Implementation prep must verify the always-tree path persists these fields on the actual committed `Recording` objects, not only on transient `FlightRecorder.CaptureAtStop` objects. In the current code, always-tree root recordings are created before `FlightRecorder.StartRecording`, and `ParsekFlight.AppendCapturedDataToRecording` / `FlushRecorderToTreeRecording` append trajectory data but do not currently make the manifest ownership invariant obvious. Before route analysis ships, tests must prove committed tree recordings carry the intended start/end manifests after normal commit, dock/undock, split/merge, scene-exit, and optimizer paths.

Connection-scoped manifests (v1 logistics prerequisite; implementation in progress):

```
internal sealed class RouteConnectionWindow
{
    public string Id;
    public RouteConnectionKind Kind;                        // DockingPort only in v1
    public double DockUT;
    public double UndockUT;
    public uint TransportVesselPidAtDock;
    public uint EndpointVesselPidAtDock;
    public List<uint> TransportPartPersistentIds;           // original transport part set
    public List<uint> EndpointPartPersistentIds;            // original endpoint part set
    public Dictionary<string, ResourceAmount> DockTransportResources;
    public Dictionary<string, ResourceAmount> UndockTransportResources;
    public Dictionary<string, ResourceAmount> DockEndpointResources;
    public Dictionary<string, ResourceAmount> UndockEndpointResources;
    public List<InventoryPayloadItem> DockTransportInventory;
    public List<InventoryPayloadItem> UndockTransportInventory;
    public List<InventoryPayloadItem> DockEndpointInventory;
    public List<InventoryPayloadItem> UndockEndpointInventory;
    public RouteEndpoint EndpointAtDock;
    public Vessel.Situations TransferEndpointSituation;     // endpoint vessel situation at docking
}

internal sealed class RouteOriginProof
{
    public uint StartDockedOriginVesselPid;                 // AS BUILT 2026-09-02: ALWAYS 0 at capture, the bind-la
ter slot
    public uint StartDockedOriginRootPartUId;               // AS BUILT: the depot's identity - a launch-unique part
flightID
    public string StartDockedOriginVesselName;              // AS BUILT: the depot's pre-dock name from the docking-
node pair
    public int StartDockedOriginVesselType;                 // AS BUILT: the depot's pre-dock vesselType - the field
the rule selects on
```

```
    public uint StartDockedTransportRootPartUId;              // AS BUILT: the OTHER half of the seam
    public int StartDockedTransportVesselType;                // AS BUILT: the OTHER half's type
    public Dictionary<string, ResourceAmount> StartTransportResources;
    public Dictionary<string, ResourceAmount> EndTransportResources;
    public List<InventoryPayloadItem> StartTransportInventory;
    public List<InventoryPayloadItem> EndTransportInventory;
}
```

Store completed connection windows on the tree recording that represents the docked merged vessel. In the always-tree flow, a dock creates a `BranchPointType.Dock` merge and a merged child recording; the later undock creates a split from that merged recording. The route window naturally spans that merged child. If the window cannot be completed with a matching undock, the route candidate is invalid.

Current implementation prep has the always-tree dock/undock skeleton in place: `RouteProofCapture` builds the dock window from the merged vessel snapshot, records transport/endpoint part PID sets, filters resource manifests by those PID sets, preserves exact `STOREDPART` inventory payload snapshots, and completes the same window at the later undock from the two split-vessel snapshots. Active-as-target dock merges must not look up the absorbed endpoint vessel after the merge, because stock may already have destroyed it; endpoint proof is captured from the dock event when available and otherwise restored from the absorbed background parent snapshot. Runtime validation is still required for the stock dock event's before/after vessel references, but missing active-as-target endpoint proof is no longer an accepted route candidate.

The docked aggregate vessel manifest is still useful for diagnostics, but Logistics v1 must compute delivery from matched transport-loss and endpoint-gain fields. Cost uses full-run fields scoped to the original transport part set. Missing connection-scoped fields mean the recording cannot become a Supply Route.

Serialized as:

```
RESOURCE_MANIFEST
{
    RESOURCE { name = LiquidFuel  startAmount = 3600  startMax = 3600  endAmount = 200  endMax = 3600 }
    RESOURCE { name = Oxidizer    startAmount = 4400  startMax = 4400  endAmount = 244  endMax = 4400 }
}
```

Stock connection target vessel PID (v1 logistics prerequisite):

```
public uint TransferTargetVesselPid;       // PID of vessel connected to at this segment boundary (0 = no route-releva
nt connection)
public RouteConnectionKind TransferKind;   // DockingPort or Grapple (classified at capture); Unknown fails closed at
admission
```

For the first implementation, `TransferTargetVesselPid` is populated from the dock-merge path as the other vessel in the stock docking event, from the active recording's perspective. It must never fall back to the post-dock merged vessel PID: if the other vessel cannot be identified, store `0` and make route analysis reject the candidate as missing endpoint proof. Route analysis should consume the generic `TransferTargetVesselPid` / `TransferKind` contract, not reach into legacy `DockTargetVesselPid` directly.

Inventory manifests (current Phase 11 base plus required Logistics extension):

KSP 1.12 `ModuleInventoryPart` items are stored parts in cargo containers. The current Phase 11 implementation returns `Dictionary<string, InventoryItem>` keyed by part name with count and slot usage. That is good for diagnostics, but it is not enough for route delivery.

Logistics v1 requires exact payload snapshots. The extension to `ExtractInventoryManifest` must walk MODULE > STOREDPARTS > STOREDPART nodes, preserve each canonical `STOREDPART` ConfigNode, record slot usage, variant identity, stored resources, and a stable identity hash. Delivery and cost matching operate on those payload identities, not just part names. This enables automated parts delivery without treating variants, stacked payloads, or resource-filled stored parts as fungible.

Crew manifests (implemented — Phase 11, not consumed by Logistics v1):

Crew composition by trait (Pilot/Scientist/Engineer/Tourist). Logistics v1 does not deliver generic kerbals; stock KSP crew are named roster entities, not fungible cargo. Crew rotation should be a later feature with explicit roster, hiring, and reservation semantics.

All manifest types are additive: missing node = no data. Route entity data in the separate `ROUTES` section does not require a recording format bump. Recording-side route proof fields target additive missing-node defaults, but the final implementation must confirm the current `RecordingFormatVersion` / `RecordingSchemaGeneration` policy with the `RecordingTreeRecordCodec` owner. If the

current schema gate requires a generation bump for newly persisted recording metadata, bump the schema and add compatibility tests instead of asserting "no bump" by default.

4.10 Actual code contract before v0 implementation

The first implementation phase is not `RouteScheduler`. It is a recorder/data-contract alignment phase. The route module can stay clean only if committed recordings expose trustworthy route-proof data first.

Current code facts:

- Always-tree mode is canonical. A Supply Run should be analyzed from `RecordingStore.CommittedTrees`, `RecordingTree.Recordings`, and `RecordingTree.BranchPoints`, not from a standalone recording list.
- `Recording.StartResources` / `EndResources` and `StartInventory` / `EndInventory` exist, and the tree codec serializes them. Tests must prove the committed always-tree path actually populates them after the active recorder flushes.
- `Recording.DockTargetVesselPid` exists for legacy dock segments. Always-tree dock merges need route-facing metadata on the dock branch/window: target PID, connection kind, endpoint situation, endpoint coordinates, and the two part PID sets.
- `BranchPoint.MergeCause` / `TargetVesselPersistentId` already serialize. `TargetVesselPersistentId` also feeds `GhostChainWalker`, so logistics must leave it at `0` unless it has explicit, tested dock endpoint proof.
- `TrajectoryPoint` has `latitude` / `longitude` / `altitude` / `bodyName`; `Recording` does not have `StartLatitude` or `StartLongitude` fields. Route origin/endpoint coordinates must come from the selected trajectory point or from `RouteConnectionWindow.EndpointAtDock`, not from nonexistent recording-level coordinate fields.
- `GameActionType` has no route dispatch/delivery entries yet. v0 decision: live route effects must use explicit route action types plus recalculation/rollback support. Route-local persisted state may cache scheduler state, but it must not be the sole authority for stock funds/cargo mutation. Shipping hidden stock mutations outside the ledger contract is not allowed.

Required prep tasks:

1. **Manifest invariant:** define and test exactly which committed tree recording owns `StartResources`, `EndResources`, `StartInventory`, `EndInventory`, and the route transport-scoped start/end manifests. This must cover single-node trees, dock merges, undock splits, scene-exit commits, and optimizer rewrites.
2. **Connection window capture:** add a serializable route connection window on the docked merged recording. Populate it at dock with endpoint PID/situation/coordinates, transport and endpoint part PID sets, and dock-side scoped resource/inventory manifests. Complete it at undock with undock-side scoped manifests.
3. **Exact inventory payloads:** add `InventoryPayloadItem` extraction/serialization while preserving the existing lightweight `InventoryItem` diagnostics. Route delivery must depend on exact payload identity, not part-name counts.
4. **Route-facing connection fields:** map existing dock data into `TransferTargetVesselPid` / `TransferKind` and stop route analysis from depending on legacy names. v1 rejects anything except `DockingPort`.
5. **Origin proof:** detect and serialize `StartDockedOriginVesselPid` only when the Supply Run starts connected to a real non-KSC depot. Non-KSC candidates without that proof are invalid.
6. **Timeline contract:** implement route `GameActionType` entries and route ledger modules before stock mutation code lands. Career KSC charges and physical origin/destination resource edits must be replayable or invalidated across save/load and rewind using the same epoch rules as other committed effects.

Until these tasks are done, route creation must report "recording lacks logistics proof data" instead of offering a Supply Route prompt.

---

5. Route Creation

5.1 Supply Run workflow

Route creation uses an automatic post-commit prompt on an eligible Supply Run. A special pre-flight mode is not required for v1.

Player flow:

1. Player flies mission normally.
2. Transport docks to the destination.
3. Player transfers resources and/or stock inventory to the destination through KSP's normal UI/systems.
4. Transport undocks from the destination so the endpoint is available for the next dispatch.
5. Player commits the recording.

6. If route analysis finds an eligible Supply Run, Parsek automatically prompts "Create Supply Route from this Supply Run?"
7. Player reviews origin, endpoint, delivery manifest, KSC dispatch cost, transit time, and interval; then confirms.
8. Supply Route goes live.

Deferred "Record Supply Run" helper:

- May mark the current recording tree as route-intended for UI filtering and prompt suppression.
- Shows a "Supply Run Active" indicator.
- Does not change recorder behavior. The same `FlightRecorder`, chain boundaries, snapshots, and manifests are used.
- Does not make an invalid run valid. The committed chain must still contain a detected docking connection, delivery cargo delta, and undock.

5.2 Route analysis engine

`RouteAnalysisEngine` (in `Logistics/`) is pure logic over committed recording-tree data. Fully testable without KSP.

Input: a committed `RecordingTree`, or a deterministic projection from that tree into the route source path. This can come from an explicit route-intended session marker, the just-committed tree, or a "Create Supply Route" action on an existing recording tree.

Current implementation prep has the first read-only analyzer slice: it scans the active source path for exactly one completed `RouteConnectionWindow`, rejects old recordings that only have aggregate Phase 11 manifests, ignores off-path windows, rejects multi-window/multi-stop candidates for v0, rejects mixed pickup/delivery windows, requires endpoint proof, and derives resource/inventory delivery manifests only when endpoint gains are matched by transport losses. It does not yet create route entities, persist route store data, prompt the user, schedule dispatches, or mutate stock resources.

Always-tree mode means the analysis input is not just a flat chronological list. The engine must inspect:

- `RecordingTree.RootRecordingId` and `RecordingTree.Recordings` for source recordings.
- `RecordingTree.BranchPoints` for dock/undock structure.
- Completed `RouteConnectionWindow` metadata stored on the docked merged recording.
- Recording UT bounds (`StartUT` / `EndUT`), source `RecordingId`s, and trajectory points for timing and coordinates.

The engine may return an ordered `RecordingIds` list for route playback, but that list is an output of tree/path analysis, not the primary source of truth.

Algorithm:

```
Walk the committed tree's source path chronologically:
  Find each completed DockingPort RouteConnectionWindow:
    Verify the tree has a dock branch and a later undock/split boundary for the same window
    stop.endpoint = {
        body, lat, lon from window.EndpointAtDock or the connection-time trajectory point,
        vesselPersistentId = TransferTargetVesselPid,
        connectionKind = TransferKind,
        isSurface = connected endpoint vessel situation is Landed/Splashed/Prelaunch
    }
    transportBefore = resource/inventory manifest for the original transport part PID set
                      captured immediately before docking merge
    transportAfter = resource/inventory manifest for the same transport part PID set
                     captured immediately after undock/separation
    endpointBefore = resource/inventory manifest for the original endpoint part PID set
                     captured immediately before docking merge
    endpointAfter = resource/inventory manifest for the same endpoint part PID set
                    captured immediately after undock/separation
    transportLoss = positive deltas from transportBefore - transportAfter
    transportGain = positive deltas from transportAfter - transportBefore
    endpointGain = positive deltas from endpointAfter - endpointBefore
    endpointLoss = positive deltas from endpointBefore - endpointAfter
    if any non-ignored transportGain or endpointLoss exists:
        reject candidate as pickup/mixed-transfer window in v1
    stop.deliveryManifest = per-resource min(transportLoss, endpointGain)
                            (delivery proof is connection-scoped, not merged-vessel aggregate)
    stop.inventoryDeliveryManifest = exact InventoryPayloadItem snapshots matched between
                                     transport loss and endpoint gain by identityHash
    stop.deliveryOffsetSeconds = delivery boundary UT - first source recording StartUT

  origin = KSC launch site, or non-KSC origin depot proven by a start-docked origin vessel
```

```
orderedRecordingIds = route playback path through the tree
sourceRefs = immutable source version/fingerprint refs for every recording used by the route
totalTransit = last source recording EndUT - first source recording StartUT
costManifest = source resource quantities used or delivered over the full run
inventoryCostManifest = exact inventory payload snapshots used or delivered over the full run
```

Derived values:

- **Origin** = KSC launch site, or a non-KSC origin depot vessel if the Supply Run starts docked to that depot and records its PID
- **Stops** = the one complete dock-transfer-undock delivery window accepted by v1; multiple delivery windows make the candidate ineligible
- **Origin vessel PID** = `0` for KSC routes; and for a start-docked origin it is `0` at capture and stamped at the UNDOCK bind only when the launch-guid gate clears it, AS BUILT 2026-09-02 (P12) - so the origin is identified by `StartDockedOriginRootPartUId` and resolved through the root-part step, with the M1 endpoint descriptor behind it. A MID-TREE docked origin still carries a real pid from its dock-time endpoint
- **Stop vessel PID** = `TransferTargetVesselPid` from the stock connection boundary
- **IsKscOrigin** = true if origin body is Kerbin and coordinates are near a launch site
- **DeliveryManifest** = matched resource amount that both left the transport part set and appeared on the endpoint part set across the dock/undock window
- **InventoryDeliveryManifest** = exact stored-part payload snapshots that both left the transport part set and appeared on the endpoint part set across the same window
- **CostManifest** = source resource quantities used or delivered over the full Supply Run
- **InventoryCostManifest** = exact stored-part payload snapshots used or delivered over the full Supply Run
- **KscDispatchFundsCost** = dry stock part cost plus stock resource/inventory cost for used-or-delivered quantities on Career KSC-origin routes
- **TransitDuration** = total source path duration (last source recording EndUT minus first source recording StartUT)
- **DeliveryOffsetSeconds** = per-stop serialized scheduler offset from cycle start to the delivery boundary; live recording duration changes never move this boundary
- **DispatchInterval** = TransitDuration (default; player can increase). For inter-body routes: defaults to synodic period of origin and destination bodies.
- **DispatchWindowEpochUT** = first recording StartUT; inter-body repeats stay phase-aligned to this UT
- **DispatchWindowPeriod** = INFORMATIONAL ONLY. Documented as "0 for same-body, synodic period for inter-body" and read as the render's scope flag, but `RouteBuilder` has always hard-coded 0.0 and no other production site assigns it, so the contract was never true of a real save. Render scope is derived from the endpoints instead (see "Route render scope" under section 17); the field is kept, still written and read by the codec (no schema move), and nothing in the logistics runtime reads it
- **RecordingIds** = ordered route source path recording IDs, derived from the committed tree
- **SourceRefs** = immutable source recording fingerprints used to detect deletion, optimizer rewrites, and superseded recordings after route creation

Start-docked non-KSC origin detection: A non-KSC route is valid only when the first source recording starts while the transport is already connected to a real origin depot. The recorder must capture the connected origin vessel PID, connection kind, origin part PID set, and transport-scoped start/end manifests without requiring a `DockUT` inside the Supply Run. If the run starts undocked away from KSC, or the start-docked vessel is a ghost/EVA/debris/invalid cargo owner, route analysis rejects the candidate instead of debiting an arbitrary nearby vessel.

AS BUILT 2026-09-02 (P12) - THE ORIGIN IS BOUND AT THE UNDOCK, NOT CHOSEN AT CAPTURE. A settled dock is ONE `Vessel`: `Part.Couple` merges the pair and destroys the absorbed half's `Vessel`, so THE CONNECTED ORIGIN VESSEL PID IS NOT KNOWABLE at recording start. What the docking-node PAIR does carry is one `DockedVesselInfo` per half - `name` / `vesselType` / `rootPartUId`, no pid and no guid, round-tripping through the node's `DOCKEDVESSEL` save node.

THE RULE, in two halves:

1. **CAPTURE RECORDS THE PAIR AND NAMES NO ORIGIN.** Both halves go onto the proof as `RouteOriginProof.StartDockedPair` - each half's `rootPartUId` (a part `flightID`, assigned per launch and not baked into the `.craft`, hence launch-unique where a `persistentId` is not), its pre-dock name, its `vesselType` **for information only**, and the part- `persistentId` set it owns on the merged vessel, derived by cutting the seam edge in the merged part tree. The only rejection at capture is the valid-cargo-owner guard - `Debris` / `SpaceObject` / `Unknown` / `EVA` / `Flag` / deployed-science, this paragraph's own reject set - plus a half with no usable identity. THERE IS NO VESSEL-TYPE DISCRIMINATOR: there is no "base" type that matters, bases are ordinary vessels, and a route candidate is defined by TRANSFERS AND DOCKS - the transport takes cargo at one docked partner and delivers it at another.

2. **THE UNDOCK BINDS THE ORIGIN TO THE HALF THE RUN DID NOT KEEP FLYING.** At the split, `RouteProofCapture.TryBindStartDockedOriginAtUndock` matches the two post-split part sets against the two captured halves; the ACTIVE side is the half the recorder follows (the transport) and the BACKGROUND side is the origin. Matching is by part set, never by vessel pid. The bound origin's live pid is stamped only behind the launch-guid gate (`VesselLaunchIdentity`), which is what refuses the self-origin. A recording that ends still docked is `UnboundAtStop` and is never an origin: nothing witnessed which half it left behind.

3. **THE PICKUP MUST BE WITNESSED AS A FLOW.** The bind compares the TRANSPORT half's own manifests at recording start and at the undock and classifies the result: a rise in an admitted resource is `Gain`, otherwise non-empty admitted cargo aboard the transport is `Carried`, otherwise `None`. **ONLY `Gain` VALIDATES**, and `RouteAnalysisEngine.HasDockedOriginProof` requires it. 19.2.2 item 2 defines Loaded as cargo that FLOWED from another vessel ONTO the transport, and 19.2.1 makes origin causal - "the witnessed event that put each unit of cargo on the transport" - so a transport that leaves a seam with the cargo it arrived with witnessed nothing there. Admitting `Carried` would have validated residual anything: a pure DELIVERY undock, where the transport's cargo went DOWN, still leaves monopropellant or leftover fuel aboard and would name the vessel it just delivered to as its supply origin. `Carried` survives as an OBSERVED classification because it is the log discriminator between "left with nothing" and "left with cargo it already had".

   THE INFLOW THAT PREDATES THE RECORDING (built 2026-09-06, closing ROUTE-ORIGIN-PROOF-PICKUP-PREDATING-THE-RECORDING). The player docks the tanker to the base, transfers fuel, quicksaves and quits; the next session's recording opens already docked and never witnesses the inflow, so the in-recording rule above can only read `Carried`. The flow WAS witnessed - by the PREVIOUS recording of the same launch, whose `RouteConnectionWindow` brackets the load - and the bind now consults it: `RouteProofCapture.TryResolveOriginProofPredecessor` walks one edge back inside the tree (parent branch point, then `ParentRecordingId`, then the chain predecessor, every candidate gated by `VesselLaunchIdentity.RecordingsShareLaunch`), and the pure `ClassifyPredecessorPickup` evaluates the LATEST window on it by dock UT - never a scan for any window that gained, which would be cherry-picking across seams. That window counts only when its transport side is the half now flying the run and its endpoint side is the half now being named the origin (the same `WindowNamesHalfAsTransport` predicate), when the launch-unique keys agree (`RouteConnectionWindow.EndpointRootPartUId`, the endpoint's root part `flightID` captured at the dock, against the origin half's `rootPartUId`; unknown on either side degrades to the part-pid overlap), and when it is entirely in the past. A COMPLETE window is measured inside itself, dock manifest → undock manifest; an OPEN one - still docked when the predecessor ended, which is WHY this recording starts docked - is measured dock manifest → this recording's own transport-half start manifests, and additionally requires the two transport part sets to be EQUAL because only that shape differences manifests taken from different records. The rise is `ClassifyOriginPickup` reused verbatim, so there is still ONE transfer rule; the result is stamped as `OriginPickupKind.GainFromPredecessorWindow`, which validates exactly like `Gain`. **IT IS CONSULTED ONLY WHEN THIS RECORDING DID NOT WITNESS THE DOCK and did not already measure a `Gain`**: a witnessed dock that moved nothing onto the transport is POSITIVE delivery evidence, and a window one recording back must never overturn what this recording saw. Every refusal is named on the bind line as `predecessorPickup=`. Derivation: `docs/dev/research/pickup-predating-the-recording.md`.

Nothing anywhere reads which half KSP made dominant on the merge (`Vessel.GetDominantVessel`: vesselType, then mass, then a guid compare), because that names the transport whenever the depot is the lighter or lower-typed half - exactly this doc's line "deducts from recorded origin depot, NOT TRANSPORT". Full derivation: `docs/dev/research/origin-proof-partner-identity-memo.md`.

WHAT THIS SUPERSEDES: the 2026-09-02 depot-TYPED selection (origin = the `Base` / `Station` half). It required the player to have retyped their base - no stock part declares either type, so an ordinary landed base reads `Ship` / `Probe` / `Lander` - and it fail-closed on the rover-to-rover relay the roadmap is aimed at, measured live (`RouteOriginProof skipped: no depot half ... seams=2 candidates=0`, `nearType=5 farType=5`). It also could not tell a depot-side start from a transport-side one. Both filed defects are closed by the binding, and ROUTE-ORIGIN-PROOF-TRANSPORT-MANIFESTS-INCLUDE-THE-DEPOT is closed for the origin proof: the bind re-scopes the start and end manifests to the transport half. `RouteRunCargoManifest` keeps the merged scope and is a separate producer.

Cost calculation:

For v1, `CostManifest` is the positive decrease in the source transport's non-ignored resources over the full Supply Run, computed over the original transport part PID set rather than the aggregate docked vessel:

```
CostManifest[resource] = max(0, startTransportResources[resource] - endTransportResources[resource])
```

This includes delivered cargo plus transit resources consumed by the flown mission. `InventoryCostManifest` uses the same principle for exact stock stored-part payload snapshots that leave the transport by the end of the run.

Implemented v0 note (verified, 19.7 item 4): the shipped code deviates from this section in two ways. `RouteBuilder` copies the connection window's delivery manifest into `CostManifest` (not the full-run decrease above), and the funds charge (`RouteOrchestrator.ComputeDispatchFundsCostForRoute` → `RouteFundsCalculator.ComputeDispatchFundsCost`) walks the FIRST source recording's end-of-recording vessel snapshot (the dock chain boundary for a `[launch..dock]` run), so its resource term is dock-arrival contents — the launch load minus pre-dock transit burn, not "delivered plus consumed". Under flow accounting (19.2.4) `CostManifest` is replaced by per-provenance terms, and the charge basis is decided in M2.

For KSC-origin Career routes, `KscDispatchFundsCost` is:

```
dry source transport part cost
+ stock resource unitCost * CostManifest amount
+ dry stock part cost for InventoryCostManifest items
```

Dry part cost means the source transport's parts with stored resources emptied and inventory contents excluded, so resource and inventory values are not double-counted. Inventory-contained parts use `InventoryPayloadItem.UnitDryCost` plus `StoredResources` from their captured `STOREDPART` snapshot, so a solar panel variant, a stacked spare wheel, and a resource-filled stored part are costed from the same payload identity that delivery will reconstruct. v1 does not apply recurring recovery credit.

Routes store `CostManifest` and `KscDispatchFundsCost` together for transparency and future re-costing. On KSC-origin routes, `CostManifest` is not physically deducted; `KscDispatchFundsCost` is derived from that same source data. If serialized values disagree after load/revalidation, treat that as a bug and recompute from the source recording rather than honoring the desync.

5.3 Validation

The route analysis pass validates the committed tree/path:

1. The committed tree contains route-proof metadata from section 4.10. If not, reject with "recording lacks logistics proof data."
2. At least one completed docking connection window exists (`TransferTargetVesselPid != 0` and `TransferKind == DockingPort`)
3. The tree branch structure has a dock merge and matching later undock/split boundary for that window
4. At least one resource or exact stock stored-part payload both left the original transport part PID set and appeared on the original endpoint part PID set between the dock and undock snapshots
5. The derived source recording path is present, playable, and has source refs/fingerprints that can be revalidated later
6. v1: exactly one delivery window is present. If two or more dock-transfer-undock delivery windows are detected, validation rejects the candidate and reports that multi-stop Supply Routes are deferred
7. v1: no pickup deltas are present in the same connection window. Formally, after EC/IntakeAir filtering, any positive `transportGain` or `endpointLoss` rejects the candidate as pickup or mixed transfer
8. v1: non-KSC origins are eligible only when the recording starts docked to a real origin depot vessel and captures that vessel PID; non-KSC candidates without that proof are rejected

If validation fails, the route confirmation UI shows what's missing (e.g., "Transport must undock from destination to enable route — the endpoint needs to be free for the next cycle").

5.4 Player confirmation

Route configuration panel shows derived values: origin, endpoint, delivery manifest, total transit time, origin cost manifest, KSC dispatch funds cost, and connection kind. Player can edit name, dispatch interval, and enable/disable. On confirm, route is created and scheduling begins.

Default name. A route created without a name (every Logistics-window create; a seam `RouteCommand action=create` without `name=`) is named after the mission it repeats: the source tree's original mission's name, else the tree's name (`RouteCreationService.ResolveSourceMissionName`). When a stored route (committed or dormant) already carries that name, the new route takes the first free `"Name [k]"`, k = 2, 3, ... (`RouteCreationFormatters.NumberRouteName`, the `MissionVesselNaming.FormatNumbered` brackets), compared ordinally and independent of route order, so the first route of a mission keeps the bare name and a second reads `"Duna Supply 1 [2]"`. Only a mission with no usable name falls back to `"Route: <origin> -> <endpoint body>"` (`GenerateDefaultRouteName`, unnumbered). The Candidates row's Route cell shows the same resolved name. The name is fixed at creation: a stored route's name is never rewritten on load or when a mission is renamed, and an explicit name is taken verbatim.

5.5 Multi-stop routes

Status: multi-stop DELIVERY SHIPPED in M4a (see 19.4 M4). The text below is the original v1 deferral rationale, retained for design context; it is superseded by M4a for the delivery case. Multi-stop routes were a natural extension not required for v1. The analysis

engine can detect multiple dock-transfer-undock delivery windows and report them in diagnostics; v1 rejected the route candidate instead of exposing only the first stop, to avoid charging/deducting the full Supply Run cost while delivering only part of the run and to keep partial-failure semantics out of v1. M4a lifts that reject for DELIVERY: the analysis accepts + orders N windows, the route builds N stops, and the orchestrator fires each delivery at its own recorded dock phase under one cycle (the full-run cost is charged once at dispatch, each stop delivered when the ghost reaches that dock — no partial-charge problem). Multi-window PICKUP / multi-origin loads remain M4b.

Multi-stop example:

```
Player flies: KSC -> Base A (dock, deliver 150 LF) -> Base B (dock, deliver spare parts) -> KSC

Future route analysis produces:
  Origin: KSC, Kerbin
  Stop 1: Base A on Mun -- delivers 150 LF, 183 Ox
  Stop 2: Base B on Mun -- delivers spare parts
  Total transit: 2d 4h
  Round-trip: Yes
```

---

6. Dispatch and Delivery

6.1 Dispatch evaluation

Trigger: The route scheduler (`RouteScheduler`) runs each physics frame (or once per second during warp) in all scenes via `RouteOrchestrator` , called from `ParsekScenario.Update` .

v0 status (items 5+6): The entrypoint is `RouteOrchestrator.Tick(currentUT)` , called from `ParsekScenario.Update` with a UT-delta accumulator at `RouteOrchestrator.TickIntervalSec` cadence (`RouteTickClock.Advance` ; the baseline is per scenario instance and re-seeds when the clock moves behind it, as the go-back rewind does one frame after the Space Center's first Update). Dispatch and delivery are end-to-end for KSC-origin single-stop routes: the dispatch-decision chain emits `RouteDispatched` + `RouteCargoDebited` (and `RouteEndpointLost` on resolution failure); when `PendingDeliveryUT` elapses the pre-evaluator delivery hook applies the manifest to the destination vessel (loaded `PartResource` writes or unloaded `ProtoPartResourceSnapshot` writes for resources; `ModuleInventoryPart` slot stores for inventory), debits Career KSC-origin funds via `Funding.Instance` , and emits `RouteCargoDelivered` with actual-vs-requested amounts. Idempotent against ELS replay via `(RouteId, RouteCycleId)` .

For each route with Status in {Active, WaitingForResources, WaitingForFunds, DestinationFull} and `NextDispatchUT <= currentUT` :

Routes with Status Paused, EndpointLost, MissingSourceRecording, or SourceChanged are excluded from dispatch evaluation. `InTransit` routes are processed by the UT-driven progression loop before dispatch evaluation.

For `WaitingForResources` , `WaitingForFunds` , and `DestinationFull` , skip evaluation until `NextEligibilityCheckUT == null || NextEligibilityCheckUT <= currentUT` . The skipped guard logs a `VerboseRateLimited` dispatch backoff line. When a wait condition is found, set `NextEligibilityCheckUT = currentUT + 60s` by default. Save/load preserves the value. Route UI actions, resource-store changes, funds ledger changes, endpoint capacity changes, and route revalidation may clear it to force an immediate retry.

`MissingSourceRecording` and `SourceChanged` routes are not retried as normal dispatch candidates. They are revalidated on save load, when the recordings store changes, and from a route UI "Revalidate" action. If every source recording ID resolves and every source ref/fingerprint still matches, the route returns to `Active` and `NextDispatchUT` is recalculated from current UT. If the recordings exist but their route-relevant fingerprint changed, the route stays `SourceChanged` and must be recreated or explicitly reanalyzed from the current source data.

Step 1: Ignore reserved round-trip fields in v1. `LinkedRouteId` is serialized for forward compatibility only. v1 dispatch does not check it.

Step 2: Check source recordings. Verify every `RecordingIds` entry still resolves to a committed recording and every `SourceRefs` fingerprint still matches. If an ID is missing, set `Status = MissingSourceRecording` and stop. If an ID exists but route-relevant timing/proof data changed, set `Status = SourceChanged` and stop. No matching proof recording means no cargo transfer.

Step 3: Check destination. Find the endpoint vessel (section 7). If NO vessel is found → set `Status = EndpointLost` , stop this dispatch attempt. If the vessel has zero capacity for all delivery resources and inventory items → set `Status = DestinationFull` , set `NextEligibilityCheckUT` , and do NOT advance `NextDispatchUT` ; the same scheduled dispatch becomes eligible as soon as capacity appears. If capacity is available → proceed.

Step 4: Check origin. For non-KSC origins, find the start-docked origin depot vessel (section 7). If no vessel is found → set `Status = EndpointLost` and stop this dispatch attempt. If the vessel lacks `CostManifest` resources or `InventoryCostManifest` items → set `Status = WaitingForResources`, set `NextEligibilityCheckUT`, and do NOT advance `NextDispatchUT`. For KSC origins in Career, check that `KscDispatchFundsCost` is affordable under the existing ledger reservation rules (AS BUILT 2026-10-21, todo ROUTE-DISPATCH-IGNORES-RESERVATIONS: `LiveRouteRuntimeEnvironment.KscFundsAvailable` gates on the smaller of the live counter and `LedgerOrchestrator.CanAffordFundsSpending`'s reservation-aware amount; before that it read the live counter only). If funds are insufficient, set `Status = WaitingForFunds`, set `NextEligibilityCheckUT`, and do NOT advance `NextDispatchUT`. For KSC origins in Science or Sandbox, skip the funds branch entirely; do not touch `Funding.Instance`.

Step 5: Dispatch. Clear `NextEligibilityCheckUT`. Create the ROUTE_DISPATCHED ledger event, then apply the origin debit/funds charge through the route ledger/applier path. Deduct `CostManifest` / `InventoryCostManifest` from non-KSC origin, or charge `KscDispatchFundsCost` for KSC origin in Career. Science and Sandbox KSC origins pay no funds. Set `CurrentCycleStartUT` to the scheduled dispatch UT being processed (`NextDispatchUT`), set `CurrentSegmentIndex = 0`, compute `PendingDeliveryUT = CurrentCycleStartUT + nextStop.DeliveryOffsetSeconds`, and set `PendingStopIndex` to the stop due at that boundary or `-1`. Tell the ghost playback engine to play the first recording in `RecordingIds` when visuals are available. Set `Status = InTransit`. Advance `NextDispatchUT`.

Implemented gate order: `RouteDispatchEvaluator.CheckEligibility` evaluates sources → endpoint resolution (every stop + non-KSC origin) → origin cargo → Career funds → destination capacity - destination LAST, unlike Steps 3-4 above; only the first-failure reason order differs from this section. **Destination capacity is ALL-OR-NOTHING as of the post-M6 destination-capacity gate** (the v0 always-true `DestinationHasCapacity` stub is retired): every stop's full delivery manifest (resources AND stored-part inventory slots) must fit its resolved destination, evaluated with the same planner+probe the delivery applier uses (`RouteDestinationCapacityCheck` over `RouteDeliveryPlanner.PrepareDelivery` + `LiveDeliveryCapacityProbe`), or the route holds `DestinationFull` naming the first item that does not fit (bare resource name, or `stored-part:<partName>` for an inventory slot shortfall). Rationale: dispatch debits the origin for the full manifest and the transport is a ghost, so a partial delivery would silently lose the remainder. The apply-time per-resource clamp remains as the backstop for capacity that shrinks between the gate and the write; such a partial delivery records `Route.LastPartialDeliverySummary` / `LastPartialDeliveryUT` / `LastPartialDeliveryCycleId` (same-cycle partial windows append into one report; a full window clears the report only when it belongs to a LATER cycle, so a multi-stop cycle's later full window cannot erase an earlier window's recorded loss) and the Logistics detail panel shows "Last delivery was partial: ...". Stops resolving to the SAME destination vessel share one capacity probe across the gate walk, so their combined manifest is checked against the tank, not each window against the full free capacity. An unresolvable stop vessel fails OPEN at this gate (the endpoint check owns that failure).

Per-tick catch-up loop: `RouteOrchestrator.Tick(currentUT)` must process one route until it reaches a stable state for the current UT: first progress any `InTransit` boundaries due at or before `currentUT`, then if the route is `Active` and `NextDispatchUT <= currentUT`, run dispatch evaluation, then immediately progress any newly dispatched zero/short-duration boundaries that are also due. Repeat until the route is no longer due, enters a waiting/error status, or hits a defensive max-iterations guard with a warning. This is what makes "warp past multiple cycles" deterministic; a single tick may dispatch, deliver, complete, dispatch again, and then stop on an origin/funds/capacity blocker.

6.2 UT-driven chain progression

The route scheduler, not ghost playback, is authoritative for route state. `RouteOrchestrator.Tick(currentUT)` processes in-transit routes in all scenes and during time warp:

1. If any source recording is missing, set `Status = MissingSourceRecording`, stop playback if active, and abort without delivery. If a source exists but no longer matches `SourceRefs`, set `Status = SourceChanged`, stop playback if active, and abort without delivery.
2. While `Status == InTransit` and `PendingDeliveryUT <= currentUT`, process the due boundary.
3. If `PendingStopIndex >= 0`, execute delivery for that stop using current endpoint capacity.
4. Advance `CurrentSegmentIndex` to the next 0-based source recording index and compute the next `PendingDeliveryUT = CurrentCycleStartUT + stop.DeliveryOffsetSeconds` / `PendingStopIndex`.
5. If the last source recording boundary was processed, increment `CompletedCycles`, reset `CurrentCycleStartUT = null`, `CurrentSegmentIndex = -1`, `PendingDeliveryUT = null`, and `PendingStopIndex = -1`. If `PauseAfterCurrentCycle` is true, clear it and set `Status = Paused`; otherwise set `Status = Active`.

`OnPlaybackCompleted` remains a visual integration hook only. It can let the route scheduler start the next ghost source recording promptly in flight, but it must not be the only path that advances route state. Save/load and high time warp are handled by the UT-driven tick loop.

Routes do NOT use the per-recording loop toggle. The route scheduler owns all timing and sequencing. Routes and loops are siblings — both use the ghost playback engine, but routes chain multiple trajectories sequentially with delivery logic between

segments.

Visual handoff when entering flight mid-transit: The scheduler elapsed time is `currentUT - CurrentCycleStartUT`. When visuals become available for an already in-transit route, the route policy resolves that elapsed time to a `RecordingIds` index and a recording-local offset, then starts/seeks ghost playback at that offset. It must not restart the ghost at recording UT 0 after the route has already advanced. If the current cycle has no remaining visible source recording to show, the route simply skips ghost rendering until the next source recording or cycle; delivery state still advances by UT.

6.3 Per-stop delivery execution

Trigger: `RouteOrchestrator.Tick(currentUT)` reaches a pending stop boundary. In flight, the same moment normally coincides with ghost source-recording completion; outside flight, delivery still happens by UT.

1. **Re-check source recordings.** If any `RecordingIds` entry is missing, set `Status = MissingSourceRecording`, create no delivery event, and abort. If any `SourceRefs` fingerprint no longer matches, set `Status = SourceChanged`, create no delivery event, and abort. No matching proof recording means no cargo transfer, even mid-transit.
2. **Find endpoint vessel** at the route endpoint (section 7). If no vessel is found after dispatch, log a warning and create a ROUTE_DELIVERY_FAILED timeline event. Transit cost has already been paid; no cargo is conjured.
3. **For each resource in the stop's `DeliveryManifest`:** apply to endpoint vessel tanks, clamped to current `maxAmount`. v1 manifests contain positive delivery amounts only. For loaded vessels: pre-check `PartResource.flowState` / `flowMode`, then use `Part.TransferResource()` on the exact target `PartResource`. For unloaded vessels: modify `ProtoPartResourceSnapshot.amount` or its backing `RESOURCE` node through stock snapshot objects, preserving `flowState` / `flowMode`.
4. **Deliver inventory** by reconstructing exact `InventoryPayloadItem.StoredPartSnapshot` payloads into stock `ModuleInventoryPart` slots. Loaded delivery may call `StoreCargoPartAtSlot(ProtoPartSnapshot, slot)` only after logistics has chosen a valid slot/stack target. Items that do not fit remain undelivered and are reported in the route event/log.
5. **Create ROUTE_DELIVERED timeline event and apply through the ledger/applier path.** Record requested and actual amounts so the player can see partial fills instead of silent loss. The persisted event must contain enough target identity and before/after delta data for epoch recomputation or rollback to undo the physical cargo mutation.

6.4 Single-delivery execution

For v1, the single-stop route is the only player-facing shape. Delivery executes once when scheduler UT reaches the recorded boundary after `SegmentIndexBefore`.

6.5 Per-resource independent delivery

Each delivery resource at each stop is evaluated independently:

```
For each resource in stop.DeliveryManifest:
    capacity = sum of (maxAmount - amount) across all stop vessel tanks for this resource
    deliver = min(manifest amount, capacity)
    actualDelivery[resource] = deliver
```

AS BUILT 2026-10-07, "all stop vessel tanks" means the endpoint's OWN tanks. When the resolved vessel is a docked composite, the capacity sum, the delivery writes, the origin / pickup cargo gates and the debits all walk only the parts `RouteEndpointPartScope` assigns to the endpoint (section 7.1, "Which parts of a docked composite are the endpoint's"); a craft docked to the station is neither counted nor filled nor drained.

Origin cost is paid at dispatch time after destination capacity has been checked. Non-KSC origins deduct `CostManifest` / `InventoryCostManifest` from the proven origin depot. KSC-origin Career routes charge `KscDispatchFundsCost` instead of deducting physical cargo from KSC. Science and Sandbox KSC origins dispatch with no funds charge.

Inventory follows the same independent rule at item granularity: deliver what fits, report what did not fit, never create extra slots or abstract storage.

6.6 Pause, unpause, and re-target

Pause: Player clicks Pause in route UI → `Status = Paused` for future dispatches. If a dispatch is already `InTransit`, set `PauseAfterCurrentCycle = true` and keep `Status = InTransit` until the cycle finishes. Delivery still executes at the endpoint; after the final boundary is processed, the route transitions to `Paused` instead of `Active`. This matches stock expectations: the supply vessel has already launched / departed, so pausing the route should not freeze cargo in mid-flight.

Pause/resume timeline rows (route-timeline events, SHIPPED): the two player-driven pause transitions emit a free-standing `RoutePaused` ledger row — the immediate player Pause (reason `player-pause`) and the armed pause-after-cycle completion at the delivery applier (reason `delivered-then-paused` / `delivered-partial-then-paused`) — and a player Activate emits a `RouteResumed` row (reason `player-activate`), so the ledger timeline carries the route's pause history. The `delivered-replay` idempotency branch honors an armed pause too: when `PauseAfterCurrentCycle` is set on the crash-recovery replay, the route consumes both one-shot flags, transitions `Paused` (reason `delivered-replay-then-paused`), and emits the `RoutePaused` marker the crash lost (the only new row; the delivery/funds dedup is unchanged). Auto-flip rows (SHIPPED follow-up): a LIVE `RevalidateSources` pass also emits `RoutePaused` (reason `AutoPause:MissingSourceRecording` / `AutoPause:SourceChanged`) when a route flips INTO a source-problem state, and `RouteResumed` (reason `AutoResume:SourcesRestored`) when recovery restores an Active-family status from `PreMissingStatus` (a restored `Paused` emits nothing - it never resumed). Emission is caller-gated (`RevalidateSources(reason, liveEmitUT)`, default -1 = silent): OnLoad revalidation passes stay silent (ledger writes during load are unsafe and load-time flicker would spam rows). Live player-driven ERS mutations emit through `ParsekScenario.BumpSupersedeStateVersionLive` - the re-fly merge commit, tree-discard purge, Re-Fly discard, revert Retry/Discard handlers, and unfinished-flight Seal/Stash - which resolves the UT defensively and forces silence while OnLoad is on the stack (a scene-change load can report a stale nonzero UT); the dormant-materialize pass passes its live tick UT directly. Deliberately-silent flips (the merge-journal finisher's crash-recovery re-drive, the mid-rewind supersede rollback, any load pass) are repaired by the CATCH-UP net: each live pass emits `RouteResumed` (reason `AutoResume:CatchUp`) for any Active-family route whose latest kept lifecycle row still reads paused, so pause history self-heals; idempotent via the latest-row check. Still not emitted (deliberate): routes created `Paused` (creation is `Route.CreatedUT`, not a pause event) and any transition whose live UT cannot be resolved (skip with Warn; a bogus-UT row would survive every rewind cutoff). All rows are retired at a rewind whose cutoff precedes them (`RouteLedgerRetire`), matching the `.sfs` status revert. A Send Once one-shot does NOT emit a resume row; its `RouteDispatched` row carries the sparse `routeSendOnce` marker instead, and the automatic post-delivery `RoutePaused` row closes the bracket around the individual run. `Route.CreatedUT` stamps the creation point at `RouteStore.AddRoute` time.

Unpause: Player clicks Resume → `Status = Active`. If the route is still `InTransit` with `PauseAfterCurrentCycle = true`, Resume clears that flag and the in-flight cycle will finish back to `Active`. Route re-enters dispatch evaluation on next scheduler tick. `NextDispatchUT` is recalculated if stale (advanced to next valid dispatch time from currentUT).

Route existence window across a rewind (dormant routes, SHIPPED): a route exists on the timeline from `Route.CreatedUT` forward. A Rewind-to-Separation whose cutoff precedes a route's creation moves it (via `ReconciliationBundle.Restore(cutoff)` → `RouteRewindClassifier.Classify` → the sparse `DORMANT_ROUTES` store list) into a DORMANT state: non-firing, non-tree-binding, definition preserved, listed only in the Logistics window's collapsed "Dormant Routes (N)" disclosure (name + the date it reappears + Delete via `RouteStore.RemoveDormantRoute`; read-only otherwise). When the re-flown timeline reaches the route's `CreatedUT` again, `RouteStore.MaterializeDueDormantRoutes` (orchestrator tick) re-materializes it Paused with fresh cycle state (player re-activation is a recorded `RouteResumed` event); a route whose source tree got occupied by a newly created route is dropped (live intent wins), and one whose sources were superseded surfaces as `MissingSourceRecording`. Routes created BEFORE the cutoff stay committed but get their forward-looking cycle state reconciled (`ResetCycleStateForRewind`) so the loop clock re-fires re-flown cycles instead of comparing against abandoned-future cursors, and then take their loop position back from the loaded save's own route copy so the crossing that passed just before the cutoff is not fired a second time (10.6, "Loop position"); the same seam then derives each kept route's timeline-correct pause state from the kept PLAYER-DRIVEN `RoutePaused` / `RouteResumed` rows (`DeriveTimelineStatus` - latest kept player marker wins; AUTO rows with `AutoPause:` / `AutoResume:` reason prefixes are skipped because they describe source validity, which re-derives from ERS after every rewind, and a derived resume requires at least one kept player `RoutePaused` row (a resume must resume something recorded); a derived pause flips only the ghost-driving/wait statuses, a derived resume only un-pauses an explicitly `Paused` route, `InTransit` and no-marker routes are never touched, and a validity status (`MissingSourceRecording` / `SourceChanged` / `EndpointLost`) keeps its live status while the verdict lands on `PreMissingStatus` so a later recovery restores the timeline-correct state), unconditionally clears `PauseAfterCurrentCycle` / `SendOnceArmed` (armed one-shots carry no timestamp, so they cannot survive time travel - the player re-arms trivially, while a phantom arm cannot be traced), and reconstructs `CompletedCycles` / `SkippedCycles` from the kept dispatch/delivery rows (`ReconstructCycleCounters`; a cycle dispatched but undelivered at the cutoff counts as skipped, keeping the cycle-id uniqueness invariant `Completed + Skipped > maxKeptOrdinal` - except when the route retained a kept pre-cutoff in-flight cycle, where the sum lands ON that cycle's ordinal so the straddling cycle keeps its id and its already-delivered windows dedup against the kept rows). Legacy routes without a `CreatedUT` stamp never go dormant. BOTH rewind exits run this reconciliation through the shared `RouteRewindClassifier.ReconcileStoreAtRewind` helper: the Re-Fly seam (`ReconciliationBundle.Restore(cutoff)`) and the plain go-back rewind (`ParsekScenario.HandleRewindOnLoad`, which additionally retires the abandoned-future free-standing route ledger rows in place via `Ledger.RetireFutureRouteActionsAtRewind` at the rewind save's own UT, before the lead-time windback (10.6, "Who runs it, at which cutoff") - the go-back path preserves the in-memory ledger, so without the retire the Rec-1 rows would survive). See `docs/dev/plans/plan-route-rewind-dormant-visibility.md`.

Cancel current dispatch: Deferred. If added later, cancellation should be explicit and should not refund already-deducted origin cargo unless the route event model explicitly records a reversible failure.

Re-target (EndpointLost recovery): Player selects a new destination vessel in the route UI → endpoint coordinates and vesselPersistentId updated, `Status` transitions from `EndpointLost` to `Active`. Same mechanism for origin re-targeting on non-KSC routes. Re-targeting is an explicit player intent declaration, not automatic proof that the original Supply Run visited the new vessel. The route keeps its recorded delivery and cost manifests, logs the re-target, and the UI should warn that future cycles reuse the proven transport/cargo run against the newly selected endpoint or origin. Automated surface fallback remains deliberately tight; broad endpoint generalization requires this explicit player action.

6.7 Hold history rows (RouteHeld)

A held run (the dispatch evaluator's first failing gate: origin short, funds short, destination full, endpoint lost on the crossing, flights unavailable, waiting for the linked route) writes a free-standing `RouteHeld` ledger row (`GameActionType` 36) so the Route History can show when and why a route stopped delivering. The row carries `RouteId`, the held cycle's `RouteCycleId` (diagnostic only), the evaluator detail token in `RouteEndpointReason`, the failure kind by name in `routeHoldKind` and the measured shortfall in `routeHoldShortfall` (both sparse, written only by this type). Adding the type changed no existing key or row, so the recording schema generation is unchanged. Downgrade: an older build reading a `RouteHeld` row logs one `Unknown action type id` Warn per row and keeps it as an inert zero-value row (the default type, `ScienceEarning`, with no subject and no amount), so no pool moves.

One row per hold episode and reason. A route can be re-checked tens of thousands of times a year while held (the loop path checks once per window crossing, the legacy wait path every 30 s), so the row is not per check. The writer (`RouteOrchestrator.TryEmitRouteHeldRow`, called right after `Route.RecordHold` at the single-stop and multi-stop loop blocked branches and at the legacy wait applier) writes only when the route's OPEN episode has no `RouteHeld` row with the same (kind, detail) pair. The open episode is the route's rows after its latest `RouteDispatched` / `RoutePaused` / `RouteResumed` / `RouteEndpointLost` row, walked in time order with ledger order breaking ties (`IsHoldReasonRecordedInOpenEpisode`). The detail is made stable first (`HeldRowDetail`): a funds hold is always `funds-short` with the amount in `routeHoldShortfall` (the legacy wait's `funds-shortfall-N` token moves with the amount), and a linked-route wait is `partner:<partner route id>` (the evaluator token names the partner, so a rename would read as a new reason); the Route History resolves that id to the partner's current name. So a route held for a year on one reason writes one row; a reason change (origin short, then destination full) writes a second; a flip-flop between two reasons inside one episode writes two rows, not one per crossing; a shortfall-only change writes nothing. Nothing marks a release: the next Sent, Paused, Activated or Stopped row closes the episode. The endpoint-lost appliers write no Held row, since `RouteEndpointLost` is already that row; an endpoint loss caught at a loop crossing (no status change) does.

The check reads the Effective Ledger Set, never `Route.LastHold*`. The rewind reconcile clears the route's hold fields when the last hold is past the cutoff while the episode's pre-cutoff `RouteHeld` row survives, and a stock revert does not touch the fields at all; a field compare would write a duplicate row after a rewind into a long hold. Reading `EffectiveState.ComputeELS()` (memoized) keeps a re-hold after a rewind at exactly one row per episode and reason.

Inert everywhere else. No resource module consumes the row (`RouteModule` ignores it, `IsResourceImpactingAction` is false), supersede never blocks on it (an explicit exclusion arm in `SupersedeCommit`, whose default would strict-block), every dispatch / pickup / delivery dedup and `ReconstructCycleCounters` is type-filtered so the row's cycle id cannot satisfy or skew them, and the career Timeline hides it with the other route rows. Rewind retires it with them (`RouteLedgerRetire.IsRouteActionType`, strict `>` cutoff); a stock revert drops it through `Ledger.PruneOrphanActionsAfterUT`. Old saves get no backfill: a route's history shows holds from the build that writes them on.

What the row can say. It names only the FIRST failing gate, so a destination that is also full stays hidden behind an origin shortage until the origin refills. The Route History renders it as `Held: <reason>`: the Logistics window's hold sentence (`LogisticsHoldPresentation.DescribeHold`) without its live-route advice, stored parts by title, no run number (`DescribeHoldForHistory`); `docs/dev/design-gui-inventory.md` section 3.7 lists the rows. Logging: Info `RouteHeld: route <id> kind=<K> detail=<D> shortfall=<S> cycle=<C> ut=<UT> - hold row written` per row, `VerboseRateLimited` `hold unchanged, skipped` otherwise.

---

7. Endpoint Resolution

7.1 Algorithm

`RouteEndpointResolver.SurfaceFallbackRadiusMeters = 50.0` in v1. The radius is deliberately tight: it allows rebuilding a surface base in-place, but avoids treating neighboring pads, rovers, or storage craft as one abstract warehouse. If this proves too restrictive for large surface installations, make it a settings-backed value later rather than silently widening v1 matching.

AS BUILT: the shipped constant is `RouteOrchestrator.SurfaceProximityRadiusMeters = 500.0` (tuned down from 2000 m after the v0 playtests; it tolerates terrain settling and floating-origin drift while a depot driven away from its dock spot still loses the route). The 50 m figure above was the pre-implementation proposal. **AS BUILT 2026-10-07, the proximity step is skipped while the**

endpoint is chain-ghosted (todo ROUTE-ENDPOINT-CHAIN-GHOST-PROXIMITY-REBIND). A base a committed dock claims is despawned by a FLIGHT load after a rewind (`VesselGhoster` , while the tip spawn UT is ahead) and stays gone in every scene until its chain tip respawns it with its identity; the root-part and pid steps miss for that span, and a proximity transfer would hand the route to any craft parked within 500 m for good (the respawned base then loses to the neighbour's root part at the first step). `RouteEndpointChainHold` holds the endpoint instead: when a non-terminated ghost chain claims the endpoint's pid (launch guid not conclusively different) and its tip spawn UT is still ahead, or the flight still lists the chain as waiting for a blocked tip spawn, the resolver returns `endpoint-chain-ghosted` without searching or rebinding. Per scene: in flight the hold covers the despawned base; at the Space Center and in the Tracking Station a base no flight load has despawned is still live in its pre-claim form (those scenes never despawn it; `ChainTipStaleVessel` replaces it when the tip spawn UT passes), so the root-part or pid step finds it and the hold is never reached, while a base a flight load DID despawn is absent there too and is held. The claims are read from the committed trees rather than flight state for exactly that last case. The route then holds its crossings like any route with a missing endpoint (a `RouteHeld` row of kind EndpointLost; the hold line reads "destination is a ghost until a recorded flight that docks with it ends - deliveries resume when it is back") and resolves by identity again once the base is back. A genuinely lost endpoint - no chain, or a terminated one - still transfers to a nearby craft as described above.

KSC-origin routes do not resolve an origin vessel: `IsKscOrigin == true` and `Origin.vesselPersistentId == 0` mean "charge/skip KSC funds branch according to game mode." A route stop endpoint with `vesselPersistentId == 0` is invalid.

AS BUILT 2026-09-02 (P12), a non-KSC origin with `vesselPersistentId == 0` is NOT invalid. A start-docked origin's pid is unknowable at capture (`Part.Couple` destroys the absorbed half's `Vessel`) and is stamped at the undock bind only when the launch-guid gate clears it, so a legitimate origin routinely carries pid 0. Its identity is `RouteEndpoint.RootPartUId` - the bound half's root part `flightID` , which is launch-unique where a craft-baked `persistentId` is not - and `RouteEndpointResolver` tries the ROOT-PART step FIRST, ahead of the pid and ahead of surface proximity. The invalid case is an origin with neither a root id nor a pid.

AS BUILT 2026-09-06 (P17), EVERY endpoint carries a root id - not only origins - and the root-part step sees through a dock. Two changes that only work together (`ROUTE-ENDPOINT-TRANSFER-DOCKED-DOMINANT-PARTNER`). (1) A DELIVERY endpoint is stamped with its own root part `flightID` at capture: `ParsekFlight.BuildRouteEndpointFromVessel` reads it pre-couple, `TryBuildRouteEndpointFromSnapshot` reads it from the snapshot's `root` index, and the window-build site backfills from the `endpointRootPartUId` it already computes. A destination stop's walk therefore starts at identity instead of at pid → proximity. (2) `RouteEndpointResolver.TryRootPartMatchPure` runs TWO passes: own-root first, then PART-SET-CONTAINS. `Part.Couple` merges into the DOMINANT half's `Vessel` (`Vessel.GetDominantVessel` : higher `VesselType` , then heavier, then larger guid), so while a dominant visitor is docked to a base the base is no vessel's root and its own pid is gone from `FlightGlobals` ; a part `flightID` is launch-unique, so the vessel CARRYING it is the craft that physically holds the recorded endpoint. Pass order is asymmetric on purpose - an own-root match beats a contains match everywhere - so the mirror direction (the destination dominating the merge) resolves through pass 1 unchanged. The composite arm logs `Endpoint resolved through a DOCKED COMPOSITE:` and does NOT rebind: a root-part hit is the recorded endpoint by identity, so the transfer - and with it the transport exclusion, which is a proximity-step guard against positional SUBSTITUTION - is never reached. Both `RouteCodec` and `RouteProofCodec` read `rootPartUId` back (the proof-side reader was missing before this pass); the field stays out of the proof hash.

AS BUILT 2026-10-07: which parts of a docked composite are the endpoint's (`ROUTE-DELIVERY-INTO-DOCKED-VISITOR`). The resolver returns the whole composite, and until this pass every writer and probe walked all of it, so a visiting tanker docked to the station filled first and left with the cargo, the capacity gate counted its tanks, and an origin debit could drain a docked visitor instead of the depot. `RouteEndpointPartScope` now decides which parts a route may touch. It cuts the composite the way an undock would: at every settled stock dock seam, i.e. a `ModuleDockingNode` (or a `ModuleGrappleNode` claw) carrying stock's `vesselInfo` whose `dockedPartUId` names its own parent or child part (`DockToVessel` sets both and `Part.Couple` makes the two port parts parent and child; `Undock` detaches exactly the child side). It then keeps the pieces holding the endpoint's root part or any part the route RECORDED as the endpoint's: the connection window's `EndpointPartPersistentIds` (matched to the endpoint by root part `flightID` , by the target pid only when the endpoint carries no root) and, for a start-docked origin, the depot side of the pair seam split out of the recording's start snapshot. Stock's records alone cannot decide this - a module docked before the route was recorded and a visitor docked after leave identical `DockedVesselInfo` pairs and no time - so the recorded part set is the discriminator. Consequences: both merge directions, and an endpoint that is itself docked INTO a larger station, use only the endpoint's parts; a station assembled from modules docked before the recording stays whole; a module docked AFTER the recording is excluded (cargo stays in the parts the route was proven against); an undocked endpoint is untouched. Fallback: with no recorded part set, or none of it aboard, the whole composite is used as before and an Info line names the reason (`Endpoint part scope undetermined: ... outcome=no-recorded-parts|endpoint-not-aboard` , rate-limited). The scope is built once per probe / writer bundle on the same loaded / unloaded branch the bundle reads and is threaded into all five live readers and writers; the delivery, debit and plan log lines carry `parts=whole` or `parts=own=N/M` . A gate that shares one probe across a route's stops (the destination capacity gate and the Logistics window's replay of it) keys it by vessel pid PLUS scope (`EndpointScopedCache`), and the pickup gate groups sources the same way (`PickupSourceResolution.PartScopeKey`): two stops whose endpoints were later docked into one composite are gated against the parts their own writers touch, while two windows to the same endpoint still share one tank.

Adopted endpoint parts (owner ruling 2026-10-07): the route's `Update parts` button. Because stock records no dock time, a module the player permanently docks to a station after recording its route is indistinguishable from a visitor and is excluded above.

The route detail block's `Update parts` single (both modes; greyed with its reason while a run is under way - a self-timer run in transit or pending delivery, or a multi-stop loop cycle that has fired some stops but not all, Paused included (Send un-pauses the route WITHOUT resetting the stop cursors, so the next tick finishes that cycle on its pre-pause dispatch; Activate resets them, so the button is never stuck) - because that run's cargo was gated against the part sets it dispatched with; `docs/dev/design-gui-inventory.md` section 3.6) calls `RouteEndpointPartAdoption.AdoptCurrentParts(route)`, which lets the player opt such modules in: for the origin (unless the route is KSC- or harvest-origin) and every stop, it resolves the endpoint with the same `RouteEndpointResolver.TryResolveEndpoint` the deliveries use, reads the resolved vessel's part and dock-node records from the store the writers use (live parts when loaded, the proto snapshot otherwise), and stores the `Part.flightID`s the pure `DecideCapture` allows in `Route.AdoptedEndpointParts`. An undocked vessel is taken whole. On a docked composite the press follows a SIZE rule the player can predict: the endpoint's own pieces, as the scope resolves them today (the piece with its root plus the pieces with its adopted, else recorded, parts), are always kept, and their part count is the endpoint's own size; with those pieces cut out of the part tree (the dock seams between the remaining pieces kept), what is left falls into foreign clusters, one per docking point (everything reachable through it), and a cluster is taken only when it has strictly FEWER parts than the own size. So a station takes the smaller modules docked to it (three new modules are three clusters), while a lander stop parked at a station sees the whole station, modules included, as one cluster and leaves it out: adopting it would make the host's tanks its own, so the lander's cargo would fill the station and a depot module's debits drain it. Parts already adopted count toward the own size. Part count, not mass, because fuel changes mass; and not the composite's root, because `Vessel.GetDominantVessel` ranks vessel TYPE first (Debris < SpaceObject < Unknown < Probe < Relay < Rover < Lander < Ship < Plane < Station < Base, mass only on a tie), so a crewed lander docked at a probe-cored depot roots the composite. A cluster with as many parts as the endpoint, or more, is left out (`leftOut=` in the log line): a tie cannot tell a host from a module, and a module larger than the station it docks to is a documented limit (it stays out of the station's set and gets no cargo; stock records nothing that tells it from a host). An endpoint whose every cluster is left out re-adopts exactly its own pieces, which is not a refusal; the refusals left are `own-parts-unknown`, `endpoint-not-aboard` and `no-readable-parts`. Inside a taken cluster, the pieces another endpoint of the same route owns and the pieces holding a root part the route's recordings name for its transport (`CollectTransportRootFlightIds`: every source recording's snapshot root - the end snapshot's root is the transport's pre-dock root, which an undock restores - and a start-docked proof's transport root, minus every endpoint root) are left out; the endpoint's own pieces are never left out. Those are the flightIDs the recordings captured, so they name the craft that flew the recorded run: a transport Parsek spawned from a recording gets fresh part flightIDs (`VesselSpawner.RegenerateVesselIdentity`) and is just another docked ship. Any other smaller docked ship is taken, so the player undocks visitors first. Each entry is keyed to the endpoint binding the route holds AFTER the resolution (a resolution may rebind through `RouteEndpointTransfer`): the endpoint's root part `flightID` when it has one, else its vessel pid gated by the launch guid, which is how the recorded sets are matched to an endpoint. Part flightIDs are launch-unique, where a part `persistentId` is baked into the .craft and reused by a later launch of the same craft once the original is gone. A repeated adoption replaces the endpoint's set; an endpoint that does not resolve keeps its earlier set; an entry no current endpoint binding matches (left behind by a transfer) is pruned. One `Endpoint part adoption: route=... adopted=A refused=R unresolved=M pruned=P [origin:'Depot' parts=12 prev=none; stop1:'Station' parts=31 prev=28 excluded=1; stop2:'Lander' parts=3 prev=none leftOut=31 own=3; stop3:unresolved('reason') kept=4; ...]` Info line per call is the press's outcome (the window has no outcome line and the press posts no screen message); the button's hover adds `Last updated on <date>.` from the latest adoption.
`RouteEndpointPartScope.ResolveOwnPartSets` consults the adoption BEFORE the recorded sets (the recordings are then not read) and, when one matches the endpoint binding, it REPLACES the recorded set: the scope keeps the pieces holding the endpoint's root or any adopted part (log token `recordedFrom=adopted`), so the adopted module is the endpoint's and a craft docked after the adoption is not; a part that has undocked is simply not aboard; with no adoption for the binding the rule above is unchanged, and with neither the root nor any adopted part aboard the whole-composite fallback applies. Lifecycle: the adoption lives on the `Route` instance like the route's other player settings. `RouteCodec` writes it as a sparse `ADOPTED_ENDPOINT_PARTS { ENDPOINT_PARTS { rootPartUId vesselPersistentId launchGuid adoptedUT uid... } }` node (omitted when there is no entry with a part, so a route never updated is byte-identical, and an older build ignores the node), so a cold load reads the save's copy. An in-session rewind or load reconcile keeps the live route and therefore the adoption (`RouteLoadReconcile.RestoreLoopPositionFromSave` restores only loop positions): in a world rewound to before the module docked, the module's parts are simply not aboard, and docking the same module again admits it again. A transfer to another vessel leaves the adoption unmatched, so the scope falls back to the recorded sets.

```
ResolveEndpointVessel(endpoint):
    1. Find vessel by endpoint.vesselPersistentId in FlightGlobals.Vessels
       → if found and compatible: return vessel
    2. If not found AND endpoint.isSurface == false:
       → return null (orbital endpoints have no fallback)
    3. If not found AND endpoint.isSurface:
       → scan FlightGlobals.Vessels for the nearest compatible vessel within
         RouteEndpointResolver.SurfaceFallbackRadiusMeters
         of (body, lat, lon, alt)
       → return that vessel, or null
```

7.2 Surface vs orbital behavior

- **Surface endpoints:** PID primary, single nearest compatible-vessel fallback within `SurfaceFallbackRadiusMeters`. `isSurface` is captured from the endpoint vessel situation (`Landed`, `Splashed`, or `Prelaunch`), not altitude, so Mun/Minmus surface bases remain surface endpoints. Handles base rebuilding without turning every nearby vessel into one abstract warehouse.
- **Orbital/in-flight endpoints:** PID only. `Flying`, `Sub-orbital`, `Orbiting`, and `Escaping` connection targets are treated as non-surface in v1. Orbital and in-flight coordinates change continuously, so proximity fallback does not work. If the vessel is destroyed and rebuilt, the player must re-target the route.

7.3 Compatible vessel definition

A compatible endpoint or origin fallback vessel must be a real stock vessel, not a Parsek ghost/map-presence vessel, and must be a vessel type that can plausibly own cargo. Exclude vessels in `GhostMapPresence` tracking sets and stock `VesselType` values such as `EVA`, `Flag`, `Debris`, `SpaceObject`, and `Unknown`. For destination resolution, the vessel must contain at least one eligible tank for a delivered resource or one `ModuleInventoryPart` that can store a delivered payload type, regardless of current free capacity; full-but-compatible destinations are handled by the destination-capacity check, not treated as lost endpoints. For non-KSC origin resolution, the vessel must contain at least one eligible store for the route cost manifest. If several compatible vessels remain within the surface fallback radius, choose the nearest one only.

7.4 Loaded vs unloaded vessels

If the endpoint or origin vessel is loaded (player is within physics range), use stock `Part.TransferResource()` for endpoint tank mutation after explicit logistics eligibility checks. `Part.RequestResource()` is a resource-flow system API, not the stock PAW tank-transfer primitive, and should not be used for deterministic endpoint edits. If unloaded, use `ProtoPartResourceSnapshot.amount` / the backing `RESOURCE` node directly through snapshot objects. Both paths apply to origin deduction and destination delivery.

Inventory delivery also has loaded and unloaded paths. For loaded vessels, use stock `ModuleInventoryPart` APIs to add reconstructed `STOREDPART` payloads after logistics validates slot limits, stack capacity, volume, mass, variant, and exact payload identity. For origin debit, remove or decrement the exact matching stored slot; do not use stock part-name-only removal helpers. For unloaded vessels, edit the relevant `ProtoPartModuleSnapshot` / `STOREDPARTS` ConfigNodes directly using the stored `InventoryPayloadItem.StoredPartSnapshot`, then update slot accounting. Both paths preserve the exact payload identity hash and report items that do not fit.

---

8. Transfer Window Scheduling

8.1 Synodic period computation

```
SynodicPeriod(originBody, destBody):
    if originBody == destBody:
        return 0  // same body, no transfer window
    if originBody is Sun or destBody is Sun:
        return 0  // no stable parent orbit to compare
    // Walk up to common parent
    a = originBody, b = destBody
    while hierarchy depth of a > hierarchy depth of b: a = a.referenceBody
    while hierarchy depth of b > hierarchy depth of a: b = b.referenceBody
    while a.referenceBody != b.referenceBody:
        a = a.referenceBody
        b = b.referenceBody
    // a and b now orbit the same parent
    T1 = a.orbit.period, T2 = b.orbit.period
    if T1 == T2: return 0
    return abs(1 / (1/T1 - 1/T2))
```

Handles cross-system routes: Mun→Laythe walks up to Kerbin/Jool orbiting Sun, then uses the Kerbin/Jool synodic period. Guards routes directly to/from the Sun and equal-period edge cases.

8.2 Dispatch interval rules

- **Same-body routes:** Player-set interval. Minimum = recording duration.
- **Inter-body routes:** Default = synodic period, phase-anchored to the original Supply Run start UT (`DispatchWindowEpochUT`). `NextDispatchUT` is always the smallest `DispatchWindowEpochUT + n * DispatchWindowPeriod` that is >= current UT and also

respects the player's minimum interval. NOT BUILT THIS WAY: M5 activated inter-body dispatch through the loop clock's re-aim window basis (`RouteLoopClock` , section 0.9), not through this arithmetic, and `DispatchWindowPeriod` is never written non-zero. This bullet is retained design intent, superseded by section 0.9.

- **Player interval override:** Player can increase the minimum spacing between dispatches, but v1 does not shift the transfer-window phase. A later advanced override may allow phase shifting explicitly.
- **Gravity assist routes:** Two-body synodic approximation (intermediate flybys not tracked). Player can fine-tune by increasing minimum spacing, not by changing the phase anchor in v1.

---

9. Round-Trip Linking

Status: SHIPPED (M4c, branch `logistics-m4-multistop` ; see the 19.4 M4 round-trip bullet for the as-built map). The text below was the v1 deferral rationale, now superseded by M4c. (In v1 `LinkedRouteId` was a reserved serialization field only: dispatch ignored it and tests only verified the field round-tripped.)

As built (M4c):

- A round trip remains two separate one-way Supply Routes, each created from its own Supply Run.
- A linked pair alternates dispatch eligibility: Route A completes → Route B may dispatch → Route B completes → Route A may dispatch. The constraint is a thin scheduling layer, not a new resource path: route B holds `WaitingForPartner` (an evaluator failure kind, NOT a new `RouteStatus` — the route stays `Active` / `GhostDriving` / renders its loop while waiting) until the partner completes a NEW cycle, tracked by the sparse `Route.LastConsumedPartnerCycle` alternation cursor. The partner gate is ordered LAST in `CheckEligibility` , so a genuinely-blocked route surfaces its real blocker first; the M6 hold reason names the linked route.
- Pausing (or Breaking) a partner does NOT block the other (10.14): a non-ghost-driving partner can never complete a cycle, so the constraint is bypassed and the route dispatches on its own schedule. A mutual A↔B link with neither route yet completed is broken by a deterministic deadlock seed (lower `DispatchPriority` , then ordinal `Id`) that dispatches first on its very first cycle.
- Alternation is per-dispatch, evaluated in route order within a tick. If both linked routes' dock phases land in the SAME orchestrator tick (e.g. two near-identical routes, or one tick straddling both phases under high time warp), the seed dispatches first, advances its `CompletedCycles` , and the partner's gate then sees that advance within the same tick and also dispatches — so the pair both-fire that tick rather than alternating. This is the accepted same-tick degeneracy (consistent with the one-cycle cold-start overlap); a genuine out-and-back pair has distinct dock phases and alternates cleanly. Documented so it is not mistaken for a regression; strict same-tick alternation would require snapshotting each partner's `CompletedCycles` at tick start.
- The pair is established from a route's detail panel: a `Link round-trip...` button opens a single-select picker of eligible partners (any other route not already linked), and an `Unlink` button breaks the pair. The control commits through `RouteStore.LinkRoutes` / `UnlinkRoute` , which set the link on BOTH routes (the gate resolves the partner only by the local `LinkedRouteId` , so the link must be mutual), reset both `LastConsumedPartnerCycle` to the 0 cold-start default (the deadlock seed picks the first dispatcher), reject a self-link or an already-linked route, and clear a partner's dangling back-reference when a linked route is deleted (`RemoveRoute`).

---

10. Edge Cases

10.1 Destination destroyed, surface base

Scenario: Mun base destroyed. Player rebuilds at same spot. **Behavior:** PID match fails. Surface proximity fallback finds new vessel within `SurfaceFallbackRadiusMeters` . Route auto-reconnects. **v1 limitation:** If rebuilt outside `SurfaceFallbackRadiusMeters` , player must explicitly re-target or create a new route.

AS BUILT 2026-09-04 (operator ruling): the proximity resolution is a TRANSFER, not a silent substitution. Until then the resolver delivered into the nearby craft while the persisted `RouteStop.Endpoint` still named the vanished one, so every cycle re-resolved by position, the produced save read the dead pid, and the Logistics window never named the craft that actually received the cargo (measured by harness lane `RVR-18` on `rover-route-recorded` , run `2026-09-03_2011` ; todo entry `ROUTE-DELIVERY-PROXIMITY-RETARGETS-ANY-NEARBY-VESSEL`). Now, when `RouteEndpointResolver` resolves an endpoint to a vessel OTHER than the recorded one, the persisted endpoint is REBOUND to it:

- **What moves:** the identity fields only - `RouteEndpoint.VesselPersistentId` (the resolved vessel's pid), `RouteEndpoint.RootPartUId` (its root part `flightID` , launch-unique where a pid is craft-baked, so the NEXT cycle wins at the root-part step instead of walking to proximity again) and, since 2026-09-06, `RouteEndpoint.LaunchGuid` (the resolved vessel's

launch guid, so that if a later cycle DOES fall to the pid step that step is gated rather than bare; cleared when unreadable, never left naming the vessel that went away). The POSITION fields (body / lat / lon / altitude / `IsSurface`) stay as the historical dock anchor - they are what the 500 m radius is measured from, and moving the anchor onto each successive substitute would let a route walk across the landscape one transfer at a time. Nothing is added to the on-disk shape: both identity keys already round-trip through `RouteNodeCodec.SerializeEndpoint`, so no schema generation moved.

- **Which endpoints:** every endpoint that resolves through this resolver, in both directions - a delivery destination (`role=destination`), a pickup source (`role=pickup`; the pickup-source gate resolves the same `RouteStop.Endpoint`, so a transfer there means the route now takes its cargo from the nearby craft) and `Route.Origin` (`role=origin`). The role is a label for the log and the message; the rebind is identical.
- **The guard:** the route's OWN TRANSPORT is excluded from the proximity candidate set with the reason `route-own-transport`. Its identities come from the route's `SourceRefs` recordings (`Recording.VesselPersistentId` + `RecordedVesselGuid`) resolved through the ERS, compared `VesselLaunchIdentity`-style - guid first, pid fallback, a relaunch of the same craft file is a different vessel. RVR-18 measured the transport as the runner-up at 16.42 m, so without this the branch would pay the route's own carrier - the "a route paying itself" case the root-part step was introduced to prevent for origins. When the exclusion leaves nothing, the miss carries its own token, `no-candidate-after-transport-exclusion`, and the route holds `EndpointLost` exactly as an empty candidate set already did.
- **What the player sees:** one `ParsekLog.Info` line, `Route endpoint transferred: route=<id8> stop=<index> role=<destination|pickup|origin> from=<oldPid>/'<oldName>' to=<newPid>/'<newName>' step=<proximity|pid> distance=<m> body=<body> at ut=<ut>`, plus ONE screen message ("Supply route : not found, now delivering to "). It cannot repeat: the stop is rebound, so the next cycle resolves at the identity steps. The `Endpoint proximity step:` line is unchanged.
- **The PID step landing on a different LAUNCH of the same craft file** (was "not covered"; closed 2026-09-06 by `RESOLVER-PID-STEP-NOT-GUID-GATED`). A `RouteEndpoint` now carries `LaunchGuid` - the guid the bind READ when it stamped the pid, persisted sparsely beside it and excluded from the proof hash - and the resolver's PID step refuses a match whose guids conclusively differ, logging `Endpoint pid step refused: ... reason=different-launch` and falling through to the proximity step, which then transfers. Unknown on either side still accepts (`VesselLaunchIdentity`'s rule), so every route built before the key existed keeps the behaviour it had. The transfer decision's own `step=pid` arm is now fed the endpoint's real guid, but it still has NO production caller: the resolver refuses at the step rather than reporting a pid-step resolution, so `step=proximity` remains the only live-reachable spelling in the transfer line.

Decision surface: `RouteEndpointTransfer.Evaluate` (transfer or keep), `IsRouteTransport` (the guard), `FindOwners` / `ClassifyStopRole` (which persisted field to rebind and what to call it), `CollectTransportIdentities`, and the two formatters - all pure, all in `Source/Parsek/Logistics/RouteEndpointTransfer.cs`, exercised headlessly by `RouteEndpointTransferTests`.

10.2 Destination destroyed, orbital station

Scenario: Kerbin station destroyed. Player rebuilds. **Behavior:** PID match fails. No proximity fallback. `Status = EndpointLost`. Player must re-target.

10.3 Origin destroyed or recovered (non-KSC)

Scenario: Minmus base recovered for funds. **Behavior:** Applies only to non-KSC routes whose Supply Run started docked to an origin depot. No vessels at origin → `Status = EndpointLost`. Vessels exist but empty → `Status = WaitingForResources`. Route persists. Resumes when resources appear. Surface origins have proximity fallback. KSC origins skip this check. **AS BUILT 2026-09-04:** a surface origin that resolves to a DIFFERENT craft through that fallback is transferred and rebound exactly as a destination is (10.1) - the route then loads its cargo from the craft standing at the recorded dock spot, announced once. The route's own transport is excluded from the candidates, so the fallback can never turn the origin debit onto the carrier.

10.4 Destination tanks full

Scenario: Route delivers 200 LF. Base has 200/200 LF. **Behavior:** `Status = DestinationFull`. No origin deduction, no delivery, no ROUTE_DISPATCHED event, and `NextDispatchUT` is not advanced. The ghost still flies the route so the world looks busy (a route that just stopped spawning ghosts reads as broken); it is purely visual this cycle and moves/charges nothing. The route status text names the blocking reason. Set `NextEligibilityCheckUT` for a rate-limited retry. The same scheduled dispatch resumes when player use or another stock process creates capacity. `SkippedCycles` is not incremented for repeated capacity-poll attempts against the same scheduled dispatch.

10.5 Destination partially full

Scenario: Base has room for 100 LF, full on Ox. Delivery: 150 LF + 183 Ox. **Behavior:** Per-resource independent: deliver 100 LF, 0 Ox. Origin pays the CostManifest / KSC dispatch cost (transit already happened). ROUTE_DELIVERED records requested vs actual amounts.

10.6 Player reverts past a dispatch

Scenario: Route dispatched at UT=50000. Player reverts to UT=49000. **Owner rulings 2026-10-21, not yet built (todo ROUTE-REWIND-REFUNDS-COMMITTED-CYCLES; analysis `docs/dev/research/route-funds-vs-cargo-2026-10-21.md`).** At a rewind (a go-back rewind or a Re-Fly start) committed route rows stop being the one exception to ruling D2. The rows below describe the code until this is built.

- A rewind deletes no route row. At the rewind UT it adds a refund for every committed cycle dispatched after the rewind point, which only raises funds. The destination stays live (holding it as a ghost until the last committed cycle was rejected: "the whole point of supply routes is that vessels can just dock with the station or base to get supplies"), and the route re-decides those cycles live as the clock passes, as today, so a cycle that can no longer deliver is never paid for. The dispatch dedup and the rebuilt cycle counters skip a refunded cycle's rows, so each re-fires once.
- The refund is the cycle's NET cost (dispatch debit minus its recovery credit), so a re-run cycle costs what the original did and repeated rewinds cannot farm credits.
- Each cycle is refunded once: the refund row names the cycles it covers, so a second rewind past the same cycles refunds nothing more.
- A cycle dispatched before the rewind and delivered after it is not refunded (it still delivers through the saved route state, "Loop position" below), and its recovery credit is not paid twice.
- A cycle that made money (its credit above its debit, a Sell-at-KSC run) stays history: it is not refunded and not run again, and when the clock passes its crossing its pickup is taken from the origin depot again, up to what the depot holds, with no new ledger row. The rewind restored the depot's cargo, so keeping the gain without this would sell the same cargo twice.
- The keep-and-drop store below stays: a later committed recording's snapshot can still carry cargo from a refunded cycle, now selected by the refunded cycles instead of the retired rows.
- A load back in time keeps today's handling: a quickload or load restores the save exactly (`docs/dev/design-load-restores-the-save.md`), and a stock revert removes only the rows the reverted session wrote (game-actions design section 3.13), so committed route rows and the refunds a rewind wrote before it stay. This last point is derived from the revert ruling, not ruled separately.
- The dispatch of a re-decided cycle is reservation-aware (section 8, step 4), so it cannot spend money a committed purchase needs.

Behavior. Every load back in time - a stock revert (to launch or to the VAB/SPH), an F9 quickload, the Esc-menu Discard Re-fly (it reloads the Rewind Point quicksave), a go-back rewind and a Re-Fly start - restores the stock vessels from the loaded save, and with them every PHYSICAL route effect (origin debit, per-window pickup, endpoint delivery). There is no per-writer undo; `RouteModule` is observe-only and mutates no cargo. The route store and the ledger are not reloaded by an in-session load (`RouteStore` loads from the save on a cold load only), so each of these loads reconciles them at the UT the world went back to:

- **Route ledger rows.** Free-standing route rows (`RecordingId = null`) with UT strictly after the cutoff are retired (`Ledger.RetireFutureRouteActionsAtRewind` ; on a Re-Fly start `ReconciliationBundle.Restore(cutoff)`). The UT-blind dispatch dedup then finds nothing for the re-flown cycles, each re-fires once, and the ledger recalc rebuilds the FUNDS effects (KSC dispatch charge, recovery credit) from the surviving rows.
- **Route store.** The shared `RouteRewindClassifier.ReconcileStoreAtRewind` : a route created after the cutoff goes dormant and comes back Paused when the timeline reaches its creation; a kept route drops cycle state stamped after the cutoff, re-derives its pause status from the kept player pause / resume rows, loses an armed Send Once / pause-after-cycle, and rebuilds its cycle counters from the kept rows.
- **Who runs it, at which cutoff.** The go-back rewind: `ParsekScenario.HandleRewindOnLoad` through `RouteLoadReconcile.ReconcileAtGoBackRewind` , at the rewind save's own UT, before the 15 s lead-time windback that sets the clock (owner ruling 2026-10-07). The route state the exit restores comes from that save ("Loop position" below), so the route rows it keeps are the rows up to the same moment: a crossing between the wound-back clock and the save's UT keeps its charge and recovery-credit rows along with its saved cursor, and a route created in that window stays committed because the save carries it. Only routes take this cutoff; the career prune, the recalculation, tech and crew stay at the adjusted UT. With no route copy read, nothing is restored and the cutoff falls back to the adjusted UT, where the reset re-fires each crossing after it once. Because the rows kept in that window lie ahead of the clock, a cutoff ahead of the clock becomes the **route state floor** (`RouteStore.StateFloorUT`): the route state is settled through it. Every save written while the clock is behind it carries it (`routeStateFloorUT` beside ROUTES), and every load keys its route cutoff and the UT of its route copy to the later of its own UT and the floor its save carries (`RouteLoadReconcile.ApplyRouteStateFloor`): a scene change, launch or F9 at the Space Center inside the window finds nothing after the floor and touches nothing; an F5 inside the window then F9 restores the window's rows with the quicksave's cursors; a revert to a launch made inside the window spares route rows up to the floor in the revert prune; a Re-Fly start from an RP quicksave written inside the window raises the bundle's route cutoff the same way; a later go-back to a rewind save written inside the window takes its copy as of the floor. A save without a floor (written before the rewind) is a load back past the window: it reconciles at its own UT and clears the floor, as does any load whose clock has passed it. Residual: route actions taken inside the window between a save written there and a load back to that save (a pause or resume

row, an armed Send Once) are not undone by that load. A Re-Fly start: the bundle, at the post-load UT. Every other in-session load: `RouteLoadReconcile.ReconcileAtInSessionLoad` from `ParsekScenario.OnLoad`, before the recalculation, at the loaded save's `flightState.universalTime` (a stock revert takes the earlier of it and the revert prune's launch boundary). An F9 in flight, a stock revert and Discard Re-fly always run it; any other in-session load (an F9 at the Space Center or Tracking Station, or into a flight quicksave from there) runs it only when a route row, a route creation or a route's cycle start / hold / partial delivery / owed-credit dispatch lies after the loaded save, so a forward scene change touches nothing. The policy row is the Routes column of `LoadReconcilePolicy`.

- **Loop position.** The store reconcile resets every kept loop cursor to -1, and the dispatch dedup keys on the counter-based cycle id, not the loop index, so on its own the reset would re-fire the crossing that most recently passed before the cutoff (delivered and charged twice). Every one of these loads therefore then gives each kept route its loop position back from the loaded save's own route copy, when the save is not newer than the cutoff: the loop anchor always (`LoopAnchorUT` is the loop clock's phase anchor and so the cursors' index space; a re-activation after the save moved it), the route cursor, per-stop cursors and window anchor when the cadence, transit, dock UTs and window basis are unchanged (otherwise the reset stands and the next crossing fires), the partner alternation cursor when the route is linked to the same partner, and the recovery credit the save still owes (only the marker; the next crossing pays it). A crossing dispatched before the save is not fired a second time and none owed after it is swallowed; a route the save does not carry keeps the reset. Where the copy comes from: an in-session load and a Re-Fly start read the OnLoad node (the loaded save's, the RP quicksave's on a Re-Fly start, through `ReconciliationBundle.Restore(bundle, cutoff, loadedSaveRoutes, loadedSaveUT)`); the go-back rewind cannot, because the Space Center it loads reloads persistent.sfs and hands OnLoad that file's node, so `RecordingStore.ExecuteRewindSaveLoad` reads the rewind save's own ROUTES from the parsed save before the scene load (`RouteLoadReconcile.CaptureRewindSaveRoutes` into `RewindContext`) and `HandleRewindOnLoad` restores from it (`RouteLoadReconcile.RestoreLoopPositionAtRewindExit`, the rewind exits' shared call). On the go-back the copy's UT is the cutoff itself (above), so the restore always runs.
- **Recovery credit.** A pending credit pays only while its cycle's `RouteDispatched` row is in the effective ledger; a credit whose dispatch was retired or tombstoned is cleared unpaid.

A cold load reads routes from the save but the ledger file as last written; the abandoned future's route rows on a cold load into an older save are open under COLD-LOAD-INTO-OLDER-FLIGHT-SAVE-KEEPS-ABANDONED-FUTURE.

A discard WITHOUT a load - the merge dialog's Discard and Re-Fly discard, the tree / switch-segment discard cores - has no save to restore the physical effects, so cargo moved and funds charged inside the discarded segment persist (maintainer option C, 2026-07-06; surfaced only as the `[Rec-3 residual]` diagnostic, `RouteRevertSafety`; reverse writers declined).

Retired crossings in a chain tip snapshot (todo CHAIN-TIP-SNAPSHOT-CARRIES-UNPAID-ROUTE-CARGO, owner ruling 2026-10-07). The quicksave restore cannot revert cargo that a committed recording's snapshot carries: a station a later mission docks to is held back by the Ghost Chain Rule after the rewind and comes back from the mission's tip snapshot, which still holds the deliveries (and lacks the pickups and origin debits) of the crossings the rewind just retired and refunded, while the replayed crossings into the held-back station are blocked. So every retire (`Ledger.RetireFutureRouteActionsAtRewind`, used by the go-back rewind and the in-session load reconcile, and `ReconciliationBundle.Restore`) passes its removed rows to `ChainTipRouteCargo.CaptureRetiredRouteCargo` (an in-session load whose reconcile finds nothing after its cutoff calls it with none, so each snapshot's watermark, the lowest retire or load-back cutoff since it was first seen, comes down at every load back in time): each cargo row (delivered, picked up, physically debited) whose endpoint, resolved from the pre-rewind route store (the stop for a delivery or pickup, the origin for a debit, plus the vessel the writer resolved), is a committed chain tip's vessel and whose UT is at or before that tip's end and that snapshot's watermark is kept in `RetiredRouteCargoStore`, tagged with the one snapshot that carries it, persisted beside the ledger actions. A spawn copy of that same snapshot then drops the cargo (flight-recorder design 12.5). A crossing the current timeline performed and paid again stays; it is recognised by route, stop, row type and nearest UT, not by cycle id: the cycle counters are rebuilt from the kept dispatch rows at every rewind and a blocked crossing writes no dispatch row, so a replay can carry another id. Route rows themselves are still retired exactly as above.

Retired crossings in any end-of-recording snapshot (todo END-SPAWN-SNAPSHOT-CARRIES-UNPAID-ROUTE-CARGO, owner ruling 2026-10-07). The same keep-and-drop runs for an ordinary leaf whose own recorded vessel is the route endpoint (a recording of the route-fed vessel made while the route delivered into it): its rows are tagged to that leaf's snapshot when the endpoint (pid plus launch guid, or root part flightID when a guid is unknown, never a bare pid) is positively the leaf's vessel, and the leaf's spawn copy drops them (flight-recorder design 12.5). A snapshot's capture is the recording's end before the optimizer trimmed its tail, noted by the trim: a tail-trimmed vessel spawns early, so the crossings of its trimmed tail are dropped from the copy and the route replays them live after the spawn, each paid once.

10.7 Time warp past multiple cycles

Scenario: Three cycles due at UT=50000, 50500, 51000. **Behavior:** The per-tick catch-up loop alternates in-transit progression and dispatch evaluation until the route is no longer due or hits a blocker. All due cycles are processed sequentially in one deterministic

route order. Each dispatch checks destination capacity and origin affordability independently. First may deplete origin or fill destination, blocking subsequent cycles without advancing the blocked `NextDispatchUT`.

AS BUILT (loop routes): missed cycles are COLLAPSED, not replayed. A loop route fires once for the highest owed dock cycle after a warp and snaps its cycle cursor forward (single-stop: `RouteLoopDeliveryFireTests.WarpJump_FiresOnce_SnapsForward`; multi-stop: each due window fires once, `RouteMultiStopFireTests.WarpPastCyclesAndWindows_FiresEachDueWindowOnce_BumpsOnce`). Delivered totals over a warp therefore depend on how far the warp jumped, by design; the sequential catch-up described above is the pre-loop-route behaviour.

10.8 Transport still docked at recording end

Scenario: Player forgets to undock. **Behavior:** Validation fails. "Create Supply Route" absent. Tooltip: "Transport must undock from destination."

10.9 No cargo transferred during connection

Scenario: Player docks and undocks without transferring delivery cargo. **Behavior:** Validation fails. Tooltip: "No resource or inventory transfer detected."

10.10 Multiple connection windows in one recording chain

Scenario: Supply Run has dock-transfer-undock-dock-transfer-undock. **Behavior:** Validation fails for v1. UI explains that multiple delivery stops were detected and multi-stop route execution is deferred. The player can re-fly or split the mission into one-stop Supply Runs.

10.11 Route dispatch while player is at destination

Scenario: Player at Mun base when delivery arrives. **Behavior:** Destination loaded. Uses `Part.TransferResource()` on eligible endpoint tanks. Resources appear in real-time.

10.12 Competing routes at same origin

Scenario: Two routes share Minmus base. Base has enough for one, not both. **Behavior:** v1 — FIFO by NextDispatchUT. Future: player-configurable priority. (As implemented, v0 processes routes in `RouteStore` commit-list order in `RouteOrchestrator.Tick`, with no `NextDispatchUT` sort; player-set priority is milestone M1, 19.4.)

10.13 Concurrent deliveries to same destination

Scenario: Two routes deliver to the same base on the same scheduler tick. The destination can accept one delivery but not both. **Behavior:** v1 processes route events in deterministic FIFO order: `NextDispatchUT`, then route `Id` as a stable tie-breaker. (As implemented, the deterministic order is `RouteStore` commit-list order — see the 10.12 note.) Each route recomputes capacity immediately before its own delivery. The first route may fill the destination, causing the later route to partially deliver or report undelivered cargo. v1 does not reserve shared destination capacity across routes.

10.14 Linked route partner paused

Scenario: Route A and B linked. Player pauses B. **Behavior:** A dispatches on its own schedule (the chain constraint is bypassed for any partner that is not ghost-driving — Paused / EndpointLost / MissingSourceRecording / SourceChanged — or unresolved, since such a partner never completes a cycle). B resumes its alternation when unpaused. **SHIPPED (M4c):** the bypass is `!RouteStatusPolicy.GhostDriving(partner.Status)` plus an unresolved partner, in `RouteDispatchEvaluator.PartnerConstraintSatisfied`.

10.15 Source recording missing

Scenario: Source recording for a route is deleted or fails to load. **Behavior:** `Status = MissingSourceRecording`. Route cannot dispatch and cargo transfers stop. UI explains that the proof Supply Run is gone and the route must be recreated or the recording restored. If the recording is restored by loading/reverting to a save where it exists, or by restoring recording sidecars, route load/revalidation clears the status back to `Active` and recalculates `NextDispatchUT`.

10.16 Source recording changed or superseded

Scenario: Source recording IDs still exist, but optimizer rewrite, re-fly supersede, sidecar epoch change, or route-proof metadata rewrite changes route timing or proof data after route creation. **Behavior:** `Status = SourceChanged`. Route cannot dispatch and in-

transit delivery aborts before moving cargo. UI explains that the proof Supply Run changed and the route must be recreated or explicitly reanalyzed. Stored `TransitDuration` and `DeliveryOffsetSeconds` remain unchanged for diagnosis, but they are not used to keep moving cargo against changed proof data.

10.17 Save/load round-trip

Scenario: Save, load. **Behavior:** All Route fields serialized in ParsekScenario OnSave/OnLoad. State restored exactly.

---

11. v1 Limitations

- **Capacity changes during transit:** Delivery clamps to `maxAmount` at delivery time. Excess is not delivered; ROUTE_DELIVERED records requested vs actual amounts. Origin was already deducted at dispatch.
- **Zero transit duration:** Dispatch and delivery may process in same frame. Acceptable.
- **EC-only delivery:** ElectricCharge remains excluded from route manifests as environmental noise, matching Phase 11 resource snapshot rules. EC-only Supply Runs are not route-eligible in v1.
- **Resource not on destination:** If delivery manifest includes a resource the destination has no tanks for, that resource is reported as undelivered, not silently skipped.
- **Concurrent endpoint production/consumption during route creation:** Delivery proof uses the conservative `min(transportLoss, endpointGain)` over the dock/undock window. If a third vessel, ISRU, converter, or drain changes the endpoint resource during that same window, route analysis may under-credit or reject the candidate rather than trying to attribute multiple simultaneous sources and sinks. v1 expects one meaningful cargo transfer per connection window.
- **Origin loaded vs unloaded:** Same loaded/unloaded distinction as delivery.
- **Scene handling:** Route scheduler runs in all scenes via ParsekScenario. `FlightGlobals.Vessels` available for endpoint resolution.
- **Revert mechanism:** Route state serialized in .sfs. Quicksave load restores Route ConfigNode and stock vessels. Parsek timeline rewinds require explicit route ledger modules for every enabled route mutation path.
- **Inventory delivery:** Inventory items delivered to destination cargo slots. If destination lacks available slots or the part type doesn't fit, excess items are reported as undelivered.
- **v1 scheduler shape:** `CurrentCycleStartUT`, `CurrentSegmentIndex`, `PendingDeliveryUT`, and `PendingStopIndex` are serialized now for chain-sequential execution and future multi-stop compatibility. Multi-stop delivery SHIPPED in M4a: the `Stops` list now drives real multi-stop firing, with each stop firing at its own recorded dock phase (`RouteStop.RecordedDockUT`, via the loop clock; `DeliveryOffsetSeconds` is codec-only and not read at fire time), not reserved-only sequencing.
- **Crew delivery:** Deferred. No generic kerbal generation in v1.
- **Route analysis edge cases:** The route analysis engine walks docking windows linearly. Complex patterns (dock to A, undock from A, dock to A again) are detected as separate candidate windows and rejected for v1. A window with no delivery cargo change is not route-eligible.

---

12. What Doesn't Change

- **Recording runtime behavior** — recording behavior is unchanged. The Recording schema is additively extended for logistics capture (§4.9): connection-scoped dock/undock manifests, stock connection target metadata, endpoint situation, and origin-dock proof. Logistics v1 reads those fields but never writes to recordings.
- **Ghost playback engine** — no execution changes to GhostPlaybackEngine, IPlaybackTrajectory, or IGhostPositioner. The route scheduler uses the same playback engine as loops for visuals, but route state advances from UT-driven scheduler ticks rather than depending on playback-completed events.
- **Loop system** — per-recording loop toggle, timing, cycle events all work as today. Routes do not use the loop system — they are siblings, not built on top of it. Both use the ghost playback engine, but through different scheduling paths.
- **Chain semantics** — dock/undock/split/merge gameplay semantics do not change. Logistics adds route-proof metadata to the existing always-tree boundaries, but it does not make the chain system responsible for dispatch or delivery.
- **Manifest capture systems** — existing `ExtractResourceManifest`, lightweight inventory manifests, and crew manifests remain available for diagnostics. Logistics adds exact inventory payloads and connection-scoped dock/undock manifests as additive fields. Logistics v1 consumes resources/inventory and connection metadata read-only after capture.
- **Merge dialog** — route creation may be offered after commit/merge when analysis finds an eligible Supply Run, but the merge semantics themselves do not change.

- **Crew reservation** — not touched in v1. Crew logistics is deferred until it can use named roster/crew-reservation semantics instead of generic kerbal generation.
- **Game actions system** — route dispatch/delivery events are new event types in the existing ledger, with route-specific modules for KSC funds, non-KSC origin debit, and endpoint delivery reversal/recompute. KSC-origin Career dispatch costs use existing funds-reservation checks, but physical cargo mutation is not enabled until the route modules prove epoch isolation for loaded and unloaded vessels.
- **Map route lines** - SHIPPED (M6). `Display/RouteTrajectoryLineRenderer.cs` draws each committed route's recorded legs on the flight map and Tracking Station, behind the `showRouteLines` setting (default on), covering both same-body and inter-body legs.

---

13. Module Architecture

The logistics route system follows the same module pattern as the game actions system (v0.6): a self-contained directory with a thin orchestrator connecting it to Parsek's lifecycle hooks. The route module is removable by deleting the directory and removing the orchestrator calls — no behavioral changes to recording, playback, or the game actions system.

13.1 Directory structure

```
Source/Parsek/Logistics/
    Route.cs                    // data model (Route, RouteStop, RouteEndpoint, RouteStatus)
    RouteStore.cs               // static storage surviving scene changes (like RecordingStore)
    RouteScheduler.cs           // dispatch/delivery evaluation + chain-sequential playback (pure logic)
    RouteDelivery.cs            // resource and inventory mutation on loaded/unloaded vessels
    RouteEndpointResolver.cs    // vessel finding by PID + surface proximity fallback
    RouteManifestComputer.cs    // derive delivery/cost manifests from recording tree resources/inventory
    RouteAnalysisEngine.cs      // tree/path walk + connection-window extraction from committed recordings
    RouteOrchestrator.cs        // thin integration layer -- called from ParsekScenario hooks
```

13.2 Integration seams (5 total)

These are the only places where logistics code touches existing Parsek code:

Seam	Where	What	How to guard
Save/Load	`ParsekScenario.OnSave` / `OnLoad`	`RouteOrchestrator.OnSave(node)` / `OnLoad(node)`	Null-check. Missing ROUTES node = no routes.
Scheduler tick	`ParsekScenario.Update`	`RouteOrchestrator.Tick(currentUT)`	Single call, no-op if no active routes.
Playback start/seek	`ParsekPlaybackPolicy` / ghost playback start path	`RouteOrchestrator.RequestVisualPlayback(recordingId, recordingLocalOffsetUT)`	Visual-only. Used for dispatch visuals and mid-transit scene entry; no route state authority.
Playback completed	`ParsekPlaybackPolicy.HandlePlaybackCompleted`	`RouteOrchestrator.OnVisualSegmentCompleted(evt)`	Visual hint only. Route state also advances from UT-driven scheduler ticks.
Timeline events	`Ledger` / `LedgerOrchestrator`	New `GameActionType` entries for ROUTE_DISPATCHED / ROUTE_DELIVERED plus route cargo/funds modules	Additive enum values, display strings, and recompute/rollback coverage before stock mutation.

No route-state changes to `GhostPlaybackEngine`, `RecordingStore`, `RecordingTree`, `ChainSegmentManager`, or `RecordingOptimizer`. Recording metadata changes are additive: logistics capture adds connection-scoped dock/undock manifests and origin-dock metadata for route analysis, and the tree codec persists those fields. The playback start/seek seam is visual only; if the existing ghost start path needs an offset parameter or adapter, that change must not make ghost playback authoritative for delivery. `GhostPlaybackEngine` continues to emit `PlaybackCompleted` at the same point it does today; only `ParsekPlaybackPolicy` / `RouteOrchestrator` interpret that event as a visual hint instead of an authoritative route-state transition.

13.3 Read-only consumption of recording data

The route module is a read-only consumer of recording data. It reads:

- `RecordingTree.Recordings` / `RecordingTree.BranchPoints` -- the always-tree source graph for the Supply Run
- `rec.StartResources` / `rec.EndResources` -- resource manifests (Phase 11)
- `rec.StartInventory` / `rec.EndInventory` -- lightweight inventory manifests (Phase 11 diagnostics)
- `rec.RouteConnectionWindows` -- transport/endpoint part PID sets and connection-scoped dock/undock resource/inventory manifests
- `RouteConnectionWindow.EndpointAtDock`, `TransferTargetVesselPid`, and `TransferKind` -- stock connection target identification
- `RouteConnectionWindow.TransferEndpointSituation` and route origin proof -- surface/orbit classification and non-KSC origin proof
- `rec.StartBodyName`, `rec.LaunchSiteName`, and trajectory points -- location context. Recording-level start latitude/longitude fields do not exist today; route coordinates come from selected `TrajectoryPoint`s or `RouteConnectionWindow.EndpointAtDock`.

The route module never writes to Recording objects. It creates Route objects in its own `RouteStore`, serialized in its own `ROUTES` ConfigNode section inside `ParsekScenario`.

13.4 Resource modification path

Resource delivery modifies vessels that are not part of the recording system — they are real KSP vessels at endpoint locations:

```
RouteDelivery.DeliverResources(endpointVessel, deliveryManifest)
    if loaded:    targetPart.TransferResource(resource, +amount, sourcePart)
    if unloaded:  protoPartResource.amount += amount
```

Loaded origin debits use the same primitive with a negative amount on the proven origin resource. Logistics pre-checks flow state/mode for both signs because stock PAW transfer does so before calling the primitive. This is independent of Parsek's recording/playback/ghost systems, but not independent of the ledger. Every enabled mutation path must be driven by a route event that records target vessel identity, route id, cycle, resource/item amounts requested and actually applied, and enough before/after information for recalculation or rollback.

The v0 skeleton ships a single `RouteModule` (registered at `RecalculationEngine.ModuleTier.SecondTier`, after `FundsModule`) that observes every route-scoped action type — `RouteDispatched`, `RouteCargoDebited`, `RouteCargoDelivered`, `RoutePaused`, `RouteEndpointLost` — and tracks per-route walk state (`RouteHeld`, section 6.7, is a history row it ignores). The module is observation-only: it does not yet apply KSC funds charges, debit origin resource/inventory, deliver endpoint payloads, or consult `Affordable`. As of item 5 the dispatch scheduler emits `RouteDispatched` / `RouteCargoDebited` / `RouteEndpointLost` rows; the module still observes them without mutating, and the remaining mutations (funds debit, origin/endpoint cargo, `RouteCargoDelivered`) land with item 6. A future phase may split `RouteModule` into separate KSC-funds / origin-debit / endpoint-delivery modules if separation of concerns demands it; the action-type vocabulary already keeps that option open.

If a module cannot find or safely reverse its target during epoch recomputation, it must mark the route/effect invalid and log a warning rather than silently leaving duplicated cargo behind.

13.5 Lifecycle isolation

Routes have their own lifecycle, independent of recordings:

- **Creation:** from a committed Supply Run after player confirmation, but the route is a separate entity with its own GUID.
- **Persistence:** own ConfigNode section (`ROUTES` in ParsekScenario), not part of recording metadata.
- **Deletion:** deleting a route does not affect its source recordings. Deleting a source recording disables the route (`MissingSourceRecording`); cargo transfers do not continue without the proof run.
- **Revert:** route state is serialized in .sfs - quicksave/load restores it. Timeline events use explicit route ledger modules for epoch isolation. Loading/reverting to a save where a missing source recording exists again revalidates `MissingSourceRecording` routes and restores them to `Active`; loading/reverting to a save where source IDs exist but fingerprints differ leaves the route `SourceChanged`.

13.6 Why not a separate assembly now

The roadmap defers assembly extraction to the future standalone ghost-playback boundary. For Logistics / Supply Routes, a directory-level module within `Parsek.csproj` is the right granularity. Routes need direct access to `Recording.StartResources` / `EndResources`, `RecordingStore.CommittedTrees`, `RecordingTree.BranchPoints`, `Ledger`, and `ParsekScenario` lifecycle. Cross-assembly access would require making all of these `public` or adding an interface layer — friction without benefit. If standalone ghost playback extraction happens, routes stay in Parsek (they are Parsek policy, not ghost playback).

14. Backward Compatibility

- **Saves without routes:** Load fine. ROUTE ConfigNode absent.
- **Saves without resource manifests:** Load fine. Route creation unavailable for old recordings (no manifest data).
- **Old recordings:** Cannot become routes. Player must re-fly to create a recording with manifests.
- **Format:** Route data is additive in its own `ROUTES` node. Missing node = no routes. Recording-side route-proof metadata should use missing-node defaults where possible, but the implementation must follow the current recording codec/schema policy. If `RecordingTreeRecordCodec` requires a schema generation bump for persisted recording metadata, bump it and add format-gate tests.
- **Reserved v1 fields:** v1 serializers preserve the forward-compatible route shape (`Stops` , `LinkedRouteId` , inventory manifests, `KscDispatchFundsCost` , `CurrentCycleStartUT` , pending transit fields). Non-null defaults are written; nullable fields use the omission-means-null convention from §4.8 and must restore as null. Skipping reserved fields turns future multi-stop/round-trip work into a save migration instead of an additive load.

15. Diagnostic Logging

15.1 Route creation

- `[Parsek][INFO][Route] Route created: id={id} name={name} recordings={count} endpoint={body}({lat},{lon}) connection={kind} cost={manifest} kscFunds={funds}`
- `[Parsek][INFO][Route] Route endpoint: {body}({lat},{lon}) vesselPersistentId={pid} delivery={manifest} inventory={manifest}`
- `[Parsek][INFO][Route] Route analysis: chain={count} recordings, deliveryWindows={count}, acceptedStops={count}`
- `[Parsek][INFO][Route] Route validation failed for chain: {reason}`

15.2 Dispatch evaluation

- `[Parsek][VERBOSE][Route] Dispatch check: route={name} nextUT={ut} currentUT={ut} status={status}`
- `[Parsek][INFO][Route] Dispatch: route={name} cycle={n} deducted={amounts} from origin at {body}({lat},{lon})`
- `[Parsek][INFO][Route] Dispatch: route={name} cycle={n} chargedKscFunds={funds}`
- `[Parsek][INFO][Route] Dispatch delayed: route={name} — origin missing {resource}={needed} (available={have})`
- `[Parsek][INFO][Route] Dispatch delayed: route={name} — insufficient KSC funds needed={funds} available={funds}`
- `[Parsek][INFO][Route] Dispatch waiting: route={name} — destination full (capacity={amounts}) nextEligibilityCheckUT={ut}`
- `[Parsek][VERBOSE][Route] Dispatch backoff: route={name} status={status} nextEligibilityCheckUT={ut} currentUT={ut}` (emitted through `VerboseRateLimited`)
- `[Parsek][WARN][Route] Dispatch disabled: route={name} — missing source recording id={id}`
- `[Parsek][WARN][Route] Dispatch disabled: route={name} — source recording changed id={id} expectedProof={hash} actualProof={hash}`
- `[Parsek][WARN][Route] In-transit delivery aborted: route={name} — missing source recording id={id}`
- `[Parsek][WARN][Route] In-transit delivery aborted: route={name} — source recording changed id={id}`

15.3 Delivery

- `[Parsek][INFO][Route] Delivery: route={name} cycle={n} requested={amounts} actual={amounts} endpointPid={pid} at {body}({lat},{lon})`
- `[Parsek][INFO][Route] Partial delivery: route={name} — {resource} delivered {actual}/{requested}`
- `[Parsek][WARN][Route] Delivery failed: route={name} — no vessels at destination`
- `[Parsek][INFO][Route] Route ledger apply: event={id} type={ROUTE_DISPATCHED|ROUTE_DELIVERED} route={name} targetPid={pid} delta={amounts}`
- `[Parsek][WARN][Route] Delivery rollback/recompute failed: route={name} event={id} targetPid={pid} reason={reason}`

15.4 Endpoint resolution

- `[Parsek][VERBOSE][Route] Endpoint resolve: {type} pid={pid} found={bool} fallback={nearest-compatible/none} candidates={count}`

15.5 State transitions

- `[Parsek][INFO][Route] Status change: route={name} {old} → {new}`

15.6 Timeline events

- `[Parsek][INFO][Route] Timeline event: ROUTE_DISPATCHED route={name} ut={ut}`
- `[Parsek][INFO][Route] Timeline event: ROUTE_DELIVERED route={name} ut={ut} amounts={amounts}`

---

16. Test Plan

16.1 Unit tests (pure logic, no Unity)

Implementation-prep contract tests

- Always-tree single-node commit persists `StartResources` and `EndResources` on the committed root recording. *Catches: manifests trapped only in transient* `CaptureAtStop`.
- `AppendCapturedDataToRecording` / active-recorder flush preserve the documented manifest ownership invariant. *Catches: route analysis seeing trajectory data without matching manifests.*
- Dock merge branch writes route-facing target PID and connection kind. *Catches: always-tree dock candidates with no endpoint identity.*
- Completed dock/undock window serializes transport/endpoint part PID sets and scoped manifests. *Catches: aggregate-vessel false proof.*
- Missing connection-scoped data rejects route creation with "recording lacks logistics proof data." *Catches: accidental prompts for old recordings.*
- Exact `InventoryPayloadItem` extraction treats variant/resource/module-state differences as distinct identities. *Catches: lossy part-name inventory routing.*
- Route origin proof is present only for start-docked non-KSC origins. *Catches: arbitrary nearby-vessel debit.*

ExtractResourceManifest

- Empty ConfigNode → empty manifest. *Catches: null deref on missing PART nodes.*
- Single part, one resource → manifest with one entry. *Catches: parsing errors.*
- Multi-part, same resource → amounts summed. *Catches: overwrite instead of accumulate.*
- Multiple resource types → all present. *Catches: single-resource assumption.*
- Missing RESOURCE node → skipped gracefully. *Catches: null deref.*
- Zero-amount resource → included (maxAmount matters). *Catches: filtering out zeros.*

ComputeDeliveryManifest

- Normal transfer on connection-scoped part PID sets: transport 200→50 LF and endpoint 0→150 LF → delivery 150 LF. *Catches: wrong snapshot pair.*
- Docked aggregate vessel unchanged while transport tank decreases and endpoint tank increases → delivery detected from connection-scoped manifests. *Catches: merged-vessel false negative.*
- Transport tank decreases but endpoint does not retain the cargo → no delivery manifest entry. *Catches: docked consumption masquerading as delivery.*
- Transport gains or endpoint loses a non-ignored resource/payload during the same window → v1 candidate rejected. *Catches: pickup/mixed-transfer windows without algorithmic detection.*
- Stored part variant/resources differ → inventory payload identities do not collapse into one part-name count. *Catches: fungible inventory cargo.*
- LiquidFuel increased across the connection window → no v1 delivery, candidate rejected as pickup-only. *Catches: accidental pickup route.*
- Mixed delivery and pickup deltas after EC/IntakeAir filtering → candidate rejected. *Catches: silently dropping pickup cargo.*

- EC and IntakeAir deltas → ignored. *Catches: environmental noise route creation.*
- No resource/inventory delta → empty manifest (validation rejects). *Catches: false positive.*

Route analysis engine

- Single always-tree dock-transfer-undock window → one stop extracted. *Catches: no stop found.*
- Two always-tree dock-transfer-undock delivery windows → validation rejects v1 candidate. *Catches: charging full multi-stop run while delivering one stop.*
- Dock without matching undock/split → validation fails. *Catches: incomplete pair.*
- No completed route connection windows → validation fails. *Catches: false acceptance.*
- Docking window with no delivery resource/inventory change → validation rejects. *Catches: empty do-nothing route.*
- Resource pickup window → validation rejects. *Catches: accidental negative delivery.*
- Multi-body chain (Kerbin origin, Mun endpoint) → correct body for endpoint. *Catches: assuming same body.*
- Non-DockingPort connection kind → validation rejects in v1. *Catches: premature claw/fuel-line support.*
- Mun/Minmus landed endpoint → `isSurface = true`. *Catches: airless-body altitude misclassification.*
- Non-KSC route without start-docked origin depot PID → validation rejects. *Catches: unproven origin resource debit.*
- Route analysis stores `SourceRefs` fingerprints and per-stop `DeliveryOffsetSeconds` from source UT boundaries. *Catches: timing tied to mutable recording duration.*
- Source recording re-optimized/superseded after route creation → revalidation detects fingerprint drift. *Catches: live route silently shifting delivery boundaries.*

Route validation

- Dock + delivery transfer + undock → valid. *Catches: false rejection.*
- Dock + undock, no delivery transfer → invalid. *Catches: false acceptance.*
- Dock + delivery transfer, no undock → invalid. *Catches: missing undock check.*
- Non-docking connection → invalid in v1. *Catches: unsupported producer accepted.*
- No docking connection → invalid. *Catches: missing connection check.*
- Missing source recording → route status becomes MissingSourceRecording. *Catches: cargo without proof recording.*
- Missing source recording restored on load/revalidation → route returns to Active and recomputes NextDispatchUT. *Catches: unrecoverable missing-source status.*
- Source recording exists but proof/timing fingerprint changed → route status becomes SourceChanged and dispatch stays disabled. *Catches: cargo moving under a rewritten proof run.*

Chain-sequential playback

- `CurrentCycleStartUT = null` and `CurrentSegmentIndex = -1` while idle; dispatch sets `CurrentCycleStartUT` and `CurrentSegmentIndex = 0`. *Catches: ambiguous idle/active state.*
- `CurrentSegmentIndex` indexes `RecordingIds`, not trajectory samples or TrackSections. *Catches: off-by-one against the wrong segment model.*
- UT reaches source recording 0 boundary → scheduler starts source recording 1. *Catches: stuck on first recording.*
- UT reaches last source recording boundary → cycle count incremented, status returns to Active and clears `CurrentCycleStartUT`. *Catches: stuck InTransit.*
- UT reaches stop boundary → delivery triggered even without playback event. *Catches: missed delivery during warp/load/non-flight scenes.*
- Enter flight scene halfway through transit → ghost seeks to scheduler elapsed offset or skips rendering, never restarts at recording UT 0. *Catches: rover-in-the-past visual rewind.*
- Playback completion event without due UT → visual-only hint, no duplicate delivery. *Catches: event/UT double-processing.*
- Source recording deleted while InTransit → status becomes MissingSourceRecording before delivery. *Catches: mid-transit cargo without proof recording.*
- Source recording changed while InTransit → status becomes SourceChanged before delivery. *Catches: mid-transit cargo under rewritten proof data.*
- Pause requested while InTransit → delivery still executes, then route becomes Paused. *Catches: frozen in-flight cargo or accidental future dispatch.*
- Warp past multiple full intervals → catch-up loop dispatches/delivers/completes sequentially until current UT is stable or a blocker is reached. *Catches: one-cycle-per-tick behavior.*

Dispatch evaluation

- KSC origin in Career, capacity available, funds affordable → dispatch and funds spending emitted. *Catches: free KSC mass.*
- KSC origin in Career, funds insufficient → `WaitingForFunds`, `NextDispatchUT` unchanged. *Catches: dispatch without funds or busy skip loop.*
- WaitingForResources/WaitingForFunds/DestinationFull before `NextEligibilityCheckUT` → no origin/destination scan. *Catches: per-frame idle polling.*
- KSC origin in Science/Sandbox → dispatch with no funds action and no `Funding.Instance` access. *Catches: career-only cost leaking or null Funding.*
- Non-KSC start-docked origin, sufficient resources/inventory → dispatch, deducted from origin depot. *Catches: skipping deduction.*
- Non-KSC start-docked origin, insufficient → delayed. *Catches: dispatching without resources.*
- Destination full → `NextDispatchUT` unchanged, `NextEligibilityCheckUT` set, origin NOT deducted. *Catches: wasted deduction and delayed retry.*
- Partial capacity → per-resource independent, full route cost. *Catches: coupled delivery.*
- Non-KSC start-docked origin → deducts from recorded origin depot, not transport or arbitrary nearby vessel. *Catches: wrong debit identity.*
- Two due deliveries to same destination → deterministic FIFO (`NextDispatchUT`, then route id), second route sees capacity after first. *Catches: hidden nondeterministic contention.*
- `LinkedRouteId` set in v1 → ignored by dispatch and no linked-partner log line is emitted. *Catches: accidentally enabling deferred round-trip behavior.*
- ROUTE_DISPATCHED and ROUTE_DELIVERED ledger modules apply and reverse KSC funds, origin debits, and endpoint deliveries in epoch recompute. *Catches: timeline events that cannot undo physical stock mutations.*

Synodic period

- Kerbin-Mun → ~6.4 days. *Catches: formula error.*
- Mun-Laythe → walks to Kerbin/Jool and returns a positive Kerbin/Jool synodic period. *Catches: cross-system hierarchy.*
- Kerbin-Duna/Eeloo → positive Sun-parent synodic period. *Catches: accidentally disabling interplanetary routes.*
- Same body → 0. *Catches: division by zero.*
- Origin or destination is the Sun → 0. *Catches: missing guard without breaking Sun-parent planets.*
- Equal periods → 0. *Catches: T1==T2 division.*
- Inter-body route after pause/load → next dispatch stays aligned to the loop clock's re-aim window basis (the as-built mechanism; the original `DispatchWindowEpochUT + n * DispatchWindowPeriod` arithmetic was never built - see section 0.9). *Catches: phase drift.*
- Player interval override → increases minimum spacing without shifting phase. *Catches: window override misinterpreted as free phase shift.*

KSC cost

- Dry part cost excludes loaded resources and inventory contents. *Catches: double-counting cargo.*
- Resource cost uses `PartResourceDefinition.unitCost * CostManifest amount`. *Catches: free delivered resources.*
- Inventory delivered part cost includes `InventoryPayloadItem` dry part/module cost and stored resources from the captured `STOREDPART` snapshot. *Catches: free inventory parts.*

Endpoint resolution

- PID exists → found. *Catches: skipping PID.*
- KSC-origin route serializes `Origin.vesselPersistentId = 0` and skips origin vessel resolution. *Catches: undefined KSC sentinel.*
- Non-KSC origin with `vesselPersistentId = 0` → invalid. *Catches: sentinel leaking into physical origins.*
- PID gone, surface, nearest compatible vessel within `SurfaceFallbackRadiusMeters` → fallback. *Catches: missing fallback.*
- PID gone, surface, nothing nearby → empty. *Catches: false match.*
- PID gone, orbital → empty. *Catches: orbital fallback.*
- Multiple surface vessels within fallback radius → one nearest compatible endpoint, not aggregate warehouse. *Catches: magic-radius transfer.*
- Ghost map vessels, EVA, flags, debris, space objects, and vessels without compatible tanks/slots are excluded. *Catches: wrong fallback target.*

16.2 Log assertion tests

- Route creation logs manifest, connection kind, and endpoint. *Catches: silent creation.*
- Dispatch delayed logs missing resource and amounts. *Catches: silent delay.*
- Delivery logs requested vs actual amounts. *Catches: silent partial delivery.*
- Status change logs old→new. *Catches: silent transition.*
- Partial delivery logs reason. *Catches: silent partial.*
- MissingSourceRecording logs the missing source id. *Catches: disabled route with no diagnosis.*
- SourceChanged logs expected and actual source proof/fingerprint values. *Catches: opaque disabled route after optimizer rewrite.*
- Dispatch backoff skip logs through `VerboseRateLimited` when `NextEligibilityCheckUT` has not arrived. *Catches: silent guard skip without per-frame spam.*
- Route ledger recompute/rollback failure logs event id, route id, target PID, and reason. *Catches: epoch isolation failure with no diagnosis.*
- `LinkedRouteId` set in v1 does not log partner checks. *Catches: half-enabled round-trip scheduling.*

16.3 Serialization round-trip tests

- Route serialize → deserialize → all fields match. *Catches: missing field.*
- Stops list round-trip with one v1 stop. *Catches: endpoint lost.*
- RecordingIds list round-trip → all IDs preserved in order. *Catches: chain ordering lost.*
- SourceRefs list round-trip preserves route proof hashes, schema generations, sidecar epochs, and source UT bounds. *Catches: SourceChanged detection lost on save/load.*
- Stop `DeliveryOffsetSeconds` round-trips with full precision. *Catches: delivery boundary recomputed from mutable source recordings.*
- ResourceManifest round-trip with full precision. *Catches: locale formatting.*
- InventoryPayloadItem manifests, stored STOREDPART snapshots, and KscDispatchFundsCost round-trip. *Catches: stock cargo/cost data lost.*
- Null LinkedRouteId survives round-trip. *Catches: empty vs null.*
- Null `CurrentCycleStartUT`, `PendingDeliveryUT`, and `NextEligibilityCheckUT` are omitted on save and restore as null. *Catches: sentinel/null mismatch.*
- In-transit route with CurrentCycleStartUT, PendingDeliveryUT, PendingStopIndex, and CurrentSegmentIndex survives. *Catches: transit state lost.*
- Waiting route with NextEligibilityCheckUT survives. *Catches: retry cadence lost.*
- In-transit route with PauseAfterCurrentCycle survives. *Catches: pause request lost across save/load.*
- In-transit route after save/load recomputes actual deliverable capacity at delivery time. *Catches: stale pending amount delivery.*

16.4 Integration tests (synthetic recordings)

- Always-tree recording with dock+delivery+undock route window → route analysis extracts one endpoint. *Catches: tree/path mismatch.*
- Always-tree recording with two delivery docking windows → validation rejects v1 route creation. *Catches: accidental multi-stop behavior.*
- Always-tree recording with dock but no matching undock/split → validation rejects. *Catches: missing tree boundary check.*
- Legacy/base-manifest-only recording without route connection windows → validation rejects with missing proof data. *Catches: false prompts on old saves.*
- Inject route into save → loads correctly with endpoint, manifests, KSC cost, and source ids. *Catches: ParsekScenario integration.*
- Inject route, supersede one source recording, then reload/revalidate → route becomes SourceChanged. *Catches: stale route proofs after optimizer rewrite.*
- Rewind/tombstone a ROUTE_DELIVERED event in synthetic ledger state → endpoint delivery is removed or masked by route module recompute. *Catches: non-recording vessel delta surviving timeline rewind.*

16.5 In-game tests (KSP runtime)

These complement the pure-logic visual-handoff tests in §16.1: unit tests prove the scheduler chooses the right recording-local offset, while in-game tests prove the live ghost playback and loaded-vessel stock APIs honor that decision.

- Loaded-vessel resource delivery uses `Part.TransferResource()`, clamps to current tank capacity, and respects `flowState` / `flowMode`. *Catches: unloaded-only implementation passing unit tests and accidental use of the resource-flow API.*
- Unloaded-vessel resource delivery edits proto resource snapshots and survives save/load. *Catches: loaded-only delivery path.*
- Loaded-vessel non-KSC origin deduction removes resources from the proven origin depot. *Catches: dispatch cost mutation hitting the wrong vessel.*
- Unloaded non-KSC origin deduction edits the proven origin depot snapshots and survives save/load. *Catches: dispatch debit only working for loaded vessels.*
- Always-tree dock merge records the other docked vessel PID and endpoint coordinates for both active-as-target and active-as-initiator paths. *Catches: route proof accidentally pointing at the merged vessel, the transport itself, or a destroyed absorbed vessel lookup.*
- Moving the same stock cargo part between two live `ModuleInventoryPart` containers preserves `InventoryPayloadItem.IdentityHash`. *Catches: unknown stock `STOREDPART > PART` fields that need to be excluded from canonical identity.*
- Loaded `ModuleInventoryPart` delivery reconstructs exact `STOREDPART` payloads and respects slot limits. *Catches: inventory slot accounting only working in serialized ConfigNodes.*
- Unloaded inventory delivery edits `STOREDPARTS` ConfigNodes directly and survives scene reload. *Catches: fragile inventory ConfigNode mutation path.*
- Flight-scene entry during an in-transit cycle starts/seeks the route ghost at scheduler elapsed time. *Catches: visual replay from the beginning after scene change.*

---

17. Deferred Work

The prioritized sequence of these items into a path from v0 to feature-complete is now owned by section 19 (the provenance model + milestone roadmap M1-M6, verified against the code; mirrored under Phase 13 in `docs/roadmap.md`). The system analysis in this section and 17.1 stays valid; section 19 carries the ordering and the target gameplay model (flow closure, provenance taxonomy, disposition doctrine).

- **Record Supply Run helper: SHIPPED (2026-07-07, branch `logistics-m6-closeout`; see 19.4 M6).** v1 now automatically prompts after eligible committed runs: a commit that produces an ELIGIBLE route candidate (RouteCandidateFinder classification; near-misses never prompt) fires a one-time non-blocking ScreenMessage + main-window banner with Open Logistics / Dismiss actions, at most once per tree via a RouteStore-persisted prompted-id set, suppressed during test batches and restore windows. A helper button for intent marking / prompt filtering remains unbuilt (nothing has needed it).
- **Non-docking stock connection producers:** claw/grapple SHIPPED (2026-07-07) as the second connection producer (`ConnectionProducerClassifier`, `RouteConnectionKind.Grapple`; both couple/release ride the same `Part.Couple` / `Part.Undock` events as docking). Stock crossfeed / fuel-line paths stay out of scope by doctrine: they are continuous-flow with no couple event, so they are not window-shaped (19.6 rule 10).
- **Pickup routes: SHIPPED (M3, 2026-06-15, PR #1160).** A route can load a resource or a stored part from a source vessel onto the transport (the reverse of delivery), and a single dock can both deliver and pick up (mixed windows); pickup dispatches and physically debits the source once per cycle via the `RouteCargoPickedUp` ledger row plus a dispatch-keyed replay backstop.
- **Non-KSC undocked-start origins (RETIRED):** v1 non-KSC routes require the Supply Run to start docked to a real origin depot. The deferred "prove origin ownership for an undocked start" item is retired by the provenance doctrine (section 19.2.2): an undocked start with unwitnessed cargo is an incoherent route shape, resolved by a workflow rejection reason (start docked to the origin, record the mining, or launch from KSC), not by a prover. `docs/dev/done/logistics-origin-ownership-proposal.md` is superseded; its origin-debit primitive analysis is absorbed into milestone M1 (19.4).
- **KSC cost tuning:** v1 charges stock-realistic funds for source vessel parts plus used/delivered resources and inventory. The Logistics UI DISPLAYS a recovery-aware net per-run cost (net = gross launch cost minus the actual distance-scaled recovery credits summed over the source tree), shown in Career for KSC-origin routes only. The per-cycle CHARGE now reconciles to this displayed net via a deferred recovery credit (`docs/dev/done/plans/logistics-recovery-credit.md`): the gross launch cost is fronted at dispatch (the dispatch debit and the `KscFundsAvailable` gate stay on gross, so you still front the full build cost), and a `RouteRecoveryCredited` ledger row credits the recovered amount back one dispatch interval later, at the next dock crossing, keyed on the prior dispatched cycle. FundsModule processes the gross debit as a spending and the credit as an earning, so both are reversed through the recalc cutoff walk (rewind) and the tombstone path (re-fly). In steady state the per-cycle net equals the displayed net; the credit is a constant deferred amount, an approximation of each run's physical recovery-landing UT (precise per-run landing with cycle overlap, plan OQ1, was CLOSED won't-build by maintainer decision 2026-07-07 - the constant deferred credit is the ratified permanent behavior; see 19.4 M6).
- **Force dispatch now (RESOLVED via Send Once; no separate action):** the shipped "Send Once" button (`RouteOrchestrator.TrySendOneCycleNow`) already provides manual dispatch: it brings `NextDispatchUT` down to the current UT

to skip the interval wait, but the per-cycle gates (funds, resources, endpoint, loop-clock dock phase, and any future orbital-alignment / transfer-window check) STILL apply, so it delivers at the next valid dock crossing / transfer window, not on click. A literal "fire immediately" that bypassed alignment would deliver with no ghost at the recorded dock (violating 0.9 DEL-2) or outside the synodic window, so it is intentionally NOT provided.

- **Map view integration:** Route lines on the map. SHIPPED (2026-07-06, branch `logistics-m6-map-route-lines`, milestone M6): each committed SAME-BODY route draws its backing recorded legs (launch → dock) as a persistent line on the flight map and the Tracking Station via a new `RouteTrajectoryLineRenderer` that rides the existing `GhostTrajectoryPolylineRenderer` machinery (reuses the pure leg builder + the `TryDrawLeg` draw helper, same stock-orbit style), behind the `showRouteLines` setting (default on). Reads route + recording data through existing accessors only (no `Route` / `RouteStore` fields added); draws in an independent map-camera onPreCull slot and skips any backing recording the per-cycle ghost polyline is drawing this frame (`IsRenderingNonOrbitalLeg`) so it never double-draws the ghost's own trajectory. FOLLOW-UP STATUS: inter-body route lines SHIPPED 2026-07-07 (branch `claude/logistics-interbody-route-lines-7phdtw` , stacked on M5 PR #1238; in-game playtest pending): an inter-body route (origin body != stop body since the 2026-09-02 scope revision below; `DispatchWindowPeriod` != 0 as originally shipped, which no production route ever was) draws its recorded non-orbital legs at the route's ENDPOINT bodies only (launch/ascent at the origin, approach/descent at the destination, still dock-clipped), resolved by the pure `ResolveEndpointBodies` / `FilterLegsToEndpointBodies` across all members; legs recorded in the TRANSFER frame (mid-course burns body-fixed to e.g. the Sun) are dropped because the M5 re-aim pipeline replaces the recorded transfer per launch window, so the transfer render stays with the backing mission's ghost (no static duplicate). A round-trip recording (origin resolves == destination, no dock clip cutting the return) stands the filter down and keeps every leg rather than dropping the far-body arc. The cache signature additionally folds the inter-body window schedule (period, epoch, cadence), and since the scope revision every route also folds its endpoint bodies; same-body behavior is otherwise unchanged. STILL DEFERRED: the static orbital-coast overview (parking/coast arc) is drawn head-gated by the stock conic today, so route lines draw the recorded non-orbital legs only.
- **Dispatch priority for competing routes:** v0 as implemented processes routes in `RouteStore` commit-list order in `RouteOrchestrator.Tick` (the FIFO-by- `NextDispatchUT` wording in 10.12/10.13 was the v0.5 design intent, not the shipped behavior). Player-set priority is milestone M1 (19.4).

Route render scope (the InterBody / SameBody / MalformedMixedBodies rule) - REVISED 2026-09-02

`RouteTrajectoryLineRenderer.ClassifyRouteScope` decides which of the two draw shapes above a committed route gets. Until 2026-09-02 it read `Route.DispatchWindowPeriod` as the authoritative flag, and NOTHING in production ever wrote that field non-zero (`RouteBuilder` hard-codes 0.0), so `InterBody` was unreachable from creation: a real Kerbin → Duna route classified `MalformedMixedBodies` , its line was skipped entirely, and `FilterLegsToEndpointBodies` - the ratified transfer-leg drop - sat behind the same branch and never ran (todo ROUTE-INTERBODY-SCOPE-NEVER-REACHABLE; walk in `docs/dev/research/g10-interbody-route-feasibility.md`).

THE RULE NOW. Scope is derived from what defines it - the pair of places the route connects - because a supply route rides the looped-mission infrastructure: the journey and its cadence are the LOOP's business (`RouteLoopClock.DeriveWindowBasis` , derived per tick from the resolved loop unit and deliberately never persisted; see section 0.9), and the route only adds the resource transfer. There is nothing for the route to compute at creation time.

1. `Route.Origin.BodyName` known and some `Route.Stops[].Endpoint.BodyName` known and DIFFERENT → **InterBody** (basis `Endpoints`). This is the one predicate, `IsInterBodyByEndpoints` , and the endpoint-leg filter gates on the SAME function, so "the filter ran" and "the scope classified InterBody" cannot disagree. Members are EXPECTED to span bodies here, so no member cross-check applies; a THIRD body among the members is the ratified transfer gap that `FilterLegsToEndpointBodies` drops, not a malformation.
2. Origin known and every known stop body EQUAL to it → a declared same-body route (basis `Endpoints`). Its members must agree with that body: a member on another body means the recorded path leaves the pair the route declares, and drawing it whole would paint a cross-body chord → **MalformedMixedBodies**. That is the whole meaning of the malformed reading, plus case 3.
3. No readable endpoint bodies (a default-constructed origin, or no stop carrying a body) → the shipped v1 member-body consistency read (basis `MemberBodies`): all known member bodies agree, or none are known → **SameBody**; they disagree → **MalformedMixedBodies**. With no endpoints there is no authority saying WHICH two bodies are the endpoints, so the safe reading is the shipped decline rather than a guessed inter-body draw.

The per-route-build log line names its own authority so a reader never has to guess which branch answered:

```
[Parsek][VERBOSE][RouteLine] Route scope: route=<8hex> origin=<body> destination=<body> scope=<...> basis=<...>
```

`DispatchWindowPeriod` is KEPT and demoted to informational: the codec still writes and reads it unconditionally, every committed save and fixture keeps its `dispatchWindowPeriod = 0` line, ROUTE nodes round-trip byte-identically, and there is NO schema bump.

It is still reported into the M-A7 render-composition manifest's route record so a reading run can see what a save carries. The route-line cache signature now folds the endpoint bodies (the scope authority) so a scope flip invalidates the cached line the way the period fold used to; an inter-body route additionally folds its window schedule, and a same-body route folds no schedule field.

17.1 Supporting systems required for future work

Getting v0 fuel delivery working required a large stack of NEW systems behind the scenes: route-proof recording metadata + connection-scoped capture (`RouteProofCapture` , `RouteConnectionWindow` , sections 4.9 / 4.10), the route model + store + codec (`Route` , `RouteStore` , `RouteCodec`), the read-only candidate / analysis path (`RouteCandidateFinder` , `RouteAnalysisEngine`), the dispatch scheduler (`RouteOrchestrator` , `RouteDispatchEvaluator` , `RouteLoopClock`), the destination-side delivery writers + capacity probe (`LiveDeliveryWriters` , `LiveDeliveryCapacityProbe` , `RouteDeliveryPlanner`), the endpoint resolver (`RouteEndpointResolver`), the funds path (`RouteFundsCalculator` , `RouteRunCostCalculator` , the Route ledger module + recovery-credit pairing), and the render seam (`RouteBackingMission` , `RouteGhostDriverSelector`). This section surfaces, up front, the NEW supporting systems each REMAINING feature will need that do NOT exist in v0, so the hidden infra cost is visible before committing to a feature. Items were originally indexed by a Tier 1-4 list in `docs/roadmap.md` ; that list is replaced by the milestone sequence (M1-M6) in section 19.4, which carries the priority order (this section still carries the system analysis; the mapping table in 19.4 maps each old tier item to its milestone). Each "Needs" entry is tagged `(shared)` when more than one future feature reuses it, `(feature-specific)` when only this feature needs it, or `(KSP API investigation)` when it needs API spelunking before it can be scoped.

The shared-foundation leverage points (read this first). Two foundations sit under several Tier 2 / Tier 3 features. Building each once unlocks multiple features, which changes how the roadmap should be sequenced:

- **A generalized transfer-DIRECTION model.** v0 hardcodes one direction: cargo leaves the transport part set and lands on the endpoint part set, written by `LiveDeliveryWriters` into the destination, gated by `LiveDeliveryCapacityProbe` ; `RouteAnalysisEngine.HasResourcePickup` REJECTS any reverse flow. Pickup, mixed pickup/delivery, and round-trip all need the reverse direction (read from a source, with an availability probe instead of a capacity probe). Generalizing capture + analysis + the writer/probe pair to be direction-aware ONCE is the bulk of pickup, and mixed and round-trip then reuse it. `(shared)`
- **A multi-WINDOW route data model.** v0 analyzes exactly ONE `RouteConnectionWindow` per run and a single `Stops` entry; `RouteAnalysisEngine` rejects multi-window runs outright, and the scheduler delivers at one dock phase. Multi-stop (several delivery windows in one run) and round-trip (two runs chained) both need the model, scheduler, and per-window delivery clock to handle an ORDERED set of windows rather than one. The `Stops` list and `LinkedRouteId` fields are already reserved in the save shape for exactly this, so the codec cost is small; the analysis + scheduler + per-window delivery cost is the real work. `(shared)`

Resequencing implication. Because the transfer-direction model is shared by pickup, mixed, and round-trip, and the multi-window model is shared by multi-stop and round-trip, it is cheaper to build those two foundations as explicit milestones rather than treat each feature as independent. In particular, round-trip linking was originally tiered as if it were standalone, but it is mostly "pickup (reverse direction) + a two-window/two-route chain constraint": once pickup and the multi-window model (built for multi-stop) both exist, round-trip is a thin scheduling layer on top, not a fresh system. The milestone sequence adopts exactly this: the direction model lands with pickup (M3), the multi-window model with multi-stop (M4), and round-trip immediately after within M4, ahead of the heavier non-docking-producer and crew-delivery items.

Tier 1

- **Mission / route structure list window:** SHIPPED in 0.10.1. Needed a new IMGUI popup window `(feature-specific)` plus a structure / segment extractor `(feature-specific)` . The extractor reads ALREADY-RECORDED data, so the backend infra was LOW: a mission's segments and intermediary points come from the recording `TrackSections` + `SegmentEvents` + `PartEvents` (staging / decouple / dock / undock) with their UTs and the per-point body / coordinate already on each `TrajectoryPoint` , and a route's origin / dock / delivery / undock / stops come from the route's `RouteConnectionWindow` (`DockUT` / `UndockUT` / `EndpointAtDock`), `RecordedDockUT` , and the `Stops` list (each `RouteStop` carries its endpoint and `DeliveryOffsetSeconds`). So this was mostly feature-specific UI over existing data; it reused the recording / route models as-is and added no new capture or scheduler systems. As shipped, the mission step list derives from the cached `MissionStructure` (branch points + terminals) supplemented with debris-staging `PartEvents` (deduped against controlled-decouple branch points by decoupler part PID); the route step list reads `Route.Origin` / `RecordedDockUT` / `Stops` plus the dock-member recording's `RouteConnectionWindow` . Per-step location is the coarse body / launch-site / terminal context; per-step coordinate resolution from `TrajectoryPoint` s is deferred. See `docs/dev/done/plans/plan-structure-list-window.md` .
- **"Dispatch now" action (RETIRED, subsumed by Send Once):** not a remaining item. The shipped "Send Once" button (`RouteOrchestrator.TrySendOneCycleNow`) is the window-aligned manual dispatch: it pulls `NextDispatchUT` to the current UT but keeps every per-cycle gate (funds, resources, endpoint, loop-clock dock phase, future transfer-window check), so it delivers at the next valid dock crossing / transfer window, not on click. A literal fire-immediately that bypassed alignment would break DEL-2 (no ghost at the dock) or the synodic window, so there is no coherent separate feature.

- **Dispatch priority for competing routes:** Needs a route-ordering policy `(feature-specific)` that replaces the v0 ordering — `RouteOrchestrator.Tick` processes routes in `RouteStore` commit-list order — with a player-set or value-based comparator applied to the tick snapshot (priority, then `NextDispatchUT`, then route id), plus a persisted priority field on `Route` (additive codec). Small, self-contained. Pulled forward into milestone M1.
- **Candidate intent helper:** SHIPPED (see 19.4 M6). Needed a per-tree intent flag `(feature-specific)` consumed by `RouteCandidateFinder` to suppress detection; as built the flag is a RouteStore-persisted dismissed-tree-id set in the consuming logistics domain, not a RecordingTree schema field.
- **Dock-side-baseline eligibility edge case:** Needs a pre-couple transport snapshot captured inside `onPartCouple` before stock equalisation, fed to the dock-window builder in place of the post-merge snapshot, OR an approximate-equalisation detector in `RouteAnalysisEngine.HasResourcePickup` `(feature-specific)`. A capture-timing fix on the existing `RouteProofCapture` path, not a new subsystem.
- **Precise per-run recovery-landing: CLOSED - won't-build (maintainer decision 2026-07-07; see 19.4 M6).** Would have needed a second recovery clock `(feature-specific)` keyed on each run's recorded recovery UT that maps run K's recovery into whichever cycle it physically lands in, with cycle-overlap bookkeeping, replacing the v0 constant deferred credit in the recovery-credit pairing (the ledger row type and reversal already exist; the clock only refines WHEN the credit lands). The decision memo `docs/dev/done/research/logistics-recovery-clock-memo.md` showed the divergence is bounded per-cycle timing lead/lag with zero total error, so the constant deferred credit is the ratified permanent behavior.

Tier 2

- **Pickup routes:** Needs the generalized transfer-DIRECTION model `(shared)` end to end: a source-side transfer capture (the mirror of v0's connection-scoped DELIVERY capture, proving cargo left the ENDPOINT and arrived on the TRANSPORT), a source availability probe (the mirror of `LiveDeliveryCapacityProbe`, "is there cargo to pick up / room on the transport"), and reverse-direction writers (the mirror of `LiveDeliveryWriters`, debiting the endpoint and crediting the origin). `RouteAnalysisEngine` must stop rejecting reverse flow and instead classify it. Also needs the existing exact stock-slot / `InventoryPayloadItem` identity tests extended to the pickup direction `(feature-specific)`. This is the feature that pays for the shared direction model.
- **Mixed pickup/delivery windows:** Needs the direction model from pickup `(shared)`, plus a per-window bidirectional manifest in `RouteConnectionWindow` and a two-direction delivery applier in `RouteOrchestrator` so one dock crossing both drops off and picks up `(feature-specific)`. Hard dependency: pickup must land first.
- **Non-KSC origin cargo debit (docked + undocked):** Needs the origin-debit primitive `(shared)` that does not exist in v0: a `LiveOriginDebitWriters` (the mirror of the destination `LiveDeliveryWriters`) that debits the recorded per-cycle cargo from the resolved origin vessel, plus un-stubbing `LiveRouteRuntimeEnvironment.OriginHasCargo` (today it returns false for every non-KSC origin with reason `non-ksc-origin-unsupported-in-v0`, holding the route in `WaitingForResources`). This single primitive gates EVERY non-KSC route, including the start-docked case that already captures origin proof (`RouteOriginProof.StartDockedOriginVesselPid`) but cannot dispatch. The primitive is milestone M1 (docked-start origins only). The UNDOCKED-start case is RETIRED, not deferred: under the provenance doctrine (section 19.2.2) an undocked start with unwitnessed cargo is an incoherent route shape, resolved by a workflow rejection reason (start docked to the origin / record the mining / launch from KSC) — no origin-ownership prover is built.

Tier 3

- **Multi-stop routes:** Needs the multi-WINDOW route data model `(shared)`: `RouteAnalysisEngine` must accept and order N delivery windows instead of rejecting multi-window runs, the scheduler / `RouteLoopClock` must fire a delivery at EACH window's dock phase in sequence, and `RouteEndpointResolver` must resolve one endpoint PER stop. The `Stops` list and `SegmentIndexBefore` / `DeliveryOffsetSeconds` fields are already reserved for this, so the codec is cheap; the analysis + multi-phase delivery clock is the real cost. This is the feature that pays for the shared multi-window model.
- **Round-trip linking:** Needs a cross-route chain-constraint scheduler `(feature-specific)` that consumes the already-reserved `LinkedRouteId` so paired routes alternate (A completes, then B dispatches). Per the leverage note, it ALSO needs the reverse-direction model (pickup) and benefits from the multi-window model, so it is cheapest built AFTER pickup and multi-stop, when both shared foundations already exist; on their own the linking layer is thin.
- **Inter-body re-aimed routes:** Needs the existing same-body delivery clock taught to use the Missions re-aim schedule `(feature-specific)`. The seam already exists and is a no-op in v0: `RouteLoopClock` threads the backing unit's `RelaunchSchedule` / `LoiterCuts` (null for same-body) into `GhostPlaybackLogic.TryComputeSpanLoopUT`. Wiring requires the route's backing Mission to carry the locked Missions layer's synodic / re-aim `MissionRelaunchSchedule` (the `bodyInfo` seam) and the delivery clock to phase-lock to the re-aimed launch UTs. No from-scratch transfer-window solver; it activates an existing seam over the locked `MissionPeriodicity` / re-aim layer.
- **Non-docking connection producers (claw/grapple, stock crossfeed/fuel-line):** CLAW SHIPPED (2026-07-07, branch `logistics-claw-producer`; design `docs/dev/design-logistics-claw-producer.md`, decompile findings `docs/dev/research/claw-grapple-coupling-internals.md`). The anticipated per-producer detection stack was not needed: the decompile proved `ModuleGrappleNode` couples via `Part.Couple` and releases via `Part.Undock`, the exact docking primitives, so

the producer abstraction is a CLASSIFICATION seam (`ConnectionProducerClassifier` stamps `RouteConnectionKind` from the live `onPartCouple` parts) plus producer-aware admission (`UnsupportedConnectionKind` reject for unknown producers; empty Grapple windows skip as non-stops). Stock crossfeed/fuel-line remains OUT with a sharper reason: no couple event, no vessel merge, no window to corner-snapshot (19.6 rule 10).

- **Crew delivery:** Needs a named-roster crew transfer system `(feature-specific)` that moves real KSP roster kerbals (not the generic kerbals the v0 manifests model) and integrates with Parsek's crew-reservation system so delivered crew are reserved/named correctly. Largest of the cargo extensions because it cannot reuse the resource/inventory transfer path; depends on the crew-reservation subsystem.

Tier 4

- **Scenario-lifecycle test hardening:** Needs a Unity test harness `(shared)` (stubbed `Planetarium.fetch`, a `GameEvents` event-bus mock, a `MonoBehaviour` shim) so xUnit can drive `ParsekScenario.OnSave` / `OnLoad` end to end instead of pinning the route hookups with source-text grep gates. Not logistics-specific: about 120 existing harness-touching tests would benefit, so it is best built alongside the in-flight logging audit rather than charged to route work alone.

---

18. Implementation Plan

Implementation should proceed in dependency order. Do not enable the route-creation prompt until phases 1-3 pass their tests.

Phase 0: code-contract alignment

- Add failing tests for the actual always-tree manifest and dock metadata contracts listed in section 4.10.
- Confirm exactly where root/child recordings should own start/end manifests after normal commit, scene-exit commit, dock merge, undock split, and optimizer rewrite.
- Add the route timeline action/module contract before any stock resource/funds mutation code lands: explicit ROUTE_DISPATCHED / ROUTE_DELIVERED `GameActionType` values plus recompute modules for KSC funds, physical origin debit, and endpoint delivery.

Phase 1: recorder prerequisites

- Persist route-facing dock metadata in always-tree mode (`TransferTargetVesselPid`, `TransferKind`, endpoint situation, endpoint coordinates).
- Add `RouteConnectionWindow` capture/serialization on the docked merged recording.
- Add transport/endpoint part PID set extraction and scoped resource manifests at dock and undock.
- Add exact `InventoryPayloadItem` extraction/serialization while preserving existing lightweight inventory manifests.
- Add non-KSC start-docked origin proof.

Phase 2: route core model

- Create `Source/Parsek/Logistics/Route.cs`, route serializers, and `RouteStore`.
- Wire `RouteOrchestrator.OnSave` / `OnLoad` into `ParsekScenario`, no scheduler behavior yet.
- Add route serialization tests for every field, especially nullable UT fields, `SourceRefs`, `DeliveryOffsetSeconds`, and inventory payload snapshots.

Phase 3: route analysis

- Implement `RouteAnalysisEngine` over committed `RecordingTree` data and completed `RouteConnectionWindow`s.
- Implement delivery/cost manifest computation from scoped transport/endpoint deltas.
- Persist immutable source fingerprints and pinned stop delivery offsets so route timing cannot drift when recordings are optimized or superseded.
- Reject old/base-manifest-only recordings with a clear missing-proof reason.
- Add synthetic always-tree route fixtures and analysis tests.

Phase 4: endpoint and delivery primitives

- Implement endpoint resolution by PID with surface fallback only for surface endpoints.
- Implement resource capacity checks and `Part.TransferResource()` / `ProtoPartResourceSnapshot` mutation for loaded/unloaded vessels.

- Implement inventory fit/delivery only after exact payload reconstruction is proven, including loaded `ModuleInventoryPart` insertion and unloaded `STOREDPARTS` editing.
- Keep physical mutation disabled until Phase 5 ledger modules can apply and reverse the same changes.

Phase 5: scheduler and timeline effects

- Implement UT-driven dispatch/delivery state progression.
- Integrate KSC Career dispatch cost, non-KSC origin debit, and endpoint delivery with explicit route ledger modules.
- Implement `SourceChanged` revalidation, DestinationFull backoff without advancing `NextDispatchUT`, and the per-tick catch-up loop that alternates in-transit progression with dispatch evaluation.
- Add rewind/save-load/time-warp tests before any UI prompt creates live routes.

Phase 6: UI and visual playback

- Add post-commit route candidate prompt and route confirmation UI.
- Add route management surface in the logistics/recordings UI.
- Add visual playback handoff/seek as a non-authoritative hint only.

Phase 7: runtime validation

- Run the headless suite.
- Add/inject synthetic recordings for route creation.
- Run focused in-game tests for loaded-vessel resource delivery, origin debit, exact inventory delivery, and mid-transit scene entry.
- Run focused in-game tests for unloaded resource and inventory delivery, including direct `STOREDPARTS` ConfigNode mutation.
- Capture `KSP.log`, `parsek-test-results.txt`, and log-validation evidence.

---

19. Provenance Model and Roadmap to Feature-Complete

The atomic route model (provenance → transfer → disposition), the flow-closure invariant that decides route validity, and the milestone sequence from shipped v0 (Parsek 0.10.x) to a logistics system that is useful, fun, complete, and functional for any route shape the player flies — including resources from other mods, so resource-gathering-to-hubs-to-colonies gameplay works through the transport network.

This section is authoritative for the feature's direction (the second authoritative layer beside section 0; see the Status header). It was merged from the former standalone `parsek-logistics-provenance-and-roadmap.md`. Design direction approved, and every implementation claim verified against the code (2026-06-10, post-0.10.1 main); 19.7 records the verification results, and the body states only verified facts. Boundary notes:

- Section 0 (route-on-Missions architecture) is NOT changed by this section. The Missions subsystem remains LOCKED (0.2); every integration below is a logistics-side consumer.
- This section supersedes the section 17 / 17.1 SEQUENCING (the system analysis there stays valid; the priority order and milestone grouping live here). The old Tier 1-4 list under Phase 13 in `docs/roadmap.md` is replaced by 19.4 (the roadmap mirrors it).
- `docs/dev/done/logistics-origin-ownership-proposal.md` (the undocked-start origin prover, Candidates A-D) is RETIRED by the provenance doctrine in 19.2.2. Its lasting contribution — the origin-debit primitive analysis (`LiveOriginDebitWriters`, the `OriginHasCargo` un-stub, the resolver reuse) — is absorbed into milestone M1.

19.1 The target gameplay, stated once

The player should be able to fly any supply run they can imagine — any direction of transfer, any number of stops, any origin, any bodies, any resources including modded ones — commit the recording, and have the network either run it as a route or tell them exactly why not, in player language. Networks emerge from composition: a mining base feeds a hub by one route, the hub feeds a colony by another, availability gating chains them. No route shape that a player can physically fly and that the system can witness should be rejected for being an unanticipated shape.

The corollary that keeps this simple rather than sprawling: route validity is decided by ONE rule (flow closure, 19.2.4), not by an enumeration of supported mission profiles.

19.2 The atomic route model

19.2.1 The atom

A route is, atomically:

```
provenance events  →  transfer events  →  disposition
(where cargo came  (dock windows where  (what happened to
 from, witnessed)   cargo moved)         the transport)
```

Origin is CAUSAL, not positional. The recording's start location is irrelevant to cargo accounting; what matters is the witnessed event that put each unit of cargo on the transport. A tug that lives docked at a station, flies empty to a refinery, loads, returns, and delivers has the refinery as the origin of the delivered fuel — not the station where the recording started.

19.2.2 Provenance taxonomy

Every unit of cargo on the transport has exactly one of three witnessed provenances:

1. **Launch cargo** — the transport left the launch site with it. Debit: KSC dispatch funds (Career) per the shipped `KscDispatchFundsCost` path; no physical debit. SHIPPED in v0. Verified property: the per-cycle funds charge (`RouteOrchestrator.ComputeDispatchFundsCostForRoute` → `RouteFundsCalculator.ComputeDispatchFundsCost`) walks the FIRST source recording's vessel snapshot, which is captured at that recording's STOP — the dock chain boundary for a `[launch..dock]` transport recording. So the resource term is what is still aboard at dock arrival (the launch load minus pre-dock transit burn) plus parts dry cost (`unitCost × amount` per resource). Cargo loaded in LATER windows happens after that snapshot, so it is structurally outside the funds charge — flow accounting cannot double-charge it; the only open funds question is the charge BASIS (dock-arrival contents vs the launch manifest), decided in M2 once full-run manifests exist (19.2.4). **M2 update (2026-06-12):** the basis is DECIDED - launch manifest for new recordings, stop snapshot otherwise; full decision record in 19.2.4. One wording correction folded in (plan round-2 finding 14): the FIRST source recording is `SourceRefs[0]`, the tree ROOT, and its snapshot is taken at the FIRST chain/branch boundary of the run - so "dock-arrival contents plus surviving parts" describes single-leg `[launch..dock]` runs exactly; multi-leg runs price the root's first-boundary snapshot.
2. **Loaded** — a recorded connection window in which cargo flowed FROM another vessel ONTO the transport (the reverse of v0's delivery direction). Debit: that source vessel's tanks/inventory, per cycle. Requires the transfer-direction model (M3) and the origin-debit primitive (M1: `LiveOriginDebitWriters` + un-stubbing `LiveRouteRuntimeEnvironment.OriginHasCargo`, today `false` with reason `non-ksc-origin-unsupported-in-v0`).
3. **Harvested** — recorded ISRU activity during the run (drills, converters) producing cargo on the transport between windows. Debit: none; the environment provided it. NEW capture work (M2). Verified: the existing snapshots do NOT bracket this — `RouteOriginProof` start/end transport manifests are only populated for start-docked runs (`RouteProofCapture.BuildStartRouteOriginProof` requires an externally-parented partner at recording start), and nothing in the recorder tracks harvester/converter activity. Module scope matters: asteroid/comet mining uses `ModuleAsteroidDrill` / `ModuleCometDrill`, SEPARATE modules from the surface-harvest `ModuleResourceHarvester` (stock drill parts carry the asteroid drill alongside the surface harvester — see `docs/dev/done/deployable-parts-inventory.md`), so an attribution capture watching only `ModuleResourceHarvester` / `ModuleResourceConverter` would leave asteroid-mining gains untracked and failing closure. All of these derive from `BaseConverter` (`TimeJumpManager` already manipulates converters at that base class), so attribution should key on `BaseConverter`-derived module activity rather than an explicit module list — which also covers most modded drills and converters for free. Harvested provenance therefore needs (a) full-run transport-scoped start/end manifests captured for ALL runs, not just start-docked ones, and (b) a harvest attribution surface (either a harvest-window capture analogous to `RouteConnectionWindow`, or per-window bracketing that pins each between-window gain to `BaseConverter` operation). Without attribution, an untracked transport gain is indistinguishable from a glitch and must fail closure. **[SHIPPED in M2 (2026-06-12)]**: both halves built - (a) as the presence-gated `RouteRunCargoManifest` on every active-stop recording and (b) as `RouteHarvestWindow` capture on `BaseConverter`-derived activity threshold crossings, consumed by the analysis gain check; per-item as-built notes in 19.4 M2. Two contracts recorded with the flip. **The catch-up / bridging contract (plan D5):** a converter already ACTIVE at recording start opens its window AT start, so the stock `lastUpdateTime` catch-up burst that fires in the first frames after a vessel loads/unpacks lands INSIDE the open window (a burst landing before the start snapshot sits in both baselines and nets zero - covered either way); a positive boundary delta BETWEEN two legs is admitted as harvested only when the seam lies strictly before the window's dock, both legs' transport pid scopes MATCH, and a window was witnessed open at the boundary - the dock-merge seam is never a bridge operand (the merged-stack manifest would credit the depot's whole inventory as harvested). The empirical load-ordering risk is pinned by in-game investigation tests (`LogisticsHarvestRuntimeTests`) with a manual drill-run playtest item still open in the todo doc. **The Parsek time-jump zero-gain rule:** `TimeJumpManager.FixResourceConverterTimestamps` already resets converter timestamps on every loaded vessel at a Parsek time jump, deliberately suppressing the burst, so a harvest window spanning a jump records zero gain - a jump is not simulated production time; this is intentional, and the recorder carries no jump-awareness of its own.

Undocked-start is not a provenance. A recording that begins with a full transport not docked to anything, where the cargo's source was never witnessed, is an incoherent route shape, not a missing feature. The resolution is a workflow rule surfaced in the rejection reason: start the supply run docked to the origin (making it a Loaded provenance via the start-docked window / `RouteOriginProof.StartDockedOriginVesselPid`), or record the mining (Harvested), or accept it as launch cargo if launched from KSC. The "undocked-start origin prover" deferred-work item (section 17; `logistics-origin-ownership-proposal.md`) is RETIRED by this rule — there is nothing to prove. Verified: the rejection-reason mechanism already carries player-language guidance — `RouteAnalysisStatus` (enum) maps through `RouteCreationFormatters.FormatRejectMessage` and `LogisticsRejectPresentation.DescribeNearMiss` into the Logistics window's near-miss list, so the workflow rule ships as one new `RouteAnalysisStatus` value plus formatter text.

A note kept from the retired proposal: a tanker that DECOUPLES from a base (separation-anchored departure) is witnessable in principle — the cargo sat on the base's combined vessel at the separation frame. If players hit the undocked-start rejection in practice and the workflow rule proves too strict, the revisit path is a witnessed "separated-with" loading variant anchored to the controlled-decouple event, not a positional prover. Deliberately out of scope for every milestone below.

19.2.3 Disposition taxonomy

What happens to the transport after its last transfer window:

1. **Recovered** — lands on Kerbin, recovery credit applies. SHIPPED (constant-deferred recovery credit, `RouteRecoveryCredited` ledger row); the precise per-run landing-timing refinement evaluated under M6 was CLOSED won't-build (2026-07-07) - the constant deferred credit is the ratified permanent behavior.
2. **Expended** — destroyed, crashed, discarded in atmosphere, deorbited. Nothing owed, nothing credited beyond what already applies. Clean.
3. **Persisting** — the transport remains in the world (stays docked, lands and stays, parks in orbit). DOCTRINE: **route transports never materialize.** A route's only real effects are ledger entries and cargo mutations; replaying a persisting disposition per cycle would mean a real vessel accumulating per cycle, which is vessel spawning, not logistics. A run whose transport persists is still a valid route — the transport's ghost simply ends with the rendered segment and no vessel spawns. Verified: no code path spawns a real vessel from a route's backing recording. The guarantee is architectural — `RouteBackingMission.ComputeExcludedIntervalKeys` excludes the post-dock tail from the route's `LoopUnit`, so excluded recordings never render and never reach the playback-completion event that feeds `GhostPlaybackLogic.ShouldSpawnAtRecordingEnd`; the spawn decision itself has no route-aware gate and does not need one. A cheap regression pin (assert route-driven loop units never produce a spawn decision) is an M4 test task, since multi-stop widens the rendered window.

Module delivery that stays docked (delivering station segments, base modules) is therefore NOT a route in this model — it is remote vessel construction, a separate deferred feature ("transfer kind = the vessel itself"). The atom accommodates it later; it is deliberately out of scope here.

19.2.4 The flow-closure invariant (route validity in one rule)

> Per resource (and per exact inventory payload item): launched + Σ loaded + Σ harvested = Σ delivered + consumed + residual, where every loaded/delivered term is a witnessed connection window and every harvested term is a witnessed gain. A recording whose flows close is a valid route; one whose flows do not close is rejected with the specific unaccounted window/quantity named.

Notes:

- The per-window scoped manifests already state how much flowed at each window (matched transport-loss / endpoint-gain via `RouteAnalysisEngine.BuildResourceDeliveryManifest`, and the reverse for loading). There is NO allocation problem and NO proportional-split heuristic: attribution is read off the witnessed windows, summed, and checked for closure.
- Consumption between windows draws from the onboard pool as fungible. Debits are defined by what left each source at its window, not by which units were later burned; fungibility never needs resolving.
- `consumed` and `residual` come from full-run transport-scoped start/end manifests, which M2 generalizes to all runs (today they exist only on start-docked `RouteOriginProof`).
- Cost restatement, verified against the code: the per-cycle funds charge never sees loaded-en-route cargo (it walks the first source recording's end-of-recording snapshot, taken at the dock boundary; see 19.2.2 item 1), so flow accounting does not risk double-charging loaded cargo as funds — the loaded windows debit their physical sources instead. Against the closure equation, the charge's resource term is dock-arrival contents, i.e. the launched term MINUS pre-window consumption (today's charge under-counts burned transit fuel relative to the legacy §5.2's stated "parts plus used/delivered" intent); once M2 lands full-run manifests, the basis can be restated cleanly on the launched term or deliberately kept as-is — an M2 decision, not a blocker. One correction to the legacy text: the shipped `CostManifest` is NOT the full-run positive-decrease described in §5.2;

`RouteBuilder` copies the window's delivery manifest into it. Under flow accounting `CostManifest` is replaced by the per-provenance terms of the closure equation (launched / per-window loaded / harvested), and the funds charge keys on the launched term (subject to the basis decision above). **M2 DECISION (2026-06-12): restated on the launched term.** `RouteFundsCalculator.ComputeDispatchFundsCost` gained a manifest-basis overload: the RESOURCE term prices the full-run START transport manifest (`RouteRunCargoManifest.StartTransportResources` of the `SourceRefs[0]` recording - the tree ROOT, whose snapshot is taken at the FIRST chain/branch boundary, per the round-2 finding 14 wording correction) whenever that manifest is COMPLETE (start half present AND `EndCaptured` - the SAME completeness gate the harvest analysis uses, so the eligibility gate and the emitted charge cannot diverge; both call the one `ComputeDispatchFundsCostForRoute`). Anything without a complete manifest - every pre-M2 recording, ForceStop and background-degraded legs - keeps the legacy stop-snapshot walk byte-identical, so existing routes keep their exact cost. Consequences accepted with the decision: new KSC routes charge the launch-tank contents (burned transit fuel is no longer free), harvested cargo aboard at dock is never billed (which would otherwise violate "Harvested - Debit: none", 19.2.2 item 3), and the PARTS term deliberately stays on the surviving stop-snapshot part set (jettisoned boosters stay uncharged - no costing-grade start part list is recorded, and charging the full launch stack is a bigger economics change than M2 needs). The applied basis is Verbose-logged per computation (`FundsCost basis=launch-manifest|stop-snapshot`).

- This rule is simultaneously the analysis spec and the player-facing error message. Every rejection names the window or quantity that fails closure, through the same `RouteAnalysisStatus` → formatter → near-miss surface verified in 19.2.2.

19.2.5 Multi-origin runs (ALLOWED)

Load 100 at depot A, 200 at depot B, deliver 300 at the station — valid. Flow accounting handles N loaded-provenance windows natively. Two semantics are locked now. **Both are IMPLEMENTED (M4b, branch `logistics-m4-multistop` ; see the 19.4 M4 multi-origin bullet for the as-built map):** the all-or-nothing N-source gate (1) runs in `RoutePickupSourceGate` derived from the route's pickup `Stops` , and the reserve-at-dispatch / debit-at-window-phase / release escrow (2) is the RAM-only `RouteStore` reservation map. The only deferred piece is INVENTORY escrow (window-local; the B3 seam reserves resource manifests only).

1. **All-or-nothing dispatch.** A cycle dispatches only when EVERY provenance source can cover its recorded contribution; otherwise the route holds in `WaitingForResources` naming the short source. No partial dispatch: the recording transferred what it transferred; delivering less is a different mission than the one witnessed. (Replay fidelity is the epistemic basis of the whole system.) Verified reconciliation with the shipped destination side: the implemented per-cycle gate order (`RouteDispatchEvaluator.CheckEligibility`) is sources-valid → endpoint resolution → origin cargo → funds → destination capacity, where a destination with ZERO capacity blocks the cycle (`DestinationFull`) and a partially-full destination does not — the per-resource clamp happens at delivery time in `RouteDeliveryPlanner.PrepareDelivery` . All-or-nothing source gating slots into the existing origin-cargo step (every source must cover its recorded outflow); the destination behavior is unchanged (zero capacity blocks, partial capacity clamps and wastes). Note the legacy §6.1 listed destination before origin; the code order above is the contract.
2. **Reserve at dispatch, debit at window phase.** All sources are gated together at dispatch; the physical debit for each loaded window applies when the route loop clock crosses that window's recorded phase (the loading analogue of DEL-2). Between dispatch and the window phase, the contribution is escrowed against the source so a competing route cannot drain it mid-run and strand an in-flight cycle. Verified host decision: the ledger reservation machinery is funds-specific (balance-projection inside `FundsModule` ; generalizing it to per-resource-per-vessel would mean a parallel projection walker for one use case), so cargo escrow is a lightweight `RouteStore` -owned reservation map (route id → vessel pid → held amounts), allocated when the cycle's sources are gated, released at each window's debit or on cycle abort, reverted with the route rows on tombstone. Timing note, verified: in the shipped loop-route path `RouteOrchestrator.EmitLoopCycle` fires the debit at the DOCK-PHASE crossing (not at cycle start), so for a single-window route there is no dispatch-to-debit gap today; the escrow window first becomes real with multi-window runs (M3/M4), which is when the reservation map lands.

Consequence accepted: **a crashed disposition still debits.** If the run loaded 200 from A and ends in a crash, A loses 200 per cycle — the outflow was witnessed; the cargo's fate afterward is the route's loss, not the depot's refund. The route UI's per-cycle flow display and hold reasons (M6) make a bleeding route visible.

19.3 Scenario families the atom covers

#	Scenario	Provenance	Needs
1	Pad-to-base rover; KSC-to-orbit resupply	Launch cargo	SHIPPED. Multi-stop DELIVERY (one run delivering at several docks, e.g. KSC → Base A → Base B) SHIPPED in M4a (single-window single-stop was always shipped; the N-window delivery firing is M4a).
2	Hub-and-spoke chains (KSC→LKO depot→Mun station→Mun base)	Launch + Loaded	Origin-debit primitive (M1). Multi-stop DELIVERY legs now ride one route (M4a); multi-window PICKUP / multi-origin loads still need M4b.

#	Scenario	Provenance	Needs
3	Shuttle/tanker (station↔refinery, one recording)	Loaded mid-run	SHIPPED (M3a single-window pickup; M4b multi-window: the refinery-load + station-deliver shuttle now rides one route, debiting the refinery at its window).
4	Mining route (drill during run, deliver)	Harvested	Harvest provenance (M2)
5	Sell-at-KSC (pick up at depot, fly home, recover with cargo)	Loaded → Recovered	SHIPPED (M3a single-window pickup) + existing recovery credit. Verified: the recovery credit is the delta in trajectory-point funds, i.e. KSP's own stock recovery valuation, which already includes onboard resources — recovered cargo IS credited. A test, not a system.
6	Multi-origin consolidation (A + B → station)	N× Loaded	SHIPPED (M4b): N loaded windows + all-or-nothing source gating (first-short names the depot) + the RAM-only cargo-escrow reservation map. The source set derives from the route's pickup stops (no new Route field, 19.2.5).
7	Interplanetary any-of-the-above	any	Re-aim seam activation (M5)
8	Modded-resource versions of all of the above	any	Resource generality (M2)

Out of scope by doctrine: persisting-transport accumulation (19.2.3), module/vessel delivery (remote construction, deferred), crew delivery (cannot reuse the cargo path; orthogonal to the resource network; deferred), stock crossfeed/fuel-line producers (continuous-flow, not window-shaped: no couple event or vessel merge means no corners to snapshot; revisit only with a new evidence mechanism), undocked-start origin proving (retired, 19.2.2). Claw/grapple was LIFTED from this list 2026-07-07 (greenlit revisit): the decompile showed it needs no new proof mechanic, and it shipped as the second connection producer (see 17.1 Tier 3 and `docs/dev/design-logistics-claw-producer.md`).

19.4 Milestones

Ordered by gameplay-per-effort. The two shared foundations from 17.1 (transfer-direction model; multi-window model) are built as explicit milestones, each paid for by the first feature that needs it. Milestone 6 runs partly in parallel with all others.

M1 — The network unlock: non-KSC origin debit

The single highest-value item: turns isolated deliveries into a network, because hub-and-colony gameplay in delivery-only form needs nothing else. Chained routes (appendix A.3) start working; `WaitingForResources` already gates route B on route A having fed the hub; the network emerges from composition.

Status: SHIPPED (0.10.2-dev, 2026-06-10, branch `logistics-m1-origin-debit`; plan `docs/dev/done/plans/plan-logistics-m1-origin-debit.md`). All six work items below are built and tested; each carries an as-built note. Two scoping decisions to keep in view: inventory-payload origin debit is DEFERRED to M3 (plan decision D6; a non-KSC route with a non-empty inventory cost manifest holds with reason `inventory-origin-debit-unsupported`), and the physical debit is LOOP-PATH-ONLY (decision D11; the legacy non-loop self-timer path keeps v0 rows byte-identical and never touches origin tanks, because it fires at cycle start rather than the recorded dock phase and has no replay backstop).

- Build `LiveOriginDebitWriters` (the mirror of destination `LiveDeliveryWriters`: same loaded `PartResource.amount` / unloaded `ProtoPartResourceSnapshot` write split, removing from the origin), plus the physical `RouteCargoDebited` row. **[SHIPPED, with an as-built correction]** This bullet's draft wording "applier path through `RouteModule` ... exactly like the funds debit" was wrong and is corrected here: nothing physical applies through `RouteModule`, which is observe-only by the pinned T-ROUTEMODULE-OBSERVE contract (mutating vessel cargo from the recalc walk is the one forbidden move). As built, the debit follows the DELIVERY pattern: a direct write at emit time inside `RouteOrchestrator.EmitDispatchDebit` (loop path only, D11), with the `RouteCargoDebited` row carrying the actual-debited manifest, requested-on-shortfall, and origin vessel pid as observe-only bookkeeping. Revert safety comes from the rewind quicksave restoring vessel state plus ELS cutoff/tombstone exclusion keeping the `(RouteId, RouteCycleId)` replay keys consistent, same as the delivery side.
- Un-stub `LiveRouteRuntimeEnvironment.OriginHasCargo` (was: `false` with reason `non-ksc-origin-unsupported-in-v0`): resolve `route.Origin` through the existing `RouteEndpointResolver` call already wired in `RouteDispatchEvaluator.CheckEligibility`, sum the live origin manifest, compare against the route's recorded outflow. **[SHIPPED]** As built: `LiveOriginCargoProbe` sums stored deliverable amounts (same flow-state / NO_FLOW gate as the writers, so the gate counts only what the debit may

remove) and the pure `RouteOriginCargoCheck.HasRequired` gates all-or-nothing against `route.CostManifest`, naming the first short resource in ordinal order.

- Docked-start origins only — `RouteOriginProof.StartDockedOriginVesselPid` is already captured and serialized; this milestone makes such routes dispatch. Known capture gap to fix here: `RouteBuilder` leaves the docked origin's `RouteEndpoint` with `IsSurface = false` and zeroed coordinates, so resolution is PID-only; populate the descriptor (situation + coordinates at recording start) so surface-base origins get the same rebuild fallback destinations have. **[SHIPPED]** As built: `OriginPartnerCandidate` / `RouteOriginProof` carry the docked partner's body, coordinates, and situation (deliberately EXCLUDED from the proof hash so existing routes do not flip `SourceChanged` on load), and `RouteBuilder` builds a surface-typed origin endpoint, so surface origins reach the destination-style proximity rebuild fallback, accepting the same nearest-vessel-within-500-m misresolution tradeoff destinations already accept.
- Add the undocked-start workflow rejection: a new `RouteAnalysisStatus` value whose formatter text states the rule (start docked to the origin / record the mining / launch from KSC), replacing the retired prover item. **[SHIPPED]** As built: `RouteAnalysisStatus.UndockedStartOrigin`, checked in the analysis engine itself, so an undocked start surfaces as a near-miss with the workflow guidance instead of failing later at create time with `endpoint-missing`.
- Pull **dispatch priority** forward from the old Tier 1 into M1: verified, today's ordering is not even FIFO-by-`NextDispatchUT` — `RouteOrchestrator.Tick` processes routes in `RouteStore` commit-list order, so contention at a shared hub would resolve by creation order. Add a persisted priority field on `Route` (additive codec) and a deterministic comparator (priority, then `NextDispatchUT`, then route id) applied to the tick snapshot. **[SHIPPED]** As built: sparse `Route.DispatchPriority` (lower dispatches first, floor 0, default 0 writes nothing), `CompareRoutesForTick` applied to the tick snapshot, and a `[-] N [+]` stepper in the route detail panel. Pre-M1 saves: equal-priority routes reorder from commit-list order to the comparator's `NextDispatchUT` mid-key order (deterministic; visible only under same-tick contention).
- Build the **scenario-lifecycle test harness** (old Tier 4; open todo item "Logistics scenario-lifecycle coverage relies on source-text gates") alongside M1, not after: origin debit is the first feature mutating non-KSC vessel state per cycle (loaded AND unloaded vessels) and deserves lifecycle-level `OnSave` / `OnLoad` tests rather than grep gates. **[SHIPPED]** As built as a 3-layer shape (plan decision D10): xUnit codec round-trips, ORDERED source-text gates (`LoadRoutesFrom` must precede `RevalidateSources("OnLoad")`), and in-game lifecycle tests (`LogisticsOriginDebitRuntimeTests`) covering the loaded and unloaded (proto-snapshot) debit through the production tick crossing, the empty-origin hold, and a real `GamePersistence.SaveGame` round-trip of the unloaded debit. The full Unity-shim harness was deliberately not built (about 120 files would be affected for no added M1 risk coverage).

M2 — Resource generality: mod resources + harvest provenance

Earlier than the old tier order implied, because the colony gameplay lives in mods (MKS, USI-LS, EL RocketParts, Karbonite) and the capture path is resource-agnostic by construction. Verified audit result: the pipeline has NO stock-name behavioral assumptions — the only hardcoded resource names are the intentional `ElectricCharge` / `IntakeAir` exclusions (mirrored in `RouteAnalysisEngine.IsIgnoredResource` and `VesselSpawner.ExtractResourceManifest`), manifests are name-keyed dictionaries, and `LiveRouteRuntimeEnvironment.LookupResourceUnitCost` resolves ANY `PartResourceLibrary` definition (modded included), returning 0 for undefined names. So M2 is fixtures + the stated rule + one new provenance kind:

Status: SHIPPED (0.10.2-dev, 2026-06-12, branch `logistics-m2-resource-generality`; plan `docs/dev/done/plans/plan-logistics-m2-resource-generality.md`, three review rounds folded in). All four work items below are built and tested with per-item as-built notes, and the 19.2.4 funds-basis question this milestone deferred is DECIDED (launch-manifest basis for new recordings; see 19.2.4). One scope correction up front: the "mostly fixtures + one provenance kind" framing understated the ROUTE side - a pure-harvest run that analyzes Eligible still could not become a route (`RouteBuilder` rejected originless runs `endpoint-missing`) or dispatch, so scenario family 4 needed a third origin kind on `Route` (plan D7) plus a debit reduction for mixed runs (plan D8); both shipped here, as-built notes inside the Harvested-provenance item.

- **Transferability rule, stated:** any resource with a `PartResourceDefinition` is routable; ElectricCharge and IntakeAir stay excluded as environmental noise; an undefined resource name in a snapshot is excluded and logged. Zero-cost defined resources route normally (they simply contribute 0 to the funds charge). **[SHIPPED]** As built (M2 Phase 1): the rule has one authority, `ResourceTransferability` (`Source/Parsek/Logistics/ResourceTransferability.cs`, `IsRoutableResource(name, out excludeReason)`): a name is routable iff it is not always-ignored (ElectricCharge / IntakeAir, the rule text migrated from `RouteAnalysisEngine.IsIgnoredResource`, which now delegates) AND `PartResourceLibrary` carries a definition for it. The undefined-name exclusion is DIRECTION-SENSITIVE (plan D2): undefined names are excluded only from admission-direction outputs (`BuildResourceDeliveryManifest`, and through it the route's `CostManifest` and funds charge) and logged per name (`Resource excluded: name=... reason=undefined recording=...`, Info on one-shot Diagnostic calls, one shared rate-limited Verbose key on the ~1/second candidate sweep); rejection-direction checks (`HasResourcePickup`) keep seeing undefined names, so uninstalling a resource mod degrades a delivery to NoDeliveryManifest or keeps its rejection but can never flip a rejection to Eligible. Capture stays permissive (`VesselSpawner.ExtractResourceManifest` untouched; recordings are immutable witnesses), so reinstalling the mod restores routability, and proof serialization plus `RouteProofHasher` stay name-agnostic (an undefined name round-trips verbatim and keeps its place in the fingerprint). A null `PartResourceLibrary` (xUnit, early load)

treats names as defined with a one-shot log instead of rejecting everything headlessly; tests inject `ResourceTransferability.DefinitionLookupOverrideForTesting`.

- Community Resource Pack test fixtures (manifest capture, analysis, delivery writers, funds cost with modded `unitCost`) — covers most of the mod ecosystem in one move. **[SHIPPED]** As built (M2 Phase 1, plan D11): one shared constants class, `Source/Parsek.Tests/Generators/CrpFixtures.cs` (`Karbonite` / `MetallicOre` as the zero-cost defined resource / `Uraninite` / `Supplies`, each with a representative unit cost, plus one deliberately UNDEFINED name standing in for an uninstalled mod's resource), threaded through the manifest-capture, analysis, delivery-manifest, origin-cargo, proof-serialization, proof-hash, and funds-cost suites. Funds fixtures price modded unit costs through the calculator's injected lookup (the seam already existed); definition lookups inject `ResourceTransferability.DefinitionLookupOverrideForTesting`; hash stability across name sets is pinned. No Unity shim and no new harness were needed.
- **Harvested provenance** (19.2.2 item 3): generalize the full-run transport-scoped start/end manifest capture to all runs (today start-docked only), add the harvest attribution capture (harvest windows or bracketed gains pinned to harvester activity), and teach the analysis engine to admit witnessed harvested gains into closure. Attribution must cover the full stock module set — `ModuleResourceHarvester`, `ModuleResourceConverter`, AND the separate asteroid/comet drills (`ModuleAsteroidDrill` / `ModuleCometDrill`); key on `BaseConverter`-derived activity so asteroid-mining routes and modded converters do not fail closure as untracked gains. **[SHIPPED]** As built (M2 Phases 2-5, plan D3-D8/D13), with the scope honesty stated where the bullet text overstated it:
  - **Full-run manifests are a NEW node, not the legacy fields.** `RouteRunCargoManifest` (transport part-pid scope captured at recording BIRTH, start/end transport resources, an explicit `EndCaptured` completion marker) is written once at birth and completed only on ACTIVE stops. The legacy Phase-11 `Recording.StartResources` / `EndResources` were deliberately left untouched: they are display-grade (full-snapshot scoped rather than transport-pid scoped, degraded by optimizer splits/merges, not proof-grade), and the new node's PRESENCE is the clean old/new discriminator - every pre-M2 recording analyzes exactly as before, no migration. "All runs" reads precisely as "all ACTIVE-STOP recordings": a ForceStop / scene-change leg never completes a manifest, and a leg that transits BACKGROUND has its manifest VOIDED; both degrade that tree to legacy analysis (a stated degrade, not a rejection).
  - **Harvest windows** (`RouteHarvestWindow`) capture `BaseConverter`-derived activity threshold crossings exactly as specified (base-class keying, no module list; asteroid/comet drills on an ALREADY-GRAPPLED asteroid are covered - a mid-run claw grab is a merge boundary, M4 territory). Open-at-start / close-at-stop handling, a rails-entry/exit re-baseline so warp-period production stays witnessed, a false-alarm-resume unwind of the abandoned-stop close, and the open-time location recorded for the route-side endpoint below.
  - **Analysis admission:** `RouteHarvestAnalysis.CheckTransportGains`, engaged only when the transport lineage carries COMPLETE run manifests (else legacy, logged). Per defined resource, the anchor-to-dock full-run gain must be covered by witnessed window deltas plus scope-matched boundary bridges (the dock-merge seam is never a bridge operand; the gain anchor is the first scope-matching lineage leg, not the tree root); an uncovered gain rejects with the new `RouteAnalysisStatus.UntrackedCargoGain` naming resource and quantity through the formatter / near-miss surface. The undocked-start gate is refined, not removed: an undocked start whose delivered resources are FULLY harvest-covered becomes Eligible as a harvest origin; partial coverage keeps `UndockedStartOrigin`.
  - **Route shape (D7, the understated part):** `Route.IsHarvestOrigin` - a display-only origin endpoint built from the FIRST harvest window's open location, an EMPTY `CostManifest` (Harvested debits nothing, 19.2.2 item 3), dispatch eligibility skipping origin-endpoint resolution, and the `RouteDispatched` / `RouteCargoDebited` row pair still emitted per cycle for row-shape stability (a structural no-op: empty manifest, zero funds, no physical write).
  - **Debit reduction (D8):** mixed docked-origin + harvest runs reduce the depot debit, `CostManifest[r] = max(0, delivery[r] - harvested[r])` with zero entries REMOVED, delivery manifests untouched. This subtracts one witnessed term from the assume-start-loaded basis; it is NOT flow accounting, which replaces it in M3.
  - **Plausibility cap (HARVEST-ROUTE-PLAUSIBILITY-CAP, owner ruling 2026-10-06):** a route credits harvest every run, so a stock catch-up burst inside an open window (production of hours of absence landing in one frame, e.g. at the load-time window open, or across a chain seam bridged by D5) must not be promised again each cycle. Each window stores `RouteHarvestWindow.PlausibleRates`, the per-resource units/s its transport's WHOLE converter set could produce (running or not), sampled at open and at an in-flight close (larger rate per resource wins): a stock surface or ocean `ModuleResourceHarvester` gives `abundance x Efficiency x EfficiencyBonus x crew bonus` (abundance read from `ResourceMap` at the vessel's location, as the drill reads it; the heat throttle only lowers the rate and is left out), a `ModuleResourceConverter` gives its recipe outputs times the same multiplier, the science lab gives nothing, and so does a `ModuleAsteroidDrill` / `ModuleCometDrill` (both stock drills carry one) on a vessel with no asteroid or comet part; any other converter (atmospheric / exospheric harvesters, an asteroid or comet drill with a space object aboard, modded subclasses) leaves the window unbounded (`PlausibleRates` null), as does a window recorded before the cap. A window's plausible harvest is `min(delta, rate x duration x 1.5)`; a D5 bridge's is `min(delta, rate x UT gap x 1.5)` with the boundary window's rates (a zero-length seam credits 0). The gain check, flow closure and harvest-origin gate keep reading the WITNESSED totals (the burst is real cargo that arrived); only the per-run credit reads the plausible totals: a harvest-origin route's deliveries are clamped to them (stops share one budget in stop order) and the D8 reduction subtracts them. The capped resources persist on the route (`Route.HarvestCaps`, sparse `HARVEST_CAP` nodes) and the route's from/to hover

says so ("Harvest per run capped at 75.0 Ore, what the drills could make in the recorded time."). The rates are hash-excluded like the other window metadata.
    - **Optimizer policy (D13):** a split voids the run manifest + harvest windows on BOTH halves (presence gate degrades that tree to legacy, cleanly); auto-merge refuses only when harvest WINDOWS are present on either side; window-less run manifests compose on matching pid scope (first start + second end) and void on scope mismatch.
    - **The M3 boundary, drawn explicitly:** M2 implements only the GAIN side of flow closure. No `consumed` / `residual` checks, no pickup direction (`HasResourcePickup` / `MixedPickupDelivery` unchanged - drilling while docked at the DESTINATION still rejects), pre-anchor gains (drilling while docked at the ORIGIN before the undock split) are outside the checked span, no inventory closure, single-window only (N>1 is M4), and BG-leg harvest capture deferred per the void-then-degrade rule above. The D5 catch-up / bridging contract is recorded in 19.2.2 item 3; the empirical catch-up ordering has in-game investigation tests (`LogisticsHarvestRuntimeTests`) plus a pending manual-playtest item in the todo doc.
- **Stated non-goal, in the doc explicitly:** background production. Stock runs no converters on unloaded vessels; logistics reads what is physically in the tank at dispatch evaluation (already true). Whether tanks refill between cycles is the player's problem or a background-processing mod's (Phase 16 mod compat). Without this sentence, the first MKS user files it as a logistics bug. **[STATED, the paragraph below is the deliverable]** Unloaded vessels run no converters between route cycles: stock executes no converter code on unloaded vessels, and Parsek adds none. A mining route therefore delivers what the RECORDED run harvested, once per cycle; it does not simulate the depot's drills running while nobody watches, and it does not refill origin tanks between cycles. Logistics reads what is physically in tanks at dispatch evaluation, so if a background-processing mod (MKS, USI, etc.) fills the tanks, routes benefit automatically; if nothing does, a depleted origin holds with `WaitingForResources` exactly like any other empty depot. Background-resource-converter simulation is a mod-compatibility concern (Phase 16), not a logistics feature.

M3 — Direction generality: pickup, then mixed windows

The generalized transfer-direction model, paid for by pickup:

Status: SHIPPED + in-game validated (M3a, branch `logistics-m3-direction` , PR #1160; plan `docs/dev/done/plans/plan-logistics-m3-direction-generality.md` , reviewed 0 blockers, full-milestone review ship-it). All 7 phases done; the 5 `LogisticsPickupRuntimeTests` passed live 2026-06-15 (loaded / unloaded / short resource debit, loaded inventory `ClearPartAtSlot` + `IdentityHash` match, identity-not-held clamp), 0 Parsek errors, full xUnit suite green (15676). M3 = the pickup DIRECTION on a single bidirectional connection window (M3a); multi-origin + escrow (the former final bullet) are RESEQUENCED to M4 (plan OQ1). Key as-planned facts: connection-window capture is ALREADY symmetric so pickup needs no recorder change; `HasResourcePickup` / `HasInventoryPickup` flip from reject to classify; a new `RouteCargoPickedUp` ledger row + a re-keyed replay backstop close the pickup-only reload-idempotency hole; inventory pickup is window-local (OQ3) and lifts the M1 inventory-origin-debit carve-out (D7).

- Source-side connection capture (mirror of delivery capture: cargo left the ENDPOINT part set and arrived on the TRANSPORT part set across the window — the same `RouteConnectionWindow` scoped-manifest mechanics, opposite sign). **As built (Phase 1, branch `logistics-m3-direction`):** NO new capture work was needed - `RouteConnectionWindow` already stored all four scoped corners (Dock/Undock x Transport/Endpoint, resources AND inventory) symmetrically, so the pickup quantities (`endpointLoss = Dock - Undock` endpoint, `transportGain = Undock - Dock` transport) are derived from existing data at analysis time. No recorder hook, no new `RouteConnectionWindow` field, no `RouteProofHasher` / `RouteProofCapture` change, so pre-M3 recordings stay byte-identical (pinned by `Hash_PreM2Recording_ByteStable`).
- Source availability probe (mirror of `LiveDeliveryCapacityProbe`). **As built (Phase 3 resource / Phase 5 inventory, branch `logistics-m3-direction`):** resource availability REUSES the M1 `LiveOriginCargoProbe` re-aimed at the per-window pickup endpoint (sums stored deliverable amounts under the same flow-state gate as the remove writer); inventory availability is the new `LiveInventoryPickupWriter.CountStored` , which counts stored slots matching a payload's `IdentityHash` (loaded: walk `ModuleInventoryPart.storedParts` ; unloaded: walk the proto `STOREDPARTS` nodes). Both gate all-or-nothing and clamp-and-warn on a short source.
- Reverse-direction writers (debit endpoint, credit transport — at window phase per 19.2.5; note the loop path already fires effects at the dock-phase crossing via `EmitLoopCycle`). **As built (Phase 3 reverse resource writer + source probe + planner refactor, D5, branch `logistics-m3-direction`):** the reverse resource writer is the M1 origin-debit machinery re-aimed at the per-window pickup ENDPOINT, decoupled so it works on a TARGET vessel + a prepared plan independent of which manifest produced the plan. `RouteOriginDebitPlanner.PrepareDebit` gained a `(Dictionary<string,double>, IOriginCargoProbe)` overload (the `(Route, …)` overload now delegates with `route.CostManifest` , so the M1 path is byte-behaviour-identical); `LiveOriginDebitWriters` no longer stores a `Route` (the M1 ctor delegates to a route-agnostic `(string routeIdForLog, Vessel, plan, isLoaded)` ctor, the removal arithmetic + flow gate untouched); source availability REUSES `LiveOriginCargoProbe` pointed at the endpoint. The reusable per-window applier `RouteOrchestrator.ApplyPickupDebit(endpoint, pickupManifest, env, routeIdForLog)` mirrors `ApplyOriginDebit` : resolves the endpoint via `RouteEndpointResolver` , captures the loaded gate ONCE for THAT vessel (per-vessel, never hoisted), probes + plans + writes, and returns an `OriginDebitOutcome` (actual debited, requested-on-shortfall, endpoint pid, short, unresolved).

RESOURCES ONLY (inventory remove is Phase 5). It is NOT wired into `EmitLoopCycle` / `ProcessLoopRoute` and emits NO ledger row — the two-direction applier + `RouteCargoPickedUp` row + replay re-key are Phase 4 (a `PickupDebitApplierForTesting` seam is staged for those tests). The "credit transport" half stays bookkeeping-only (no physical write).

- `RouteAnalysisEngine.HasResourcePickup` stops rejecting reverse flow (`MixedPickupDelivery`) and classifies it; flow accounting (19.2.4) replaces the assume-start-loaded analysis. **As built (Phase 1, branch `logistics-m3-direction`):** `RouteAnalysisEngine` now builds a `ResourceLoadManifest` (the sign-flip mirror of the delivery manifest, `loaded = min(endpointLoss, transportGain)` per routable name, undefined names excluded just like delivery; `HasResourcePickup` stays as the rejection-direction presence detector that keeps seeing undefined names). A resource pickup classifies and admits (pure-pickup / mixed both Eligible) instead of rejecting; `MixedPickupDelivery` (value 4, unchanged) now fires only for a stored-part (inventory) pickup, which Phase 5 lifts. The `NoDeliveryManifest` gate widened to "no delivery AND no load" so a pure-pickup window reaches Eligible (D4 gate fix a), and the M2 harvest gain check now treats the window's loaded term as a witnessed source so a pickup does not false-reject as `UntrackedCargoGain` (D4 gate fix b). Accepted behavior: a mixed window carrying a DEFINED delivery plus an UNDEFINED-name pickup admits as pure delivery and silently drops the undefined pickup (the undefined name is excluded from both admission manifests), consistent with the pre-1.0 drop-undefined-on-admission doctrine - no phantom flow in either direction. New `ComputeFlowClosure` (presence-gated on complete run manifests per OQ2) rejects `FlowDoesNotClose` (value 8) only on over-delivery (pre-clamp slack `< -GainEpsilon` : more left aboard than `launched + loaded + harvested - delivered`); positive slack is legitimate consumption. The closure sums over a window LIST (length 1 in M3a) so M4 multi-window is a fill-in. **As built (Phase 2 route-shape + codec, D8/D9):** `RouteStop` gains `PickupManifest` (`Dictionary<string,double>`) + `InventoryPickupManifest` , the exact mirror of the delivery manifests, serialized by `RouteCodec` as sparse `PICKUP_MANIFEST` / `INVENTORY_PICKUP_MANIFEST` nodes (verbatim copies of the delivery call sites, omitted when empty, empty→null on load), so a pre-M3 / delivery-only route writes nothing new and round-trips byte-identically. `RouteBuilder` populates `PickupManifest` from the Phase-1 `ResourceLoadManifest` (defensive copy) and ADMITS an originless pure-pickup run (no KSC launch, no docked-origin proof, no harvest origin, a populated load manifest, no delivery): the dock endpoint becomes the display origin (`origin=pickup:pid=...` , debited later at the per-window applier, NOT funds), so the route's `CostManifest` stays EMPTY (mirrors harvest origin). A MIXED window with no conventional origin still rejects `endpoint-missing` (the pure-pickup admission is `hasResourceLoad && !hasDelivery` -gated). No `RouteConnectionWindow` field and no `RouteProofHasher` / `RouteProofCodec` / `RouteProofMetadata` change - the pickup direction is DERIVED at analysis time, so the recording proof hash stays byte-identical (pinned by `Hash_PreM2Recording_ByteStable` , and `RevalidateSources` does not flip pre-M3 / pickup-carrying routes). `InventoryPickupManifest` is route-shape only in Phase 2 (left null for every built route); the inventory pickup applier is Phase 5. Phase 1 (analysis) + Phase 2 (route-shape + codec) + Phase 3 (reverse resource writer + source probe + planner refactor, see the reverse-direction-writers bullet above) are done; the two-direction applier + `RouteCargoPickedUp` ledger row + replay re-key (Phase 4), and inventory pickup (Phase 5) follow.
- Exact stock-slot / `InventoryPayloadItem` identity tests extended to the pickup direction. **As built (Phase 5 inventory pickup, D7 / OQ3, branch `logistics-m3-direction`):** the inventory pickup is the sign-flip mirror of inventory delivery, WINDOW-LOCAL per OQ3 (no full-run `Start/EndTransportInventory` fields on `RouteRunCargoManifest`). Analysis: `RouteAnalysisEngine.BuildInventoryLoadManifest` builds the load manifest per exact `InventoryPayloadItem.IdentityHash` (`loaded = min(endpointLoss, transportGain)` , identity carried intact, the source-side STOREDPART as the canonical copy), and `HasUnwitnessedInventoryGain` is the non-fungible window-local closure: a transport inventory GAIN with NO matching endpoint LOSS fails closed (inventory has no harvested provenance) — `MixedPickupDelivery` (value 4) now fires ONLY for that unwitnessed gain, not for a clean inventory pickup, which classifies into `RouteAnalysisResult.InventoryLoadManifest` and admits. `RouteBuilder` populates `RouteStop.InventoryPickupManifest` from it (deep copy; Phase 2 left it null). Source probe + remove writer: the new `LiveInventoryPickupWriter` LOCATES a stored item by `IdentityHash` (loaded: walk `ModuleInventoryPart.storedParts` , hash each slot's `StoredPart` via `StoredPart.Save` → `ComputeInventoryPayloadIdentityHash` , lowest-slot deterministic partial-match; unloaded: walk the proto `STOREDPARTS` nodes) and REMOVES it (loaded `ClearPartAtSlot` , unloaded STOREDPART proto-node removal — the inverse of the delivery store); the transport credit is bookkeeping only (no physical transport write). The identity hash is stable cross-vessel (vessel-local transients stripped) and proto-vs-loaded (the `StoredPart.Save` shape equals the recorded depot snapshot shape after the canonical slotIndex/quantity strip). Applier + ledger row: `RouteOrchestrator.ApplyInventoryPickupDebit` (sibling of `ApplyPickupDebit` , its own `InventoryPickupApplierForTesting` seam) is wired into `EmitPickupHalf` alongside the resource pickup under one `cycleId` ; the `RouteCargoPickedUp` row gains sparse `ROUTE_INVENTORY_MANIFEST` / `ROUTE_REQUESTED_INVENTORY_MANIFEST` codec nodes carrying the picked-up stored-part payloads (omitted when empty, so a resource-only pickup row stays byte-identical) — the FIRST ledger-row inventory manifest (delivery inventory is applied physically, never recorded on the row). Idempotency rides the Phase-4 dispatch-keyed replay backstop unchanged. **Carve-out (D7):** the M1 `inventory-origin-debit-unsupported` hold (`LiveRouteRuntimeEnvironment.OriginHasCargo` + `RouteOriginCargoCheck.RequiresInventoryDebit`) is LIFTED — the same `LiveInventoryPickupWriter` now serves the origin-dispatch inventory debit (`ApplyOriginDebit` removes `route.InventoryCostManifest` items, the `RouteCargoDebited` row carries them via the same sparse codec), gated all-or-nothing by `RouteOriginCargoCheck.HasRequiredInventory` (`CountStored` by identity), so a docked-origin route that delivers stored parts now debits them at the origin instead of holding forever. The retired `inventory-origin-debit-unsupported` token is replaced by an `inventory:<hash>` short token rendered by `LogisticsHoldPresentation` as a missing-stored-part hold. **SUPERSEDED 2026-09-02 ON THE MATCHING UNIT ONLY (branch `inventory-kind-matching` , operator ruling): stored inventory cargo is NOT non-fungible.** Parts inside an inventory are

GENERIC - identity matters for a mission-defining part only while it is PART OF A VESSEL, and a vessel whose core is pocketed into an inventory has ended its mission. `InventoryPayloadItem.IdentityHash` keeps its name and its serialized `identityHash` key (no format change, no schema-generation bump) but now carries a KIND KEY: `VesselSpawner.ComputeInventoryPayloadKindKey` hashes part name (dot-form) + the selected variant (`ResolveStoredPartVariantName`) + per stored resource its name plus a fill bucket (`empty` under 1% of max / `full` over 99% / `partial`; ElectricCharge and IntakeAir excluded), and NOTHING else - module state is ignored entirely apart from the variant selection, and slot index, quantity, stack capacity and the ProtoPartSnapshot transients never enter the string, so the old `TransientStoredPartProtoValueNames` skip-list is gone. That closes LOGISTICS-INVENTORY-IDENTITY-HASH-BREAKS-ON-A-LIVE-CARGO-MOVE: stock's `StoreCargoPartAtSlot(Part, int)` re-serializes a LIVE part on every in-flight move, so any module writing a computed value in `OnSave` used to re-hash the item in transit. Everything else above stands - the closure is still window-local and still fails closed on an unwitnessed gain, only the witness is now per KIND, quantities aggregate per kind wherever items are collected, and saves written before the ruling self-heal at load through `VesselSpawner.NormalizeLoadedInventoryPayloadItems` (recompute from each item's own `STOREDPART_SNAPSHOT`, merge items that now share a kind). The player-facing `MixedPickupDelivery` copy no longer claims inventory is non-fungible, and the reject reason now reaches the log at Info level naming the part.

- Then mixed pickup/delivery windows: per-window bidirectional manifest in `RouteConnectionWindow`, two-direction applier in `RouteOrchestrator` at one dock crossing. **As built (Phase 4 orchestrator two-direction applier + pickup ledger row + replay re-key, D6 / OQ4, branch `logistics-m3-direction`):** `RouteOrchestrator.EmitLoopCycle` now fires, under ONE `cycleId`, the dispatch-debit half (`RouteDispatched` Seq0 + `RouteCargoDebited` Seq1) → the pickup half (`EmitPickupHalf` calls the Phase 3 `ApplyPickupDebit` and emits ONE new `RouteCargoPickedUp` row at Seq2, after dispatch) → the delivery half. A PURE-pickup route SKIPS the delivery half (no `RouteCargoDelivered` row; previously `EmitLoopCycle` always called `ApplyDelivery`, emitting a delivered row even on an empty plan), gated on `RouteHasDeliveryManifest`; the pure-pickup branch bumps `CompletedCycles` itself so the cycleId advances. The new `RouteCargoPickedUp = 29` GameActionType (appended after `RouteRecoveryCredited = 28`) carries the actual debited manifest + requested-on-shortfall manifest + ENDPOINT pid, and emits ZERO funds (a pickup debits its physical source, never funds): the mirror of `RouteCargoDebited` minus funds. Five integration sites: `GameAction` enum + funds-less serialize/deserialize (route rows ride the existing codec via `Ledger`, so `ParsekScenario` OnSave/OnLoad is unchanged); `LedgerLoadMigration.IsResourceImpactingAction` → false (mirrors `RouteCargoDelivered`); `FundsModule` non-funds (intentionally uncased, falls through and moves no funds); `SupersedeCommit.IsWorldStateChangingRecordingAction` → false (a NEW type does NOT inherit route-rows-false, so the explicit case is required or supersede strict-blocks); `RouteModule.ProcessCargoPickedUp` observe-only (mirror of `ProcessCargoDebited`, mutates nothing, T-ROUTEMODULE-OBSERVE). **The replay re-key (OQ4, the correctness fix):** `EmitLoopCycle`'s per-cycle ELS idempotency guard was keyed on the `RouteCargoDelivered` row (which a pickup-only route never emits), so a pickup-only route's endpoint debit re-applied every save/reload; it is now keyed on the DIRECTION-AGNOSTIC `RouteDispatched` row over `(RouteId, RouteCycleId)`, preserving the `CompletedCycles` bump on the replay branch. LOOP-PATH-ONLY: the legacy `ApplyDelivery`-path delivery-keyed check stays on `RouteCargoDelivered` (dead for loop routes, M1 D11). The stop summary label is now direction-aware ("Pick up (...)" / "Deliver (...) / Pick up (...)"). RESOURCES ONLY; inventory pickup is Phase 5, mixed-window net-zero ordering refinement is Phase 6. **As built (Phase 6 mixed bidirectional window applier ordering, D6 / D3, branch `logistics-m3-direction`):** an audit confirmed the Phase-4/5 applier ALREADY fires the two physical halves in the CORRECT order, so Phase 6 is an explicit-ordering HARDENING (comment + regression tests), not a code rewrite (no behavior delta, so no CHANGELOG line). The MIXED-WINDOW ORDERING RULE, now pinned in code: at a mixed window on ONE endpoint vessel both halves resolve the SAME `stop.Endpoint` ⇒ the SAME physical vessel, so the endpoint DEBIT (pickup, frees tank space) MUST run BEFORE the endpoint CREDIT (delivery, fills the tank). The shipped order (`EmitLoopCycle`: dispatch-debit → pickup half → delivery half) satisfies this: the pickup debit frees capacity first, so `ApplyDelivery`'s live `LiveDeliveryCapacityProbe` + `LiveDeliveryWriters.WriteResourceLoaded` (which read LIVE tank state AFTER the debit and clamp the fill at `free = maxAmount - amount`) see MAXIMUM free space, and a same-resource deliver+pickup nets correctly with NO phantom clamp. The wrong order (delivery-then-pickup) would clamp a delivery into a near-full endpoint, silently waste the surplus, then debit from the post-clamp state - a different, lossy result than the witnessed recording. The rule is replay-deterministic (fixed statement order under one cycleId, idempotent across re-presentation via the Phase-4 dispatch-keyed backstop). Inventory pickup vs inventory delivery act on different identity/resource spaces (STOREDPART slots by `IdentityHash` vs resource tanks), so they rarely collide; the same debit-before-credit order holds and is harmless otherwise. Flow closure already handles a mixed window's BOTH terms - `ComputeFlowClosure` sums `loadedWindows` + `deliveredWindows` per resource name, so the same resource in both lists balances (`slack = launched + Sum(loaded) + harvested - Sum(delivered) - residual`). Pinned by `RouteLoopPickupFireTests.MixedSameResourceWindow_PickupFiresBeforeDelivery` (records both halves' invocation order, asserts pickup-before-delivery) + `RouteAnalysisEngineTests.ComputeFlowClosure_MixedSameResourceWindow_BothTermsBalance` / `..._OverDelivery_Rejects`.
- Multi-origin semantics (19.2.5) — **RESEQUENCED to M4** (2026-06-14, `docs/dev/done/plans/plan-logistics-m3-direction-generality.md` OQ1): true multi-origin (load at depot A + load at depot B then deliver at the station) is N loaded windows, which needs M4's multi-window acceptance (lifting the `MultipleConnectionWindows` reject), so the all-or-nothing source gate + the `RouteStore` cargo-escrow reservation map land in M4 where they are first exercisable. M3 ships the pickup DIRECTION on a SINGLE bidirectional window only.

M4 — Shape generality: multi-stop, then round-trip

The multi-window model, paid for by multi-stop:

- **Multi-stop DELIVERY — SHIPPED (M4a, branch `logistics-m4-multistop`; plan `docs/dev/done/plans/plan-logistics-m4-shape-generality.md`).** A route can deliver to several stops in one run, firing each delivery at its own recorded dock phase. As built: `RouteAnalysisEngine` (both entry points) now accepts EVERY complete `RouteConnectionWindow` and orders them ascending by `DockUT` (`MultipleConnectionWindows` re-purposed to the genuinely-unorderable duplicate/NaN-DockUT case); `RouteAnalysisResult` widened to a per-stop `List<RouteAnalysisStop>`; run-level flow closure sums N windows (residual = last window). `RouteBuilder` builds N ordered `RouteStop`s with per-stop `RecordedDockUT`; the per-window FIRING (plan OQ2 Horn A) fires N windows under ONE cycleId with BOTH replay guards re-keyed to `(RouteId, cycleId, stopIndex)`, per-window `Sequence` uniquified, `PendingStopIndex` driven to the firing window, per-stop `LastFiredCycleIndex` as the fire-once-across-reload sub-gate, the route-level cycle snap deferred until the last dock is reached (so a tick between dock A and dock B fires only window A and resumes window B on a later tick), `CompletedCycles` bumped exactly once per cycle, and a cross-cycle one-dock-cycle-per-pass catch-up loop. The render end-trims at the LAST dock and never spawns a real vessel (the no-spawn pin holds with the widened window); a single-stop / pre-M4 route stays byte-identical (sparse per-stop codec keys, the `deliveryOffsetSeconds=0` placeholder preserved). Mid-recording-START shapes (undock → undock shuttles beginning inside a pre-dock recording) reject as `RouteAnalysisStatus.MidRecordingStartTrimUnsupported = 9`, a documented-limitation rejection reason emitted by a REAL detector since M-MIS-5 P2a (`RouteAnalysisEngine.ClassifyStartTrimShuttle` at both undocked-start gate sites, naming the recognized origin dock UT in the reject detail; unrecognizable undocked starts keep the general status 6; the start-trim ACCEPTANCE shipped as M-MIS-5 P2b 2026-07-07 - the well-formed docked-origin-window start is now Eligible and status 9 names only the family's remaining degenerate shapes). Multi-WINDOW PICKUP (>1 source-debiting window), multi-origin, and cargo escrow are NOT in M4a — they are M4b (next). `Stops` / `SegmentIndexBefore` / `DeliveryOffsetSeconds` were already reserved in the save shape, so the codec cost was small as predicted.
- **Multi-origin (19.2.5) — SHIPPED (M4b, branch `logistics-m4-multistop`; plan `docs/dev/done/plans/plan-logistics-m4-shape-generality.md`).** A run can load from several depots and consolidate the delivery, or load at a refinery then deliver at a station (the shuttle), all in one route. As built: the source set is DERIVED from the route's pickup `Stops` (no new persisted `Route` field — OQ5; each pickup `RouteStop` already carries `Endpoint` + `PickupManifest`). All-or-nothing N-source gating runs in `LiveRouteRuntimeEnvironment.OriginHasCargo` → the pure `RoutePickupSourceGate` (group pickup stops by RESOLVED live pid, SUM the per-pid manifests so same-pid windows under-gate-guard correctly, capture the `loaded && !packed` gate ONCE per vessel, `HasRequired` per pid NET of escrow); the first short source — ordered by the source's earliest dock UT — names the source in the hold token (`source:<pid>:<name>:<resource>`, rendered by `LogisticsHoldPresentation`). The cargo escrow is a RAM-only `RouteStore`-owned reservation map (`routeId -> pid -> resource -> held`, NOT serialized): reserve-at-dispatch (`ReserveCycleEscrow`), debit-at-window-phase + release-per-window (`ReleaseWindowEscrow` inside `EmitPickupHalf`), re-establish for the un-fired windows of an in-flight cycle at the start of every tick, before any route runs and whether or not a window is due (`RouteOrchestrator.ReEstablishInFlightCycleEscrow` → `ReEstablishEscrowForUnfiredWindows`, idempotent on `HasEscrow`; ROUTE-ESCROW-LOST-ON-SCENE-SWITCH), cycle-complete drop (`RouteStore.DropRouteEscrow` in `ProcessMultiStopCrossings` when the last dock is reached), cleared on any within-game scene change (`onGameSceneSwitchRequested`) + game unload (`OnMainMenuTransition`) + route removal (`RemoveRoute`) — DROP-not-revert (the physical debit's revert is the rewind quicksave + ELS replay keys; escrow has no ledger row). A competing higher-priority route's gate nets the reservation via `RouteStore.OtherRoutesReservedFor` (a route never subtracts its OWN reservation), so it cannot drain a reserved depot mid-run. The per-window SOURCE debit reuses M3a's `ApplyPickupDebit` per window under the windowed replay key (per-stop `Sequence` stride + the C-2 `RouteCargoPickedUp` row backstop). Funds: loaded-en-route cargo debits its physical source, NEVER funds (19.2.4); the launch-manifest funds basis is untouched. The single docked-origin (`Route.Origin`) provenance and the per-stop pickup-source provenance gate INDEPENDENTLY (distinct launched / loaded-en-route provenances, no double-count). The pure RAM escrow dict passes the ERS/ELS grep gate. INVENTORY escrow is deferred (window-local) — the B3 seam only reserves resource manifests; a multi-window inventory consolidation would add the inventory-escrow analogue. Byte-identity is trivial (no new persisted field). What remains out of M4b: round-trip linking (NEEDS M4c).
- **Round-trip linking — SHIPPED (M4c, branch `logistics-m4-multistop`; plan `docs/dev/done/plans/plan-logistics-m4-shape-generality.md`).** Two one-way routes linked as a round-trip pair alternate (A completes a run, then B dispatches, then B completes, then A) so only one transport's worth of effects is in flight at a time, modeling a single reused vehicle (5.4 the Routine-Mission-Manager profile). As built (the thin `LinkedRouteId` chain-constraint scheduler predicted by the resequencing note in 17.1, NOT a system): a new `RouteDispatchEvaluator.EligibilityFailureKind.WaitingForPartner` (an evaluator enum value, NOT a new `RouteStatus` — OQ8; appended LAST so the three `RouteStatusPolicy` throw-on-default switches are untouched and a waiting route stays `Active` / `GhostDriving` / renders its loop, the wait only naming the hold reason); a partner gate in `CheckEligibility` ordered LAST (after sources / endpoint / cargo / funds / capacity, so a genuinely-blocked route surfaces its REAL blocker first), resolving the partner via `RouteStore.TryGetRoute` (the same static seam M4b used to read escrow — NO new `IRouteRuntimeEnvironment` member, so the ~14 env fakes are untouched, RANK-10), holding `WaitingForPartner` while `partner.CompletedCycles <= route.LastConsumedPartnerCycle`; a new sparse scalar `Route.LastConsumedPartnerCycle` (the alternation cursor — codec-omitted at the 0 default, so an unlinked / pre-M4c route round-trips byte-identically; advanced to the partner's current `CompletedCycles` on a genuine dispatch in both firing paths via

`RouteOrchestrator.AdvancePartnerAlternationOnDispatch`, NOT on the ELS-replay backstop, NOT reset on Activate / cadence-rebase since it is chain state not per-cycle fire state); a bypass of the constraint when the partner is missing / Paused / Broken (`!RouteStatusPolicy.GhostDriving(partner.Status)` plus an unresolved partner — design 10.14, since those partners never advance `CompletedCycles` and holding on them would stall the route forever); a deadlock seed for a mutual A↔B link (the SEED — lower `DispatchPriority`, then ordinal `Id` via `IsChainSeed`, a stable structural property omitting the schedule-state middle key — dispatches its FIRST cycle, scoped to both routes at `CompletedCycles == 0` so it fires exactly once); and an M6 hold reason ("waiting for the linked route '{partner}' to complete its run", `LogisticsHoldPresentation.DescribeHold`). Byte-identity is trivial (one sparse scalar, omitted at default); no `RouteConnectionWindow` field, no proof-hash change. With round-trip done, **M4 (shape generality — multi-stop + multi-origin + round-trip) is COMPLETE.**

- Missions-boundary verification result (**SHIPPED, M4a**): in always-tree mode a dock ALWAYS splits recordings (the dock-merged child is a separate recording), so multi-stop END-trims — render `[launch..last dock]`, skipping docked stretches at A — are expressible with today's interval boundaries; `RouteBackingMission.ComputeExcludedIntervalKeys` already generalized (the caller passes the LAST stop's dock UT, the single-UT exclusion drops everything at/after it, and the terminal-undock-child scoping keeps the intermediate-undock survivor rendered) — VERIFY-ONLY, no route-side code change. The Missions locked layer was consumed READ-ONLY (its git diff is EMPTY). What remains out: shapes that START mid-recording ("undock → undock" shuttle runs whose run begins inside a pre-dock recording) hit the locked Missions layer's gap 1 (dock is not an interval boundary INSIDE a recording; `design-mission-abstractions.md` "Docking & undocking (v1)"). That was a documented limitation surfaced in the `MidRecordingStartTrimUnsupported = 9` rejection reason, NOT a Missions edit; the lift shipped as M-MIS-5 (P1 dock edges 2026-07-04; P2b start-trim acceptance 2026-07-07).
- Regression pin from 19.2.3 (**SHIPPED, M4a**): a pure xUnit + an in-game pin assert route-driven loop units never produce a real-vessel spawn decision now that the rendered window widens past the first dock — every kept member (including the intermediate-undock survivor + the last dock's combined leg) is a loop-unit member (loops, never spawns), and every recording at/after the last dock is a non-member.

M5 - Reach: inter-body route activation - BUILT (2026-07-04, branch `logistics-m5-interbody`; plan `docs/dev/done/plans/plan-logistics-m5-interbody.md`). GATE PASSED IN-GAME 2026-07-08 (automated builder-shape gate green: basis=ReaimWindows, residual N modulo fires/skips purely; real-save M-MIS-2 P4 confidence tests green); merge hold lifted, PR #1238.

As built: a route whose backing Mission carries a Missions-layer window schedule dispatches on the SAME launch UTs the ghost renders on, and `CadenceMultiplier` `N` applies as a RESIDUAL modulo on the window index - the one route-side cadence rule, with the residual depending on which mechanism spaces the rendered windows. The basis (`RouteWindowBasis { FlatInterval, ZeroDriftSchedule, ReaimWindows }`, `RouteLoopClock.DeriveWindowBasis`) is derived per tick from the resolved unit and never persisted:

- **FlatInterval** (no schedule - incl. a cross-parent build whose re-aim DECLINED, the honest faithful fallback): modulo OFF. `N` is fully consumed by the flat cadence `DispatchInterval = N x TransitDuration`; v0 behavior, byte-identical (pinned by `Serialize_SingleStop_ByteIdenticalToBaseline`).
- **ZeroDriftSchedule** (same-parent zero-drift `MissionRelaunchSchedule`): residual = 1. `N` is ALREADY consumed Missions-side - the builder's `minSpacing = cadence = DispatchInterval` throttles the schedule generator, and `sIdx` indexes the THROTTLED launch list - so delivery fires on EVERY rendered launch and ghost/delivery stay in lock-step at every `N`. CORRECTION to this section's original sentence ("only the route-side modulo on that index"): a route-side modulo on `sIdx` would apply `N` TWICE on this path; the modulo is route-side only, but its residual here is 1. Granularity is intended, do not "fix" it: at `N=2` the schedule places "the next faithful window after 2 x interval", not exactly every 2nd window of the `N=1` schedule.
- **ReaimWindows** (cross-parent re-aim: supported plan + valid synodic schedule): residual = `N`. The build discards `DispatchInterval` (`effectiveCadence = max(span, synodic)`), the ghost renders EVERY synodic window, and delivery fires on every `N` th rendered window, anchored at the first fired window (`Route.WindowAnchorCycleIndex`, sparse; the first crossing after creation / activation / rebase always delivers and re-anchors). A modulo-SKIPPED window advances the observation marker ONLY - no `SkippedCycles` bump, no hold, no ledger rows, no escrow, no partner-alternation advance; the prior cycle's pending recovery credit flushes on it - the ghost still flies ("world looks busy, transfers nothing"). Warp fires once for the highest deliverable window in the owed range. Multi-stop: the modulo decision is cycle-level on the lowest owed cycle, before the dispatch guard / escrow reserve, snapping every stop atomically on a skip.

Basis flips are re-baselined fail-closed (`Route.ReaimWindowBasisEngaged`, the persisted flip detector): a DECLINE re-baselines the cycle cursors to the current index in the new (flat) space with no fire that tick (the mis-fire guard); a re-ENGAGE re-baselines them into window space with exactly the current window owed (the permanent-silent-skip guard - a stale flat-space cursor would exceed every future synodic index forever). A cadence edit under `ReaimWindows` re-baselines to the current dock cycle instead of the flat -1 reset (no duplicate delivery of the already-delivered window under a fresh cycleId). UI: a basis label after the cadence ("(Duna transfer)" / "(launch window schedule)"), windowed stepper wording ("2x (every 2nd window)"), and a "Next launch window" T-countdown computed off `unit.PhaseAnchorUT` / `unit.CadenceSeconds` only (never raw `ReaimSchedule` fields - pad-align and loiter compression shift the unit anchor away from them). No Missions file changed; `CycleIndexKind` (a re-aim unit's runtime index deliberately classifies Flat - it doubles as the synodic window index) is complemented by the route-side basis, not widened. This completes "any route the player can fly, the network can run", pending the playtests above.

M6 — "Simple and it works": legibility (parallel track) — COMPLETE (2026-07-07)

Half of "no matter what the player does, it should work" is never leaving the player staring at a silent route. Every M6 item below is SHIPPED or explicitly closed by decision: hold reasons (row-level treatment included), structure list window, candidate intent helper, the Record Supply Run prompt (the section 17 helper, below), map-view route lines, and the per-cycle flow display; the precise per-run recovery landing is CLOSED won't-build by maintainer decision (2026-07-07, per `docs/dev/done/research/logistics-recovery-clock-memo.md`) - the constant deferred recovery credit is the ratified permanent behavior.

- **Every non-dispatching route says why, in player language, in the Logistics window.** Partially shipped: candidate near-misses surface reasons (the `LogisticsRejectPresentation` near-miss list), and LIVE route hold states now name theirs too (as built, 2026-06-11: the orchestrator persists the last hold kind / raw reason token / funds shortfall / UT on the `Route` at the loop-path blocked crossing and the legacy wait / endpoint-lost appliers, clearing on an eligible crossing, a legacy dispatch, or a player Activate; the pure `LogisticsHoldPresentation` renders it as a yellow detail-panel line with a "checked {age} ago" suffix plus a status-cell tooltip clause, built in the ~1 Hz legibility cache). Flow-closure rejections naming the unaccounted quantity per 19.2.4 SHIPPED with M3 and are VERIFIED end-to-end (2026-07-04): `ComputeFlowClosure` names the offending resource + amount ("Ore: 30.0 over-delivered") in `RejectDetail`, which threads `RouteAnalysisResult` → `RouteNearMiss` → `LogisticsRejectPresentation.DescribeNearMiss` → `RouteCreationFormatters.FormatRejectMessage` into the near-miss row text; pinned by `LogisticsRejectPresentationTests.DeriveNearMisses_OverDeliveryTree_TextNamesResourceAndQuantity` (the run-level closure is a whole-run balance, so the naming is resource + quantity, not a per-window attribution). The escrow-hold gap from the M4 UI-coverage audit - a pickup-source hold caused ONLY by a competing route's cargo escrow rendered identically to a physically-empty depot ("X is out of Y" when the cargo existed but was reserved) - is SHIPPED (2026-07-04, branch `logistics-m6-escrow-hold-naming`): the pickup-source gate classifies physical-short vs escrow-short from the raw vs escrow-netted availability (`RoutePickupSourceGate.IsEscrowCausedShort`), an escrow-caused short emits a `source-reserved:<pid>:<name>:<resource>:<routeName>` token naming the competing route with the largest reservation (`RouteStore.TryGetReservingRoute`, pure RAM, own-route excluded), and `LogisticsHoldPresentation` renders it as " has reserved by route '"; genuine physical shorts (including mixed shorts where the depot is short even before netting, and every inventory short) keep the byte-identical `source:` token and text, and an escrow-caused short whose reservation vanished before the lookup falls back to the physical token. The row-level visible treatment SHIPPED (2026-07-07, branch `logistics-m6-closeout`): while a displayable hold is recorded, the Status cell itself carries a compact specific reason ("Held: Depot A out of Ore", "Held: waiting for 'Return Run'") instead of the generic per-status sentence - the pure `LogisticsHoldPresentation.StatusCellText` / `CompactHold` formatters cover the same total kind+token table as `DescribeHold` (funds, source:, source-reserved:, inventory:, WaitingForPartner, endpoint-lost, fallback), hard-truncated to 60 chars with the full clause still in the tooltip and detail panel; built in the same ~1 Hz legibility cache (no new per-frame work, no new hold computation - pure presentation of the persisted `Route.LastHold*` fields), the Status sort key follows the visible text, and a held route reads yellow "Held: ..." in either table. With this, every non-dispatching route names its reason AT ROW LEVEL, and the M6 headline item is COMPLETE.
- Structure list window (old Tier 1; reads already-recorded data — `TrackSections` / `SegmentEvents` / `PartEvents` / `RouteConnectionWindow` / `Stops`).
- Candidate intent helper (per-tree flag consumed by `RouteCandidateFinder`) - SHIPPED (2026-07-04). As built: the flag lives in the consuming logistics domain as a RouteStore-owned persisted set of dismissed candidate tree ids (sparse `DISMISSED_ROUTE_CANDIDATES` sibling of the ROUTES save node; the RecordingTree schema untouched), consumed by `RouteCandidateFinder.DeriveCandidates` / `DeriveNearMisses` so a dismissed tree leaves the whole Candidates section; Dismiss controls on candidate and near-miss rows, restore any time from the collapsed "Dismissed (N)" subsection, stale ids swept at load. The COMPANION "Record Supply Run" prompt (section 17's "v1 should automatically prompt after eligible committed runs") SHIPPED 2026-07-07 (branch `logistics-m6-closeout`): when a tree commit produces a tree `RouteCandidateFinder.DeriveCandidates` classifies as an ELIGIBLE candidate (single-tree finder pass: fully sealed + analysis-eligible + not already promoted + not dismissed; a near-miss, including the common not-yet-sealed shape, never prompts and never burns the shot), a one-time NON-BLOCKING prompt fires - a ScreenMessage announces it and the main Parsek window shows a banner with Open Logistics / Dismiss actions plus a cyan Logistics-button highlight (deliberately NOT a modal: the commit-time "Create Supply Route?" modal was removed in the three-section window rework for interrupting gameplay). At-most-once per tree, ever, via a RouteStore-persisted prompted-tree-id set (sparse `PROMPTED_ROUTE_CANDIDATES` sibling of the dismissed node, same codec/sweep/byte-identity contract; NOT a RecordingTree schema field); banner Dismiss also adds to the dismissed set; no prompt during in-game test batches (`ActiveTestBatchMarker`) or restore windows (`restoringActiveTree` / committed-tree restore attempts). Hooked at both commit funnels (`MergeDialog.MergeCommit` and `ParsekFlight.CommitTreeFlight`, the latter covering the pre-switch-dialog patch paths); pure decision predicate `RouteRunPrompt.ShouldPrompt` is xUnit-pinned.
- Precise per-run recovery landing (second recovery clock keyed on recorded recovery UT, replacing the constant deferred credit; recovery-credit plan OQ1). **CLOSED - won't-build (maintainer decision 2026-07-07)**, per the recommendation in `docs/dev/done/research/logistics-recovery-clock-memo.md` (divergence is bounded per-cycle timing lead/lag with zero total error; the divergence harness `RouteRecoveryLandingHarnessTests` pins the gap with numbers and its skipped ideal-model test

remains the acceptance fixture if this is ever reopened). The constant deferred recovery credit is the ratified PERMANENT behavior, not a stopgap.

- **Map-view route lines: SHIPPED (branch `logistics-m6-map-route-lines`).** The section 17 "Map view integration" deferral (never in the old tier list). Each committed SAME-BODY route draws its backing recorded legs (launch → dock) as a persistent line on the flight map and the Tracking Station, so the player sees WHERE a route runs, not just its panel. As built: a new `Source/Parsek/Display/RouteTrajectoryLineRenderer.cs` rides the existing `GhostTrajectoryPolylineRenderer` machinery (reuses the pure `BuildLegsForRecording` leg builder + the `TryDrawLeg` draw helper, so route lines match the ghost trajectory line's stock-orbit grey / width / material rather than inventing a new style), keeps its own per-route cache + VectorLines (signature-gated on member content hashes + dock UT + same-body flag), reads route + recording data through existing accessors only (`RouteStore.CommittedRoutes`, `RecordingStore.TryFindCommittedRecordingById`; no `Route` / `RouteStore` fields added), clips legs to `Route.RecordedDockUT`, and draws statically (no head-UT gate) behind the `showRouteLines` setting (default on, persisted). It draws in an independent map-camera onPreCull slot after the ghost draw and skips any backing recording the ghost polyline is drawing this frame (`IsRenderingNonOrbitalLeg`), so it never double-draws the ghost's own trajectory, and it publishes no ownership signal. FOLLOW-UP STATUS: inter-body route lines SHIPPED 2026-07-08 (branch `claude/logistics-interbody-route-lines-7phdtw`, originally stacked on M5 PR #1238; gate automated via the in-game `RouteLine_InterBodyDrawsEndpointLegsDropsTransferLeg` test) - endpoint-body legs only, transfer-frame legs dropped at build (`FilterLegsToEndpointBodies`), the re-aimed transfer stays with the backing mission's ghost render, schedule folded into the cache signature, same-body behavior byte-identical; see section 17. STILL DEFERRED: the static orbital-coast overview (parking/coast arc) stays head-gated on the stock conic, so route lines draw the recorded non-orbital legs only.
- **Per-cycle flow display — SHIPPED (branch `logistics-m6-percycle-flow`).** What each cycle debits where and delivers where — the visibility that makes the crashed-disposition bleed of 19.2.5 a player choice rather than a surprise. As built: the route DETAIL panel (not the row) gains a "Recent cycles:" block — one compact newest-first line per completed cycle, bounded to the last 5 (`LogisticsFlowPresentation.MaxCyclesShown`), e.g. "Cycle 3 (2.1h ago): paid 500 funds at KSC; picked up 20.0 Ore from Mining Rig; delivered 150.0 LiquidFuel to Munar Station", shortfall cycles tinted yellow with the short segment annotated ("origin was short" / "source was short" / "some cargo did not fit"). Data is READ-ONLY from the ledger rows the routes already write (`RouteCargoDebited` / `RouteCargoPickedUp` / `RouteCargoDelivered`, grouped by `RouteCycleId`, multi-stop rows aggregated under one cycle line in (UT, Sequence) firing order) — no new ledger fields or rows, and the collection extends the SAME single `ComputeELS()` walk the H2/H3 delivery summary already does (`LogisticsFlowPresentation.CollectRows`), built on the ~1 Hz legibility cache, never on the IMGUI draw path. A KSC funds debit renders the charge and skips the informational cost manifest (no phantom "took from KSC" double-count); endpoint pids resolve to live vessel names with a "vessel pid=N" fallback for vanished vessels; delivery destinations resolve per stop by baked pid with recorded-coords fallback. Zero completed cycles renders nothing (no empty header).

Milestone → old tier mapping

Old tier item	Milestone
Dispatch priority	M1 (pulled forward)
Scenario-lifecycle test harness (Tier 4)	M1 (pulled forward)
Non-KSC origin debit (docked)	M1
Non-KSC origin debit (undocked-start prover)	RETIRED (19.2.2)
Pickup routes; mixed windows	M3
Multi-stop; round-trip linking	M4
Inter-body re-aimed routes	M5
Structure list window; candidate intent helper; precise recovery landing	M6
Map view integration (§17 deferral, not in the tier list)	M6
Claw/grapple connection producer	SHIPPED 2026-07-07 (`docs/dev/design-logistics-claw-producer.md`)
Crossfeed/fuel-line producers; crew delivery	Out of scope by doctrine (revisit on demand)
"Dispatch now"	Already RETIRED (subsumed by Send Once), unchanged
Dock-side-baseline edge case	Already SHIPPED in 0.10.1, unchanged

19.5 Completeness test

After M5, "complete" has a concrete form: any committed recording whose flows close (19.2.4) becomes a route (any provenance mix, any number of windows in either direction, any stops, any bodies, any defined resources, across BOTH admitted connection producers: docking port and claw/grapple) and any recording whose flows do not close is rejected with the specific unaccounted window named. Crossfeed/fuel-line producers, crew delivery, persisting-transport materialization, and undocked-start origin proving remain out by doctrine, each with a stated reason, not by accident.

19.6 Doctrine summary (the rules that keep this small)

1. **One validity rule:** flow closure (19.2.4), not a profile enumeration.
2. **Origin is causal:** provenance is the witnessed event, never the start position (19.2.1–19.2.2).
3. **Undocked-start is a workflow error,** not a feature gap (19.2.2).
4. **Route transports never materialize** (19.2.3).
5. **All-or-nothing at the sources, clamp at the destination** (19.2.5).
6. **Reserve at dispatch, debit at window phase;** a crashed run still debits (19.2.5).
7. **Any defined resource routes;** background production is explicitly not logistics' problem (M2).
8. **The locked Missions layer owns periodicity;** logistics activates seams, never edits Missions (M5).
9. **Every non-dispatching route names its reason in player language** (M6).
10. **Connection producers are classified, never inferred:** a window is stamped by the producer that made it (docking port or claw), unknown producers reject by name (`UnsupportedConnectionKind`), and every admitted producer passes the same flow-closure rule. Rule 4 reads unchanged with Grapple included: a grapple transport is still a ghost re-run. Continuous-flow paths (crossfeed/fuel-line) have no couple event and no window shape, so they stay out rather than getting a weaker evidence rule.

19.7 Verification results (2026-06-10)

The draft of this section carried ten `VERIFY` items; all were checked against the code (post-0.10.1 main). Verdicts, with the corrections folded into the body above:

1. **Harvest bracketing (19.2.2):** NOT covered by existing snapshots. `RouteOriginProof` start/end transport manifests exist only for start-docked runs (`FlightRecorder.CaptureStartRouteOriginProofIfDocked` → `RouteProofCapture.BuildStartRouteOriginProof`); no ISRU module activity is recorded anywhere in the recorder. M2 adds full-run manifests for all runs + a harvest attribution capture. Post-merge addendum (2026-06-10): attribution scope corrected — asteroid/comet mining uses `ModuleAsteroidDrill` / `ModuleCometDrill` , separate modules from `ModuleResourceHarvester` , so the capture keys on `BaseConverter` -derived activity (19.2.2 item 3).
2. **Rejection guidance (19.2.2):** CONFIRMED. `RouteAnalysisStatus` → `RouteCreationFormatters.FormatRejectMessage` (`Source/Parsek/Logistics/RouteCreationFormatters.cs`) → `LogisticsRejectPresentation.DescribeNearMiss` → the Logistics window near-miss list. New reasons are one enum value + formatter text.
3. **No-spawn (19.2.3):** CONFIRMED, architecturally. Route-excluded recordings never become `LoopUnit` members (`RouteBackingMission.ComputeExcludedIntervalKeys` → `MissionIntervalSelection.ComputeRenderWindows`), so they never reach a playback completion that feeds `GhostPlaybackLogic.ShouldSpawnAtRecordingEnd` . No route-aware spawn gate exists or is needed; M4 adds a regression pin.
4. **Cost restatement (19.2.4):** CONFIRMED in conclusion, corrected in mechanism. The funds charge (`RouteOrchestrator.ComputeDispatchFundsCostForRoute` → `RouteFundsCalculator.ComputeDispatchFundsCost`) walks the FIRST source recording's vessel snapshot, captured at that recording's stop — the dock chain boundary for a `[launch..dock]` run — so the resource term is dock-arrival contents (launch load minus pre-dock burn), not the launch manifest. The no-double-charge conclusion holds: later-window loaded cargo is structurally outside the charge. Corrections: the shipped `CostManifest` is a copy of the window delivery manifest (`RouteBuilder`), not the §5.2 full-run decrease, and §5.2's "parts plus used/delivered" funds wording overstates the resource term; flow accounting replaces `CostManifest` with per-provenance terms and decides the charge basis in M2. (§5.2 now carries the matching implemented-v0 note.)
5. **Gating order (19.2.5):** Reconciled against the IMPLEMENTED order — `RouteDispatchEvaluator.CheckEligibility` : sources → endpoint → origin cargo → funds → destination capacity. Zero-capacity destination blocks the cycle; partial capacity clamps per-resource at delivery (`RouteDeliveryPlanner.PrepareDelivery`). All-or-nothing source gating extends the origin step; destination behavior unchanged. (§6.1's destination-before-origin step order is the v0.5 design intent; §6.1 now carries the implemented-order note.)
6. **Escrow host (19.2.5):** Lightweight `RouteStore` -owned per-vessel reservation map. The ledger reservation machinery is funds-specific (`FundsModule` balance projection) and not worth generalizing for one consumer. Also verified: the loop path debits at the dock-phase crossing (`RouteOrchestrator.EmitLoopCycle`), so escrow only becomes live with multi-window runs. **SHIPPED (M4b):** the RAM-only `cargoEscrow` (`routeId -> pid -> resource -> held`) with reserve / release / drop / clear /

`OtherRoutesReservedFor` net, wired at dispatch / per-window debit / cycle-complete / scene-change / unload / route-removal, and re-established on an in-flight resume.

7. **Recovery credit values cargo (19.3 scenario 5):** CONFIRMED. The credit is the recording's trajectory-point funds delta at recovery (`LedgerOrchestrator` recovery pairing; summed by `RouteRunCostCalculator.SumRecoveredCredits`), i.e. KSP's stock recovery valuation, which includes onboard resources.
8. **Stock-name audit (M2):** PASSES. Only `ElectricCharge` / `IntakeAir` are hardcoded, as intentional exclusions; manifests are name-keyed; `LookupResourceUnitCost` resolves any `PartResourceLibrary` definition (modded included), 0 for undefined.
9. **Missions dock-boundary gap (M4):** End-trims `[launch..dockN]` are expressible today (docks always split recordings in always-tree mode); mid-recording START-trims (undock→undock shuttle shapes) hit gap 1 and were a documented limitation until M-MIS-5 (P1 dock edges + P2b start-trim acceptance, SHIPPED 2026-07-07).
10. **Name cross-check:** All symbols verified as named, including `OriginHasCargo` + `non-ksc-origin-unsupported-in-v0` , `StartDockedOriginVesselPid` , `HasResourcePickup` , `TrySendOneCycleNow` , `CadenceMultiplier` / `ApplyMultiplier` , `Stops` / `SegmentIndexBefore` / `DeliveryOffsetSeconds` / `LinkedRouteId` , the `RouteLoopClock` → `TryComputeSpanLoopUT` schedule passthrough, `RouteRecoveryCredited` / `RouteDispatched` / `RouteCargoDebited` / `RouteCargoDelivered` , and the `RouteStatus` enum (matches §4.5). One naming correction: the connection window carries `DockUT` / `UndockUT` ; `RecordedDockUT` lives on `Route` . One behavioral correction: competing-route ordering is `RouteStore` commit-list order, not FIFO-by- `NextDispatchUT` as 10.12/10.13's v0.5 wording — folded into M1's dispatch-priority rationale and noted in 10.12/10.13.

M1 implementation addendum (2026-06-10): M1 shipped (0.10.2-dev, branch `logistics-m1-origin-debit` ; per-item as-built notes in 19.4). One verification-grade correction surfaced during implementation and is folded into 19.4 item 1: the physical `RouteCargoDebited` debit does NOT apply through `RouteModule` (observe-only, T-ROUTEMODULE-OBSERVE intact); it is a direct write at emit time inside `RouteOrchestrator.EmitDispatchDebit` , loop-path-only (plan decision D11), with the row as bookkeeping and revert safety via the rewind quicksave + ELS replay keys. Inventory-payload origin debit is deferred to M3 (decision D6).

M2 implementation addendum (2026-06-12): M2 shipped (0.10.2-dev, branch `logistics-m2-resource-generality` ; per-item as-built notes in 19.4, plan `docs/dev/done/plans/plan-logistics-m2-resource-generality.md` with three review rounds folded in). Verification-grade corrections folded into the body: item 1's "full-run manifests" landed as a NEW presence-gated `RouteRunCargoManifest` rather than promoting the display-grade Phase-11 `StartResources` / `EndResources` (old recordings analyze byte-identically); item 4's funds wording is tightened - the charge walks `SourceRefs[0]` , the tree ROOT, whose snapshot is taken at the FIRST chain/branch boundary, so "dock-arrival contents" holds exactly for single-leg runs; and the 19.2.4 charge-basis question is DECIDED (launch manifest when a complete run manifest exists, stop-snapshot walk otherwise; existing routes keep their exact cost). Honesty notes carried with the flip: run manifests complete only on ACTIVE stops (ForceStop / background legs degrade that tree to legacy analysis, never a false rejection), and the route side needed more than the milestone framing implied (harvest-origin routes D7 + the depot-debit reduction D8, both shipped).

Appendix A: Gameplay Scenarios (from Step 2)

A.1 Fuel Delivery Rover

Base with empty fuel tank near KSC. Rover drives to base, docks, transfers 150 LF through stock resource transfer, undocks, drives clear. Route: 150 LF per cycle, KSC origin with Career dispatch cost, interval = recording duration.

A.2 Orbital Monoprop Resupply

Kerbin station at 100km. Capsule from KSC: launch, dock, transfer 650 MP, undock, deorbit. Ghost visible during launch and station approach (RELATIVE frame). Transit invisible.

A.3 Minmus Ore Delivery (chained routes)

Route A: KSC → Minmus base (fuel). Route B: Minmus base → Kerbin depot (ore). B is gated by resource availability at Minmus. A feeds B.

A.4 Eeloo Supply Run (interplanetary)

Dispatch at Kerbin-Eeloo synodic period (~1.9 years), phase-anchored to the original Supply Run start UT. Gravity assists use the same two-body approximation. Player can increase spacing but v1 does not shift the phase anchor.

A.5 Failure Cases

Destination destroyed (surface: nearest compatible fallback; orbital: route pauses). Origin empty (delayed). Destination full (waiting, no deduction). Source recording missing or changed (route disabled). Revert (route ledger epoch isolation). Time warp (sequential processing). Transport still docked (validation rejects).

Appendix B: Reference Documents

- `docs/dev/done/plans/phase-11-resource-snapshots.md` — archived Phase 11 plan (base resource snapshots; older route-recording workflow superseded here)
- `docs/dev/done/research/logistics-network-design.md` — logistics network research
- `docs/dev/done/research/loop-playback-and-logistics.md` — loop mechanics and orbital drift
- `docs/dev/done/research/resource-snapshots-preparation.md` — infrastructure analysis and unloaded vessel modification
- `docs/mods-references/` — background resource processing patterns from other KSP mods
- `docs/roadmap.md` — Phase 11, 11.5, 12